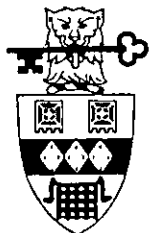


VECTOR

APL89: New York, New York ...plus:

- * 16-page Educational Supplement
- * Two reviews of IBM's APL2/PC
- * Ziemann with the latest from the ISO
- * Joseph de Kerf on APL Punctuation



*The Journal of the
British APL Association*

ISSN 0955-1433

Vol.6 No.2 October 1989

A Specialist Group of the British Computer Society

Contributions

All contributions to VECTOR may be sent to the Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (ideally with a photograph of the author and a brief biographical note). Unless otherwise specified, each item will be considered for publication as a personal statement by the author.

Please supply as much material as possible in machine-readable form, ideally on an IBM PC compatible (DSDD) diskette. APL code should be kept separate from the text, and can be accepted as camera-ready copy, or as APL*PLUS/PC workspaces or IBM .AIO files.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the editor of your intention to re-use material from VECTOR.

Membership Rates 1990

Category	Fee	Vectors	Passes
Non-voting Student	£5	1	1
UK Private	£10	1	1
Overseas Private	£18	1	1
(Supplement for Airmail)	£8		
UK Corporate Membership	£85	10+	5
Overseas Corporate	£140	10+	
Sustaining	£360	50+	5

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Treasurer on the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per full page, £125 for half-page or less (There is a £50 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary

Advertisements should be booked with, and sent to, Alison Chatterton, whose address is given with the Index to Advertisers.

CONTENTS

EDITORIAL:	Adrian Smith	Page 3
APL NEWS		
Quick Reference Diary		5
General Correspondence		7
British APL Association News	Graham Parkhouse	11
News from Sustaining Members	Alison Chatterton	13
The Education VECTOR	Alan Sykes	17
REVIEWS		
APL Product Guide	Alison Chatterton	33
New Product Announcements	Adrian Smith	44
IBM's APL2 on the PC (16-bit)	Dick Bowman	45
IBM's APL2 on the PC (32-bit)	Nicholas Small	52
RECENT MEETINGS: APL89 in New York City		
Introduction	Adrian Smith	59
Conference Roundup	Anthony Camacho	60
Statistics Tutorial Day	Chris Brady	65
Conference Summary	Peter Merritt	66
Notes on the Papers	Camacho, Searle et al	70
Tutorial Sessions	Jonathan Martin	101
The Exhibition	photos by David Ziemann	106
The Bonfire of the Variables	John Searle	108
APL90 Competition	Jill Moss	110
TECHNICAL SECTION		
Hackers Corner: 256 VGA Colours	Adrian Smith	114
Technical Correspondence		115
APL Standards Update	David Ziemann	118
Punctuation in APL	Joseph De Kerf	123
Typesetting APL	Adrian Smith	140
British APL Association Software Library		
Order Form		142
Index to Advertisers		143

APL Analyst – Programmers

SWINDON

LARGE IBM SITE

£16 – 23K PACKAGE

Is this you?

- Minimum 12 months APL or APL* Plus experience
- Excellent communication and analytical skills.
- Intelligent, flexible, self-motivated

Reader's
Digest



If so, consider the benefits of joining Reader's Digest at their brand new data centre near Swindon. The company's direct mail marketing operation relies heavily on models written in APL* Plus (running under MVS/XA on an IBM 3090) to provide accurate projections of product sales, workloads, and operating costs.

The successful candidates will work closely with the end users in marketing and Finance, taking projects from instigation through to implementation and beyond. There will be pressure, variety, and challenge, as well as the satisfaction of working in one of the most dynamic installations in the country.

The positions will initially be based in Central London until the move to Swindon takes place early next year. The new site is close to the M4 (J16) and offers a superb working environment including social and sports amenities.

Comprehensive relocation packages are offered in appropriate cases, as are travelling expenses to London.

An excellent benefits package also includes annual bonuses, BUPA, and the usual large company benefits.

For an initial discussion, please call David Morris on the numbers below, or send a full CV to him at McCourt Consultants.

Office Hours (0734) 509441.

Other Hours (0703) 846132.

MCL
MCCOURT CONSULTANTS LTD
Recruitment Consultancy:
66-68 St. Mary's Buns, Reading, Berkshire. RG1 2LG
Telephone / Fax Reading (0734) 509441

Editorial: We are not Alone!

by Adrian Smith

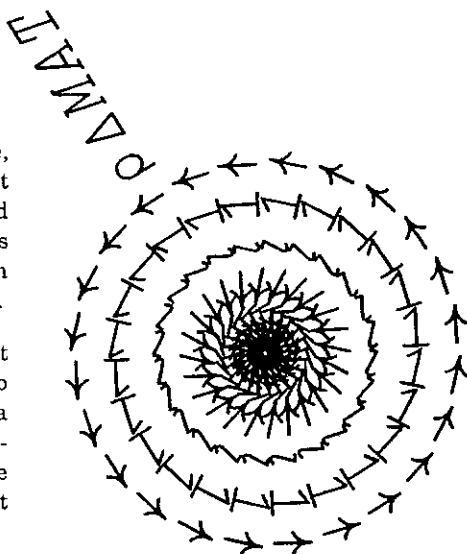
One of the highlights of the recent APL in Statistics conference was a splendid rendering of "APL is a very funny language" by the Alan Sykes Trio on the second evening. (Full lyrics to be published in Vector 6.3 unless Alan gets to them first!) I suppose we ought to admit that there is something a bit unusual about APL, but perhaps we are not as far from the mainstream as we thought?

I have recently been experimenting with a very well-established programming language, among whose attributes are numbered:

- general arrays. Arrays may be made up of numbers, text, and other arrays. There is a language primitive to automate the processing of each element of an array.
- direct definition. There is a language primitive which takes as arguments a function name, and a text array of executable code.
- no predefined hierarchy of arithmetic functions. Incidentally these convert automatically between integer and floating point as required. The user has no control over (or indeed knowledge of) data types.
- a workspace in which defined functions and data arrays may be saved, and an execution stack where temporary variables may be kept.

The parallels are really rather remarkable, and I could keep going for some time about the benefits of straightforward recursion and the interpretive environment! However as you probably haven't yet guessed what I'm on about, the language in question is PostScript.

The example to the right takes about eight very simple lines of coding, and serves to demonstrate rather nicely the power of a language whose sole object is the manipulation of graphics and text. Oh, and by the way, it does it all with this very funny concept of matrices ... now who would credit that?!





for 386 PCs under MS-DOS

Dyalog APL for 386 PCs under MS-DOS is a complete implementation of Dyadic's popular APL for Unix systems, AND fully exploits the power of a DOS 386-PC.

JUST LOOK AT THE FEATURES . . .

- *An exciting new Screen Manager in which the contents of the screen are 'shared' with a nested system variable $\square$ SM (Screen Map).
Facilities include:*
 - Pop-up windows by localisation of $\square$ SM
 - Scrolling (vertical, horizontal, linked)
 - GENUINE numeric fields
 - Date fields
 - NESTED forms
- Fully integrated window-based Session Manager, Editor and Debugging Environment.
- Compatible with the Draft ISO standard for APL
- Huge Workspaces (16Mb and MORE!)
- Virtual Memory Support (using on-chip demand paging)
- Full 32-bit processing speed
- *APL2 features including:*
 - Nested Arrays
 - Defined Operators
 - Selective Specification
- *Commercial features including:*
 - $\square$ FMT
 - Component Files
 - Error Handling
 - "AP124-like" Screen Manager
- External Variables
- Interface to GSS*CGI for fast device-independent graphics
- C Interface

. . . THE BENEFITS SPEAK FOR THEMSELVES

Requirements: 386/486-based PC or PS/2, 640K DOS Memory plus 2Mb RAM, EGA or VGA (colour STRONGLY recommended), 80387 coprocessor STRONGLY recommended, DOS 3.3 or later.

For further information contact:
Dyadic Systems Limited

Park House, The High Street, Alton, Hampshire, GU34 1EN
Telephone: (0420) 87024 Fax: (0420) 89886 Telex: 85881

Quick Reference Diary

Date	Venue	Event
27 October	London/Heathrow	APL at British Airways Demonstrations and Discourse by the APLers at BA.
24 November	London	BAA Meeting: APL in Finance
12 January 1990	London	BAA Meeting
16 February	London	BAA Meeting
23 March	London	BAA Meeting
20 April	London	BAA Meeting
8th June	London	BAA Annual General Meeting

British APL Association meetings are to be held at the Royal Over-Seas League, Park Place, near Green Park Tube station. Meetings begin at 2.00pm and close at 5.30pm.

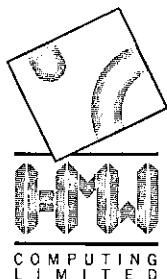
Topics for Forthcoming BAA Meetings

by Peter Branson

- APL at British Airways. A review of some of their major applications using APL. (This meeting will be held at the BA headquarters at Heathrow Airport, where a full demonstration of the systems is possible.)
- APL in Finance. 3 presentations on the use of APL in the financial community.

Dates for Future Issues of VECTOR

	Vol.6 No.3	Vol.6 No.4	Vol.7 No.1
Copy date	30th Nov 89	1st March 90	1st June 90
Ad booking	8th Dec 89	15th March 90	16th June 90
Ad Copy	23rd Dec 89	25th March 90	30th June 90
Distribution	January 90	April 90	July 90

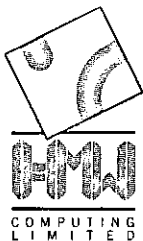


SOLUTIONS FOR THE CITY

We are a specialist firm of consultants with a prestigious list of clients including major City institutions.

For one of these clients we have recently developed a foreign exchange dealing system with the following features:—deal entry, deal enquiry, multi-currency position keeping, basket currency calculators, arbitrage calculators and ticket production.

If you are interested in foreign exchange dealing, treasury operations, portfolio management, commodity broking or similar areas we would be delighted to discuss how we could help. Call Stan Wilkinson, HMW COMPUTING LIMITED Hamilton House 1 Temple Avenue London EC4Y 0HA Telephone 01-353 4212



EXCITING POSITIONS AT THE LEADING EDGE

With top City institutions and leading Blue chip companies amongst our client list we have a lot to offer experienced APL programmers.

Call Stan Wilkinson to discuss your future.

HMW COMPUTING LIMITED
Hamilton House 1 Temple Avenue
London EC4Y 0HA Telephone 01-353 4212

General Correspondence

The VECTOR working group welcomes correspondence on any topic affecting the APL community. All letters should be addressed to the Editor, and should indicate whether they are for the general or technical section. (Letters containing APL code will normally be classed as 'technical'.) The editor reserves the right to edit any letter unless the writer states that it is to be published in full or not at all.

From: N.J.Small

23rd August 1989

Second Generation APLs - Comment on Items in Vector 5.4

Editorial

One way of avoiding Heffalump traps is not to behave like a Heffalump - though this may be easier said than done!

The problems described arise from the freedom that strand notation and mixed arrays allow. One defensive action that can be taken is to tighten up one's programming style, in particular by using unnecessary parentheses to make programs more readable - especially in distinguishing functions and data. Another thing one can do is to learn how to get out of the traps as quickly as possible. A little experience soon teaches one to recognize the type of trap one has fallen into, and that is usually half the way to escaping.

Whilst these new facilities are of great value in creating data structures, I feel that they are not really what one needs to solve the problem of passing multiple parameters to functions. I consider it very inelegant to have to start each function by unpacking parameters (though erasing the original parameter is probably unnecessary with the new data-pointer storage systems). An alternative solution has already been implemented in embryo form in the system function `⌈FMT` used by STSC and Sharp; a typical function header would look something like:

```
(Z1;Z2;Z3) ← (L1;L2) FN (R1;R2;R3;R4)
```

(In most cases, left arguments would not be required.)

With data structures one needs to take care if they are to contain more than a small number of elements at the highest level, as there is an overhead of about 30 bytes per element in mixed or nested arrays, plus a performance penalty. For example, if one has a

long list of serial numbers and names, these should be stored as a two-element nested vector, rather than as a two column-mixed array.

Article by Peter Cyriax on APL*PLUS II

- (i) Peter must have a disk cache or some other facility stored in the upper end of extended memory; with all of a 1 Mbyte extended memory available one will get a workspace of 915120 bytes.
- (ii) As described in the Release Notes, when / and \ are used to the left of another operator there is an ambiguity, because the symbols represent both functions and operators. To indicate interpretation as a function, enclose the symbol in parentheses. For example:

```
      1 2 3(/)''ABC'  
A BB CCC
```

(Without the parentheses one gets NONCE ERROR.)

Articles by Ajay Askoolum on APL*PLUS II

- (i) There is a SYSHELP.SF file, but in my release (1.1) it contains very little. For a help file to be accessible, it must be named as the desired help file in the configuration file; perhaps STSC would like to reinstate the system variable `HNAMES`. As F1 is generally recognised as the standard key for help, one could use the keyboard mapping facilities to give this the same value as Alt-F6. Alternatively, one could put one's home-made help facility on F1 and map Alt-F6 to that.
- (ii) The function `LIBD` is implemented, as in APL*PLUS/PC, so libraries may be defined from within a program, as well as in the file `CONFIG.APL`.
- (iii) The session manager editor does not add extra bytes to DOS files.
- (iv) A numeric editor will be incorporated in release 2 of the interpreter.

Migration from APL*PLUS/PC will, in most cases, be fairly straightforward. Appendix D of the reference manual gives details of the differences between the systems, and suggests three ways of transferring code (none of which will move locked functions):

- Source Level Transfer (functions implementing the Workspace Interchange Convention of the international APL standard)
- functions `DTFN` and `LFFN`, which use a native file

- functions `WSTOFILE` and `PCFILETOWS`, using an `ASF` file (this is the quickest and easiest option). The workspace `PCFILES` contains functions to transfer component files and workspaces in either direction between the two systems (though, obviously, one cannot transfer mixed or nested arrays to the PC version).

File and workspace transfer is very quick and easy; modifying code may take somewhat longer, but can be speeded by making use of the supplied functions `WSSHOW`, `SCAN`, and `FNREPL` to find and edit strings within functions.

Of course, it helps to migrate one's utility workspaces first, so that functions from them can be copied into application workspaces.

The following are probably the items that will most commonly necessitate code changes:

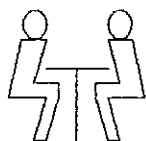
- `INKEY` - the code for Home has changed, and there are some new codes
- `KEYW` is no longer supported
- `WGET` - type 4 is no longer supported (use type 3 instead)
- `POKE` - a few of the more obscure values do not get translated by the interpreter, and `SEG` must be null or 0
- functions from the `ASMFNS` workspace should be replaced by their new versions.

In general, the better one's programming practices and the more modular one's code, the less work one will have to do. Now may be the time to write those little cover functions that one always intended to do tomorrow!

Yours sincerely,

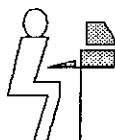
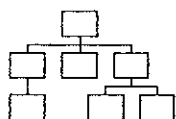
N.J.Small
Assistant Research Manager
IT Systems Support Section
National Westminster Bank

A single source for APL



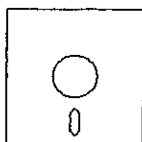
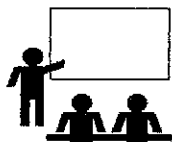
Advice

Design



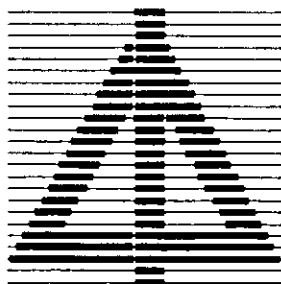
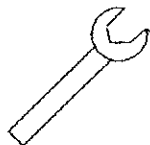
Application
Development

Education



APL software

Support



COCKING & DRURY LTD.

THE APL PROFESSIONALS

180 Tottenham Court Road,
London W1P 9LE

Tel: 01 - 436 9481

British APL Association News

from Graham Parkhouse

The first committee meeting of the current year began with our Chairman, Peter Donnelly, suggesting that we scrap a tradition of going over the actions outstanding from previous meetings and introduce more spontaneity. The meetings in May and June were well attended. It was suggested that the AGM should be in June next year, instead of April, to enable the treasurer to be able to present our accounts properly audited. So the next AGM will be on Friday 8th June.

It is time to be thinking about who will be on next year's committee. Adrian Smith is now in his third year as Editor of VECTOR, and according to the constitution therefore his last. Running a voluntary organisation like ours taxes our time and requires a great deal of motivation, and this certainly applies to Adrian and whoever will follow. We are hoping his successor will come from the Journal Working Group. But none of them has volunteered yet, possibly because they know what they would be letting themselves in for! Producing VECTOR is doubly exciting, with the challenge of typesetting APL characters as well as the challenge of presenting APL as a subject. Adrian has



Alison Chatterton

achieved much on both fronts: I could have written fonts, because he has been developing an APL font for PostScript to look like the original IBM golf ball font as used in Gilman and Rose. Streamlining the production process has been just one of Adrian's achievements which we need to build upon.

Streamlining the processes of a voluntary organisation is vital for its success. People on a committee should be there for their initiative. As much as possible they should be spared the work which is common to every business. Where it is not feasible to employ the help needed there may be the chance to provide tools to make the work easier. An example of

the benefits to be gained from this strategy are becoming apparent in the BAA. Alison Chatterton has been working part time as our administrator for over a year now. The Association provided her with a PC on which she keeps the membership data and the software library.

She has helped Adrian by compiling the news from sustaining members and the APL product guide, looking after advertisements and typing articles. She has taken a substantial amount of work off the Treasurer, who used to maintain the membership data, and is an invaluable support and prompt to the Secretary. You may have already noticed some of the benefits like receiving your July VECTOR in July! While Adrian will willingly bear responsibility for that, Alison helped him achieve it.

That Alison should be a maths graduate and an ex-member of the committee may appear a little over the top, yet that is our good fortune! She first met APL in the OR Department at SE Gas, then worked for Tesco in Cheshunt for nearly three years before going into the family business which allows her, when not looking after Julia (6) and Rachel (4) and her husband, Steve, to work for the BAA. She weathered that initial period when everyone wonders whether their computer is for or against them, and I think she wondered when we were going to give her anything to do. That is now past: she has found her feet and is very busy.

All of this is very gratifying to the Secretary, now in his third year. Who is willing to take over my job in June?

Graham Parkhouse

News from Sustaining Members

compiled by Alison Chatterton

HMW Computing Limited

HMW Computing Limited continue to develop and market "4XTRA" our front end Foreign Exchange Dealing and Position Keeping System. We are exhibiting "4XTRA" at "Computers in The City" on 14-16 November 1989 in the Barbican Centre London. If you would like to visit the exhibition and see our product call Stan Wilkinson on 01-353-4212 for a complimentary ticket.

Our employment agency for the APL community continues to attract permanent positions with a variety of clients throughout the U.K. Currently we have many vacancies for APL technicians from juniors through to consultants with attractive salaries and interesting work content.

To find out what we can offer, please call Val Lusmore on 01-353-4212. If you would prefer to phone out of office hours Val's home number is 07618-727.

APL People Limited

Attempts to make APL People into THE premier APL recruitment agency in the world continue! Jill's recent visit to New York for the APL '89 Conference proved most useful - she made contact with a number of companies outside the UK who have vacancies for skilled APL programmers. A subsidiary of a major Publishing company in Washington is prepared to explore the possibility of recruiting a non-US National and all the bureaucracy that involves, so if anyone's interested in working in the States, please get in touch!

On the home front, we currently have a number of exciting new openings in the UK for people with between 1 and 5 years' systems development experience in APL - mostly in London and the South East.

In addition, sales of the software marketed by our sister company, Applied Production Logic, are considerably up on last year, and we are anticipating a lot more interest following the Computers Manufacturing Show in October at which we will be exhibiting.

Dyadic Systems Ltd

Dyadic is pleased to announce the availability of the DOS/386 version of Dyalog APL that was successfully launched at APL89 in New York. American reactions to the new product were extremely positive, although one well-known user was heard to say "it may be better, but it's a foreign APL!". Dyalog is extremely pleased with the reaction to the $\square$ SM screen manager, and the new window-based development environment. Several early customers have already reported significant productivity gains.

Coinciding with the launch of the new product is a new corporate image designed to match that of Dyadic's parent company, Hunting plc. The Dyadic and Dyalog APL logos have been revamped, and brown, cream and gold discarded in favour of blue and white.

The new (blue and white) Dyalog APL manual-set comprises two volumes: a user-guide and a reference manual. These are supplemented with spiral-bound booklets for the CGI Graphics, Oracle, and (forthcoming) X-windows interfaces. It is symptomatic of the trend in APL that the amount of documentation concerned with user and other interfaces dominates the Language Reference Manual.

On the Unix front, Dyadic reports a growing demand for Dyalog APL, especially on high-performance workstations like the Sun SPARC range and the widely rumoured new IBM RISC machine.

I.P.Sharp Associates

Our London office has moved to new premises with effect from 4th September. The new address is on the back cover of this VECTOR.

In June, the worldwide launch of AllRound View took place. This product is based on Viewpoint, I. P. Sharp's data management system, and is largely written in Sax - Sharp APL under Unix. AllRound View will be marketed by Philips on its Unix-based P90x0 computers. AllRound View provides end-users with access to corporate data residing on IBM mainframes.

Cocking and Drury Ltd

Cocking and Drury are proud to announce ACT-CALC, an APL*PLUS based package for actuaries, which we have jointly developed with Mercer Fraser Ltd. ACT-CALC is a set of actuarial utility functions available on any system on which APL*PLUS is installed, from PC to Mainframe. The product will help the actuary solve everyday calculations within minutes. The functions are named so that they correspond closely with the

symbols of standard international actuarial notation and as a result they feel natural to the actuary. The package provides considerable flexibility, combining various mortality and interest assumptions. It may be used stand-alone, or can be incorporated into other applications.

We have been shipping Release 2 of the APL*PLUS II System for 80386 machines since late August. Customers who bought their APL*PLUS II System from us will by now have received their free upgrade.

The company has been using Oracle in the development of further internal PC based computer systems.

Our 2-day "APL*PLUS Nested Arrays" course has been successfully run within the company, and has been improved as a result. The course is now available to customers in-house or in our London office. This course is aimed at APL*PLUS programmers who want to make appropriate use of their nested array environment. When effectively used, nested array structures and techniques can increase programmer productivity, reduce software development costs and increase the maintainability and robustness of APL applications.

Also on the course front, the availability of IBM's APL2/PC means that it is now possible to offer our standard "Introduction to APL2" and "APL2 In-depth" courses as public courses in our London office. For further information about any of these courses, please contact Beverley Satterley.

MicroAPL Ltd

Richard Nabavi and David Eastwood of MicroAPL attended APL89 to give papers on some of our current work. Richard was discussing the design of MicroAPL's WIMP environment and David gave a presentation on one of MicroAPL's programming projects. We also gave a vendor presentation and exhibited our current products in partnership with our US distributors the Spencer Organisation. Our exhibition stand concentrated on the latest Mac software and previewed the APL.68000 Amiga upgrade. We were also showing one of the latest Unix-based versions of APL.68000.

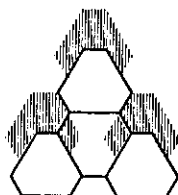
The visit to New York was a welcome chance to meet some of our North American clients and to see what software they are producing. A number of papers at APL89 dealt with applications based on APL.68000 and we were able to look at some of these applications at the show.

MicroAPL is continuing to expand the availability of APL.68000 for Unix-based systems. The latest new port of APL.68000 is for the Nixdorf Targon family. We have also

upgraded our NCR Tower release to 7.30 and other ports are in progress. For details of the range of machines that can run APL.68000, contact MicroAPL.

Version 7.30 for the Commodore Amiga is now under final test and we will be mailing users with upgrade details in the near future. In addition to the work on APL.68000, we are recoding some of the Auxiliary Processors which we wrote for the Mirage environment to run in an Operating System independent fashion. This should widen the range of utility software available for our Unix versions - details will be announced in forthcoming issues of Vector and MicroAPL News.

MicroAPL's consultancy section has signed a number of major new contracts to supply systems on DOS, Mirage and Unix machines. We have also entered into software maintenance agreements with a number of clients to maintain and enhance existing APL systems. As usual, we are still looking for APL programmers for our consultancy activities.



APL PEOPLE

THE APL EMPLOYMENT AGENCY

- ★ Permanent and Contract Vacancies
- ★ UK and Overseas
- ★ Free Advice on career matters
- ★ Discreet, confidential service

We currently have many openings for programmers
with 1-3 years APL experience (any type).

Contact Jill Moss on 0225 462602 during office hours
or 0225 333618 eves & w/ends

or write to her at The Old Malthouse, Clarence St., Bath BA1 5NS

**THE
EDUCATION
VECTOR**

October 1989

Guest Editor Anthony Camacho

This Education Vector has been distributed free to everyone on the I-APL mailing list.

It is reprinted from VECTOR Vol.6 No.2. VECTOR is the Quarterly Journal of the British APL Association; for more information about the British APL Association, contact the Secretary: Graham Parkhouse, Dept. of Mech. Eng., University of Surrey, GUILDFORD.

Contents

Editorial - Programming Style	Anthony Camacho	18
"APL Programs for the Mathematics Classroom" by Norman Thomson	Reviewed by W. W. McLean	20
APL "RISC Programming Style"	G. A. Langlet	23
Start Building Your Utilities	Peter Branson	25
Minutes of I-APL Board Meeting	Sylvia Camacho	29
I-APL Order Form	I-APL Ltd.	32

Editor
Dr Alan Sykes,
Dept of Management Science and Statistics,
University College of Swansea,
Singleton Park, SWANSEA SA2 8PP

The Education Vector

Guest Editorial

by Anthony Camacho

The gap between APL 89 and the British APL Association's conference on the APL Statistics Library is too short. Dr Sykes is overloaded and rather than let the set of Education Vectors become a sparse matrix I offered to have a go at filling the gap. I am asked to make it clear that this is not to be allowed to become a habit. Alan will be back in his editorial chair before you read this. I am starting work on 11 September - after the final copy date - so what you get is what comes to hand.

I-APL Ltd held an open board meeting at APL 89. The minutes are reproduced here. One of the main decisions taken was that, to survive, I-APL must become more commercial. This means that we will begin to pay directors, porters and staff for their services and that we will act as distributors for material which was not produced by I-APL. For example we have begun to distribute Norman Thomson's book "APL Programs for the Mathematics Classroom", published by Springer Verlag. This Education Vector begins with a review of it by William McLean, a physics teacher of Dunoon, who has just begun to try out I-APL. Education Vector ends with a new I-APL order form, simplified by including post and packing charges in the price of each item; there is no price increase.

One of the perennial APL arguments is about style in coding and if there is a theme to this Education Vector it is that. If you read the account of APL 89 you may notice that Adin Falkoff (the man who gave APL its name) is keen on brackets and semicolons and Gerard Langlet would like to persuade everyone to write APL using vectors rather than matrices wherever possible. There is a note from Gerard on the same theme here. Before you reject it out of hand, try it. The effort of trying to adapt to a new style will give you a new insight into APL. In another style we have some advice for beginners from Peter Branson.

Many APLers have strong views about style and there are disagreements that go right back to the roots of the language. If either of these contributions makes your blood boil, please write a rebuttal or a rejoinder. Remember that in the Education Vector we are primarily interested in teaching with APL and the urge for speed which so distorts the style of some commercial programs is less appropriate.

If someone objects that a discussion of style is out of place in Education Vector, saying that new APLers have enough to learn without bothering about it, I answer that the way you say something is indivisibly a part of the thing said, that APL is first a notation and the form of expressions is the essence of notation, and that the decision about what style

you adopt should not be left until you have formed habits by default, some of which may be bad.

APL does have the equivalent of spaghetti in BASIC. It is 'semi-globals'. Nobody has a good word to say for them. Globals are bad enough, from the press they get. Just as G. A. Langlet would like to program without parentheses, Eugene McDonnell would like to program without variables. He has produced a beautiful example published by APL press as "The four cubes problem" and the functions are in the I-APL workspace INSANITY. The amount of work this program does is not trivial - it occupies a standard PC for about 28 minutes!

We have received two useful letters in response to my comment in Education Vector 5.4 about transfer of workspaces between BBC, PC clones and Archimedes. They are from John Dallman, who suggests using Kermit and from Malcolm Rigg who uses Beebdos software. Thanks to both.

Kermit is a serial transfer protocol and has been implemented on almost every computer ever heard of that has a serial interface. A simple cable (only three connections, I believe) joins the two machines and you set one machine to listen and the other to shout and the file is transferred rapidly and reliably. Kermit is in the public domain and is distributed by: Kermit Distribution, Department of Computing, Computer Centre, Lancaster University, Lancaster LA1 4YW. Cables may be obtained made up for the machines of your choice from: Fraser Electronics, 42 Elm Grove, Southsea, Portsmouth, Hants or RSD Connections, PO Box 1, Church St, Ware, Herts SG12 9EN

Beebdos is a program which runs on a PC clone and reads or writes disks which can be read by a BBC - DFS or ADFS. Malcolm says the software is widely advertised.

"APL Programs for the Maths Classroom"
by Norman Thomson
(Springer Verlag £14.50)

Reviewed by W. McLean

This review has to be personal. In Brookhaven, New York, I had just inscribed a birthday present, "Recipes in Mathematics" (in C) for my son and heir with the words: "APL must be able to do it better!" He works for IBM and has a Macintosh. I returned from Long Island in August to find Norman Thomson's book waiting for review. It has been done for the High School!

I'm a physics teacher, who first had APL up and running on the BBC Master 128 only a few weeks before going on holiday to the USA. Long enough to get through Linda Alvord and Norman Thomson's Tutorial and be very impressed with the language's possibilities. (I have taught BASIC and am still enthusiastic about Forth.) The ease of getting a polynomial best-fit equation had me singing!

Last year I bought a second-hand book on "Short cuts in Arithmetic", purely for a 3D diagram illustrating the expansion of $(a+b)^3$. I found it had interesting checks of adding and multiplying by casting out 9s (or 11s), converting yards to miles, pounds to tons (a peculiar APL function), division by continued fractions, decimal fractions and "circulating" decimals, using $(x+a)^2$, $(x-a)^2$ & $(x-a)(x+a)$, $(a+b)^2 + (a-b)^2 = 2(a^2 + b^2)$, $(a+b)^2 - (a-b)^2 = 4ab$ for quick calculations.

Elegance and Variety

My point here is that Norman Thomson has provided functions, conveniently available, to play about with these short cuts. You might say "Why bother? You have APL to do any calculation." But I'm thinking of Richard Feynman's love of playing with numbers calculating e , recurrent decimals ($1 \div 7 =$), logs of certain numbers, squares, roots, cube roots, tangents, integrating $1 \div (1+x^4)$.

Chapter 2: Arithmetic and Numbers

This chapter gives square and triangular numbers and numbers which are both square and triangular and which have their last three digits equal to the number ... yet stops short and leaves cubes (sums of consecutive cubes (from 1 up) are the squares of triangular numbers), hexagonal and octagonal numbers to the teacher.

Primes, factors, HCF & LCM, recurring decimals, we've mentioned. Number bases avoid hex or higher bases. Roman numerals remind me of the complicated BASIC program for

conversion. A dip into encoding and decoding. Casting out nines. Precision of computer numbers. Complex numbers.

Chapter 3: Algebra and Sets

This includes $(a^2 - b^2)$, $(a + b)^2$, $a^2 + b^2$, expansion of $(ax + by)(cx + dy)$, roots of quadratics - BASIC or Pascal just don't have a look in! Then matrix operations, determinants, evaluating polynomials, adding and multiplying and differentiating and integrating polynomials. Arithmetic and geometric progressions. I suspend judgement on sets until I can try APL with digital electronics examples.

Chapter 4: Series

Chapter 5: Formulae and Tables

From Chapter 5 a teacher might choose to introduce compound interest as a problem (repayment every half/whole year who earns more?) or use the formula to introduce e and e^x (which are in the previous chapter on series). Chapter 4 also has an interesting collection of series relating to π (which reminds me to mention the interesting collection of derivations for random numbers later). Recurrence relations, convergence, Pascal's triangle, successive differences of series, Fibonacci numbers, root of 2, trig series, continued fractions, interpolations. Area of a triangle, given the sides, but not area = $(a^2 \tan B \tan C) / 2(\tan B + \tan C)$ or area = $(ac \sin B) / 2$. Interesting applications of sin and cos formulae. The longest/shortest journeys and Pythagorean theorems.

Chapter 8: Probability and Statistics

This chapter must be useful to the APL Statistics Group. I need to use averages and standard deviations - to convert test scores to the same average and standard deviation. I don't see the function but no problem! there's enough to go on. My classes already print out best fit line graphs with a BASIC program but now they will be able to do a polynomial fit ($s = ut + (1/2)at^2$)).

Chapter 9: Combinatorics

Combinatorics is permutations and combinations, derangements, selections, compositions and partitions, Latin and magic squares. Appendix 2 supplies idioms & utilities, including matrix multiplication, bit manipulation, string handling.

Magic Squares

Chapter 10: Games and Miscellaneous

In Port Jefferson, Long Island, I bought a second-hand copy of Rouse-Ball's "Mathematical Recreations" and in many of the explanations I could see how APL could be used beneficially - e.g. with magic squares. Norman Thomson gives a function for a magic square of odd order. No explanations. A breakdown will show the steps, hint at a general solution, but it is a start to enlarging the enquiry. We ask "How do you add diagonals?" The answer comes quickly. Does everyone do it that way?

The games (whist hand, mastermind, life, tower of Hanoi, Optical illusions) are a tiny introduction to the marvellous possibilities.

Appendix 1: Graphics and Chapter 6: Geometry and Pattern

The graphics appendix describes a number of primitive graphics functions, so that, if a package is not available, one can be put together from these and the machine's facilities. "An exposition of mathematics is greatly enhanced by a display of results." Geometry and Pattern uses parametric plotting and the conic sections, epi/hypo-cycloids and astroid are plotted using envelopes. Rotation in 2D and 3D and perspective drawing are followed by co-ordinate geometry equations, polar and cartesian co-ordinates.

Summary

Norman Thomson's book is an "ideas" book of short programs for bringing into the classroom "with a degree of idiosyncrasy in the choice of topics - algorithms that have proved useful in real_work or have caught his imagination as a candidate for demonstrating the value of APL as a mathematical notation." To be fully comprehensive can be dull. It seems that someone might have said, "How about a set of programs in BASIC for maths pupils?" and Norman replied "Much better in APL. More concise. More elegant. More for your money!"

There is an excellent index of topics and an index of programs and variables (280 of them).

APL "RISC" Programming Style

by G. A. Langlet

In general, APL functions listed as examples in books or tutorials can be improved in many ways. Let us take as an example the nice COUNTALL function presented by Alan Sykes in Vector Vol.5 No.3 page 31. This well known algorithm returns the frequency table of vector X with unique values in ascending order within the first column.

```

VR←COUNTALL X;I;N
[1] X←X[ΔX] ⋄ R←1 2 ρ(1↑X),ρX
[2] →(X[1]=X[ρX])/0
[3] I←(1⊂X)≠X ⋄ N←I/⌈ρI
[4] R←(I/X),[1.5]N-0,~1+N
V

```

It works perfectly except when X is not a vector (RANK ERROR), ⌈IO is set to 0 (INDEX ERROR), or X is an empty vector (INDEX ERROR). Let us now introduce the following version.

```

VR←COUNTALLV X
[1] X←,X ⋄ R←~1+1↑X←X[ΔX] ⋄ R←X≠1⊂X←R,X ⋄ X←1+R/X ⋄
R←X,[⌈IO+.5]1+R~-1⊂R←R/⌈ρR
V

```

```

272  ⌈NSIZE'COUNTALL'          (SIZE OF OBJECT IN APL*PLUS/PC)
192  ⌈NSIZE'COUNTALLV'        (I.E. 40= LESS)

```

COUNTALLV runs correctly for any numeric array in either origin. Both functions still produce DOMAIN ERROR when X contains characters; people would probably prefer to take another algorithm for counting words in a text anyway. Benchmarks have shown that both functions consume the same CPU time in various situations, e.g. about 2 seconds on an 8MHz PC-AT for 100=ρX.

Why use goto, locals and, most of all parentheses when their use is absolutely unnecessary, even for didactic purposes?

I have written thousands of functions in APL.68000, Dyalog, APL90, Sharp and APL*PLUS without labels, branches, parentheses (except for ⌈SS in APL.68000) - and of course without any diamond in APL2 and I-APL. Moreover, the use of matrices or higher arrays is to be avoided when possible. So many nice algorithms can be developed with

vectors! Matrices are nice for output only ... when the specification requires them. One can also almost completely discard execute and also encode, decode, transpose, outer product, $\wedge$, $\vee$, $+$, and catenate on large objects. The use of trigonometric functions and of the domino can be drastically reduced in graphics with a minimum of thinking. The 'each' operator will also remain a monster as long as APL interpreters or compilers do not really optimise its effects.

The "RISC" style will consist in using:

- a) Simple and fast primitives, most of the time on vectors only.
- b) Structured programming so that any function will always be executed from the first to the last line.
- c) Short modules: no unlocked function should exceed one screen in size.

I even strongly discourage people from buying or reading books in which branches are a strong topic. In all cases, functions provided in courses should take care of the $\square$ IO effects, and give no errors or yield explicit messages when arguments are empty or have the wrong rank; they should respect the ISO standard, be executable in several implementations without pernicious side effects and continue to work when the workspace is almost full; the use of locals, parentheses or long lines often eats space unnecessarily. People should teach how to build APL applications from small bug-free modules and how to avoid the traps.

P.S. A compromise between size, readability, execution time and portability has to be permanently found; this is true for all programming languages, but very difficult with APL. It was a good reason to discard the parentheses, especially when I discovered that parenthesis-free code was executed 20% faster, on average, by APL68000. You can read it quicker, you are obliged to split your expressions, you get less WS FULL hazards.

```
(If you
      (have
        (friends
          (within
            (the
              (Lisp community)
            )
          )
        )
      )
    )
```

It might be a good suggestion there too.

Start Building your own Utilities

by Peter Branson

What are Utilities?

As you get going with APL, you may find it useful to start building up a library of utility functions. These are functions not tied to any particular APL application, but come in useful all over the place, such as alphabetical sorting, selecting a particular name from a list of names. It may be that your APL has the nucleus of a good set in a supplied workspace. Use them. The odds are that, sooner or later, you will want some of your own. In any case writing utility functions helps extend one's knowledge of APL, since some care is needed to make them robust.

A Useful Example

Let's look at an example. When handling character (text) data, there is often a need to join two matrices of characters one above the other - the technical term is catenation on the first axis. The direct APL way of doing this is:-

```
(3 3p'CATANDDOG') ,[1] 2 3p'XYZ'
```

```
CAT
AND
DOG
XYZ
XYZ
```

Here the bracket index join , [1] tells APL that we want to join the right argument onto the bottom of the left argument. Note that the example above will only work when the two matrices have the same width. If they are not the same you will get a length error (try it and see).

Now, what if the matrices do have different widths, which occurs fairly often? A simple line of APL will no longer suffice, so let's set up a utility function for this. I will call this VCAT with a header R←A VCAT B. Here A and B may well both be matrices, but it would be useful if the function would work with vectors as well.

For example, we might sometimes want to add just one name to a namelist, in which case B could be a vector. And while we are at it let's allow for the case where A or B is a singleton; a one-element vector or even a scalar.

How shall we do this generalisation? Well, your first thought might be to do some tests on the shape of A and B and, according to what type of object each is, to branch to different parts of the function to do what is needed. But this is APL, remember, and of course there is a better way, without branches! Here is one way of writing the VCAT function:

```

      ∇ R←A VCAT B;MAXW;TOP;BOT
[1]  ⍺ VCAT: VERTICAL CATENATION
[2]  ⍺ EXTEND SINGLES AS REQD. FOR MATRIX RESULT - NUM OR CHAR
[3]  ⍺
[4]  A←(¯2↑ 1 1,⍺A)⍺A
[5]  B←(¯2↑ 1 1,⍺B)⍺B
[6]  MAXW←(¯1↑⍺A)∖¯1↑⍺B
[7]  TOP←((1↑⍺A),MAXW)↑A
[8]  BOT←((1↑⍺B),MAXW)↑B
[9]  R←TOP,[1] BOT
      ∇

```

Have a look at line 4; this finds the shape of A (the result of shape is always a numeric vector), catenates this onto the vector 1 1 and uses the last two elements of the result to reshape A. If A is a two-dimensional array to start with, it will be unaffected; the reshape will be to its original shape. If it is a vector it will be reshaped to a one-row matrix of the original vector length and if it is a scalar (the shape of a scalar is the empty vector) then it will be reshaped to a one-row, one-column matrix.

The next line does the same for B, and line 6 just gets the larger of the two widths. One of lines 7 and 8 will always be redundant; if either of A and B has fewer columns than the other then a take (+) with the MAXW number of columns will pad it with blanks if the arguments are characters and with zeros if the arguments are numeric. This capability of take is often called 'overtake', a term worth remembering if it helps to fix this in your memory. And as you will have realised VCAT has the added bonus that it works for numeric data as well as characters.

A Documentation Utility

Looking back at VCAT you will see that the first line is a comment showing the function name with a brief description of what it does. Many experienced APLers put comments like this in all their functions, so that by running a suitable documentation utility, a list of names and outline descriptions can be obtained for all the functions in the workspace. This can be useful as the number of functions builds up; it is only too easy to forget what some of them do, and their names (often kept short for typing convenience) may not be mnemonic enough so JFNS won't do the job.

Let's see what is needed. We will be obtaining the function as a character matrix from $\square CR$ and can pick the appropriate line by indexing. The line lengths will vary but $VCAT$ will allow for this when forming the result.

A suitable function, $DOC1$, is listed below. I hope the comments obviate further description:

```

R<DOC1;NAMES;N;J;CR
-----
[1]      A LIST FN NAME AND 1ST COMMENT LINE OF ALL FNS IN WS
[2]      A NAMES IS LIST OF ALL FN NAMES, N IS NO OF NAMES
[3]      N<1+pNAMES<NL 3
[4]      A R IS INITIAL RESULT - EMPTY CHAR MATRIX
[5]      R< 0 0 p''
[6]      A SET COUNTER J TO 0
[7]      J<0
[8]      A NOW TAKE EACH NAME IN ORDER
[9]      LOOP:-(N<J+J+1)/END
[10]     A CR IS THE FN AS A CHARACTER MATRIX
[11]     CR<CR NAMES[J;]
[12]     A GO TO NEXT IF HEADER LINE ONLY OR DIRECT DEFINITION
[13]     -(1≥1+pCR)/LOOP
[14]     A DROP LEADING 'A ' AND TRAILING BLANKS FOR NEATNESS
[15]     CR<1+('A'=CR[2;1])/CR[2;]
[16]     CR<(+/\\" ' '=CR)φCR
[17]     CR<(+/\\" ' '≠CR)↑CR
[18]     A R IS PREVIOUS RESULT PLUS (VCAT) THIS COMMENT LINE
[19]     R<R VCAT NAMES[J;],':',CR
[20]     →LOOP
[21]     END:
      V

```

Can you amend this function so that it will select all the comment lines in any index origin? (Ed: Can you rewrite it without parentheses?)

Comment or Decoment

Experienced APLers often recommend that all functions be extensively commented, either to explain some of the tricky APL or to help describe the overall task. There are times when you may want to remove the comments e.g. if space is short.

You could)SAVE the workspace under a different name and then run DECOMM and)SAVE with the usual name. Don't fail to keep a commented copy.

```

      ∇ R←DECOMM FNAME;CR;MASK
[1]  R DECOMM: DECOMMENT A FUNCTION
[2]  CR←⊂CR FNAME
[3]  MASK←CR[;1]≠'A'
[4]  CR←MASK/CR
[5]  R←⊂FX CR
      ∇

```

Here we form CR as before and then do a not-equals comparison of the first column with the lamp symbol. This gives us a boolean vector with ones for the lines we want to keep and zeros for the lines whose first characters are lamp. Then we compress CR to remove the unwanted lines and finally convert CR back to a function with ⊂FX. As it has the same name it will overwrite the previous version.

One disadvantage of DECOMM is that it will remove the first line comment. The simplest modification will keep it; add line:

```
[3.5] MASK←MASK∨(ρMASK)≠0 1
```

This ensures that line one is always represented by a 1 in MASK.

Meeting of the Board of I-APL and other Interested Parties on 6th August 1989 at the New York Hilton

by Sylvia Camacho

Present: Anthony Camacho, Ed Cherlin, Paul Chapman, Dave Ziemann, Norman Thomson, Garry Helzer, Linda Alvord, Sylvia Camacho, Rolf Levenbach, Per Gjerlov, Kim Andreasen, Lee Dickey, Ed Shaw, George Cherlin, Tama Traberman, Curtis Jones.

Anthony Camacho opened with a suggested Agenda:

- (1) Last year's Minutes
- (2) Commercial decisions
- (3) Progress to date
- (4) Plans for Conference
- (5) AOB

(1) Last Year's Minutes

Minutes from last year were taken as read (see Vector Vol.4 No.4).

(2) Commercial Decisions

Anthony Camacho suggested the commercial decisions to be taken as:

- (a) Whether to stock and distribute Norman Thomson's "APL for the Mathematics Classroom".
- (b) Whether to print a manual and distribute TryAPL2.
- (c) Whether to distribute Sharp APL PC.

In order to do these things I-APL would have to become a profit making (though not profit distributing) organisation which would pay royalties to authors and commercial rates for other work. So far I-APL has not paid anything but expenses to its board members. To make it possible for them to continue the work payment is becoming essential.

Anthony said he would like to hear the opinion of all present on whether I-APL could make this change with a good conscience. Each in turn spoke to this point and there were several differences of emphasis.

Ed Cherlin stated his need to generate cash flow to pay for publicity, admin effort and distribution. Ed favoured extending the products carried with this end in view.

Norman Thomson supported this view and suggested adding paid consultancy.

Paul Chapman suggested other possible products.

Ed Shaw expressed fears that extending the product range would overstretch the organisation (evidence being that he has not yet been invoiced for some stock).

Lee Dickey would support a move to increase the number of distributors.

Dave Ziemann expressed support for increasing the product range providing the effort can be found and that it would not alienate the contributors.

George Cherlin suggested applying for a National Science Foundation grant.

There was general agreement that I-APL must not be seen to favour any commercial vendor or any flavour of enhanced APL.

The proposal that I-APL should move towards being a more commercial enterprise was passed nem con. The proposal to carry other APL products was carried 7 in favour, 2 against and the rest abstaining. The implied advice was that the Chairmen should only do it if there was no danger of diverting effort from important work such as ports to school machines.

(3) Progress to Date

Anthony Camacho reported:

- Well over 1000 copies of I-APL have been sold.
- The manual has gone into a second edition including BBC and Archimedes machines. [After doing the basic port to these machines without payment, the porter was paid £800 to provide DFILE, graphics, sound and colour interfaces.]
- Tama Traberman's 'APL in Social Studies' has been printed and added to the product list.
- Norman and Linda's Tutorial has been revised and reprinted.
- Garry has revised An Encyclopedia of APL and the second edition has been printed.
- The current financial position is that there is about £2000 in the U.K. bank account and £19,000 (at full selling price) of stock.

- The BAA has paid for production and mailing of the Education section of Vector to the 1400-1500 I-APL mailing list at an approx. cost to them of £600 per quarter. There seems to be no reason why they won't continue to do this - it is bringing them new members.
- There are porters working on Apple, Amstrad, Atari, Macintosh, CP/M and Transputer machines but none for Commodore 64.
- Paul has produced a 32-bit version of I-APL which is currently ported to the Archimedes and gives workspace up to the memory installed. It also runs faster than the version with 16 bit addresses as in that version each address has to be reduced to 16 bits by masking. This 32-bit version can be distributed in the same way as I-APL. As workspaces will not be loadable from the 16-bit into the 32-bit or vice versa, we feel it necessary to provide means to transfer workspaces between the two systems before issuing any 32-bit version.
- Paul has also produced a compiled and speeded up version for the Archimedes which he would like to earn royalties on and charge a higher price for.
- The 32-bit IAPL.DEB file will run with an appropriate DEGO on any 32-bit machine. It could therefore be ported to 68000 and 80386 machines and give workspaces up to the size of the memory. It was agreed that such ports could be commissioned and issued.

Rolf Levenbach reported progress on the Apple Port. Most of testport was working and he was part of the way to)SAVE and)LOAD but when he tried the complete interpreter it crashed after displaying the sign-on message. Paul agreed to spend some time with him to see whether he could help. Anthony said that when the interpreter was working, I-APL would pay Rolf to write DFILE and provide appropriate access to colour, graphics and any other Apple][facilities.

(4) Plans for the Conference

Ed Cherlin asked for volunteers to man the I-APL Booth, received 6, but in the event this did not happen.

(5) AOB

Per Gjerlov asked that there should be some I-APL papers at APL90 and that some information on the 32-bit version and experience with I-APL in Schools should be included.

Order Form: I-APL Ltd February 1989

The address for queries is:

2 Blenheim Road

St Albans

Herts AL1 4NR

The address for orders is:

56 The Crescent, Milton

Weston-super-Mare

Avon BS22 8DU

Please supply:

Part	Item	(pounds sterling)		
		No copies	Price	Total
PC12	PC & clones disk & manual	_____	£4.50	_____
BBC12	Master 80T disk & manual (also B with 32K sideways RAM)	_____	£4.50	_____
A12	Archimedes disk & manual	_____	£4.50	_____
M2	Manual (PC/BBC/ARM/Nimbus)	_____	£2.50	_____
T3	Tutorial by Thomson & Alvord	_____	£2.50	_____
E4	Encyclopedia by Helzer	_____	£5.50	_____
S5	APL in Social Studies by Traberman	_____	£2.50	_____
N6	APL Programs for the Maths Classroom by Norman Thomson (pub Springer Verlag)	_____	£14.50	_____

Packing charge per order £1.00

Prices include VAT (reg no 479 1634 13) and postage

I enclose cheque to I-APL Ltd for: _____

N.B. We take no credit cards.

Name: _____

Full address: _____

_____Please send me a VAT invoice (tick) ☐ Vat is 27p on disk only.

Office use only

APL Product Guide

Compiled by Alison Chatterton

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry - see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- * Complete APL Systems (Hardware & Software)
- * APL Timesharing Services
- * APL Interpreters
- * APL Visual Display Units
- * APL character set printers
- * APL-based packages
- * APL Consultancy
- * APL Training Courses
- * Other services
- * Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'.

All contributions to the APL Product Guide should be sent to Alison Chatterton, at the address on the inside back cover.

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Dyadic	APL Coprocessor	3,500+	32-bit coprocessor board for IBM PC. NS32000 cpu with FPP, up to 4Mb RAM, 16Mb virtual memory. Software includes Unix V.2, Dyalog APL, graphics support, DOS interface. Provides multi-user Unix/DOS environment.
	IBM 6150	15,000+	Multi-user Dyalog APL system with Fast 32-bit RISC processor, FPP, up to 8Mb RAM, 210Mb Disk, 16 users. Interface to SQL, graphics and APL support for standard IBM peripherals.
	Altos 3068	25,000+	Multi-user Dyalog APL system with MC68020 cpu & MC68881 FPP. Also features a LAN which supports IBM PCs as Dyalog APL terminals.
	Sun 3	15,000+	Multi-user Dyalog APL systems which can be configured as a network of workstations and or a traditional time-sharing cpu. With its 25MHZ 68020 cpu, the Sun 3/200 is the fastest APL microcomputer on the market.
HMW	Powertools	poa	PCs, APL*PLUS/PC and Powertools supplied with bespoke turnkey systems.
Inner Product M.B.T.	IBM PC	2,000-6,000	IBM PCs supplied for turnkey applications
	MBT Series 10	poa	UNIX/68010 based multi-user APL system
	TORCH	poa	68000/280 multiprocessor
MetaTechnics	-	poa	Details on application -- IBM PC compatible
MicroAPL	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
	Spectrum	7,000+	Expandable multi-user APL computer using Motorola 68000. Std. configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
	Spectrum/386	2,000+	80386-based PC to run APL*PLUS II. Fully expandable.
	Atari 1040ST	799	1 Mb Mono/Colour System, includes 1 Mb disc drive & mains transformer built into Console.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES (£)	DETAILS
Mercia	APL*PLUS	poa	STSC's Mainframe Service - MAILBOX etc.
I.P. Sharp	SHARP APL	poa	International Network application systems and public databases.
Uniware	APL*PLUS	call	STSC's mainframe service

APL INTERPRETERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL Software	Dyalog APL	1,000-8,000	See Dyadic Systems entry
Cocking/Drury	APL*PLUS/PC Rel 8	475	STSC's full featured APL for IBM PC, PC/AT and PS/2. Now with virtual workspace, quad-MF, GSS graphics, Network and VGA support. Includes session ring editor.
	Run-time	poa	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL*PLUS II	2,000	includes 1st year maintenance
	runtime	250	
	site licence	poa	
	APL*PLUS UNX	poa	STSC's 2nd gen hi-performance APL for IBM PC/RT, Sun, DEC, HP and major Unix computers and workstations.
	APL*PLUS VMS	poa	STSC's 2nd generation APL for DEC VAX computers running under VMS. Comparable in feature to APL*PLUS II system, APL*PLUS Unix, and APL*PLUS mainframe.

	APL*PLUS mainframe	poa	STSC's 2nd gen APL interpreter. Enhances VS APL with component files, nested arrays, formatting, user command processor, fastfns and other productivity features. For VM/CMS, MVS/TSO: offers simple upgrade from VS APL.
Dyadic	Dyalog APL	795-10,000	2nd gen. APL for UNIX systems, e.g. IBM 6150, Sun, Vax, NCR, HP9000, AT&T, Aitos, Apollo, Whitechapel, Sperry, etc.
Gen. Software	APL*MYRIADE	poa	Runs on Texas Instruments TI990 range.
HMW	APL*PLUS/PC	poa	STSC's powerful APL Interpreter
IBM UK	IBM PC APL2	348	APL2 for the IBM PC. Program 5799-PGG/1, PRPQ number RJ-0411. From all IBM dealers.
	Mainframe APL2	poa	APL2 release 3
	APL2 Appl. Evt	poa	
Inner Product	VIZ::APL	250-350	8-bit Zilog Z-80 CP/M
	APL*PLUS/PC	600	See under Cocking & Drury
M.B.T.	Dyalog APL	poa	See Dyadic Systems entry
	MBTAPL	poa	Enhanced Dyalog APL for MBT hardware.
	VIZ::APL	poa	Customized for TORCH hardware
MetaTechnics	APL*PLUS Rel 6	475	Discount on quantity.
MicroAPL	APL68000	1,000-2,000	Full implementation with component files, error trapping etc. for SPECTRUM, HP300, SUN, NCR etc.
	QL/APL (keyword)	87	Full keyword APL for QL with many extra features.
	QL/APL (APL chars)	87	VSAPL compatible APL for QL with many extra features.
	APL68000 Apple Macintosh	87	
	APL68000 Apple Mac II	260	
	APL68000 Amiga	87	Full APL interpreters with support for windows, mouse, graphics etc.
	APL68000 for Atari ST	87	
	APL68000 for IBM-PC	995	
	APL*PLUS/PC - REL 8	425	
	APL*PLUS II	1,395	
Portable	PortAPL	\$195	IBM PC Software
		\$275	Mackintosh
		\$2,995	DEC VAX
I.P. Sharp	APL*PLUS/PC	480	STSC APL
	Upgrade V7 to V8	100	
	Upgrade any to V8	200	
	Sharp APL/PCX V1	1,650	For IBM XT/AT 370
	Sharp APL/PC V2	40	shareware for IBM PC/XT
	Sharp APL	poa	for IBM mainframes
Uniware	APL*PLUS/PC	call	STSC's full feature APL for IBM PC/XT/AT and PS/2
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers.
	APL*PLUS II	call	STSC's full feature APL for 385 computers

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Dyadic	Lynwood j300	1,560	Monochrome ANSI 3.84 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.
	Lynwood j500	2,295	Colour ANSI 3.84 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.
	IBM 3163	791	Low-cost Monochrome APL vdu. Supports downloaded Dyalog APL font.
	IBM 3164	1,093	Low-cost Colour APL vdu. Supports downloaded Dyalog APL font.
Farnell	Tandberg TDV 2221	995	Ergonomic design APL terminal, 50-19200 baud, 15f8i5 anti-reex screen, low profile keyboard

	Tandberg TDV 2271	1,195	Combined APL/ANSI ergonomic terminal as above.
Gen. Software	Mellordata	400	Second-hand Elite 3045A
M.B.T.	various		Contact MBT for details
Meta Technics	IBM EGA compatible	299	Emulates EGA & Hercules, Half Card
MicroAPL	Insight VDT-1	795	Inexpensive APL VDU
	Insight GDT-1	1,450	With monochrome graphics
	Concept201	1,295	APL VDU with 8 page memory
	Concept 201G	1,650	Graphics VDU
Shandell	HDS2010	1,215	ANSI3.64 DEC VT52/100/220 compatible. 15" tilt/swivel screen, low profile keyboard 8 page memory, windows, viewports, 80/132 columns, full overstrike, 2 or 3 comms. ports, 55 PF keys. NVM storage.
	HDS2010G/GX	1495+	As above plus Tektronix 4014, Retrographics VT640/D0640 and Visual 500 compatible. 1024 x 390 or 1024 x 780 resolution.
Tektronix	4114B	13,500+	19" D V.S.T.:Graphics: 3120 x 4096 displayable; Intelligent: up to 800K memory; APL keyboard (option 4E)
	4125	21,550+	19" 2D colour graphics; Workstation (1280 x 1024);Intelligent: up to 800K memory; APL keyboard (mod AP)
	4128	26,822+	As 4125 plus 3D wireframe

APL PRINTERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Datatrade	Datasouth DS180+	1,295	180 cps matrix printer with 4K buffer, 9 x 7 dot matrix and APL option.
	Datasouth DS220	1,695	Letter quality; graphics capability, APL option (both available with IBM Twinex or Coax interface).
Dyadic	IBM 4201 Proprinter	poa	100, 200, 40(nl) cps, matrix printer, with graphics Supports downloaded Dyalog APL font.
	Toshiba P351	poa	24 pin high-quality matrix printer 100 cps letter quality, 192 cps draft.
Inner Product	Epson FX80	500	Soft char. set, 160 cps, 80 column
	Anadex 9620	1,150	200 cps., 132 col., tractor feed
	Siemens PT88	620	180 cps., 80 col., silent
	TGC Starwriter	1,180	40 cps., letter quality
M.B.T.	Facit 4565	poa	40 cps letter-quality
	Facit 4510/11/12	poa	Matrix printers
MetaTechnics	Quen-data	295	Low-cost APL Daisy-wheel printer
MicroAPL	Datasouth DS180+	1,195	See Datatrade entry
	Philips GP480	2,137	Matrix printer with letter & draft quality and APL (480 cps).
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL-385	APL-385	50(IPC),125(mf)	including ...
	FSM-385		Screen development
	DRAW-385		Screen design
	DB-385		Relational W.S.
	GEN-385		Miscellaneous Utilities
APL Software Ltd (mainframe)	AFM/AP	11,035	Interprocess Software for VM/CMS & MVS/TSO.
	- Keyed Access	2,650	Component File Management System (VSAPL/APL2)
	- Interactive Link	1,325	
	- Mail Exchange	2,650	

	CALL/AP	4,030	Non-APL program execution (VSAPL/APL2)
	APLPRINT	2,205	Output to high speed line printer or 328x devices (VSAPL/APL2)
	ENHANCED FORMAT	2,205	Extends Format operator to full "Quad-FMT" status (VSAPL/APL2)
	ISP	750	Input and Output Stack Processors for manipulating terminal I/O
	OSP	2,205	with facilities for Error Trapping (VSAPL)
	DISPLAY CAPTURE	poa	Allows terminal output to be collected and held for retrieval by an APL function (APL2)
	UCF	poa	User Communication Facility for data transfer between users (APL2)
	RDS	poa	Relation Data Base System
	PANEL	poa	Fullscreen management system
	PFS	poa	Program File System - APL Systems development aid
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beta-plan	BETA-FONT	poa	Multiple font PC character generator. Dealers required for non-Scandinavian countries.
APL IMPETUS	Impetus	poa	Hierarchical Planning System
Butel	Merlin	5,000	Mainframe APL spreadsheet runs under VM/CMS, TSO, VSPC
	Merlin/PC	poa	Version for APL*PLUS/PC
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad- functions & nested arrays for IBM VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	First APL compiler. Available with APL*PLUS enhancements and Sharefile under VM/CMS & MVS/TSO
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Convert non-APL files to APL files
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Powerful Information Centre spreadsheet incorporating data exchange between APL and FOCUS, IFPS, SAS, APL/DI, ADRESII, LOTUS123, VISICALC, MULTIPLAN, DIF files
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(microcomputer)	STATGRAPHICS Rel 2.6	645	Integrated Statistics and graphics on IBM PC, AT, PS/2 and compatibles
	Update 2 to 2.6	140	
	Update 1 to 2.6	330	
	APL*PLUS PC Tools	325	Utilities including: RAM disk, full screen data entry, menu input, report generation, file documentor, exception handling and games.
	IRMA Module	120	327x IRMA support.
	Fin & Stat. Library	350	Financial & statistical routines
	SPREADSHEET MGR	195	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII, LOTUS
E&S	PROTOPAK		Packages for prototyping management information systems - consisting of: PC & mainframe modules
	RMS		Relational databases.

	AMS		Multi-dimensional arrays
	RAMS		Combined RMS & AMS.
	BMS		Dynamic nancial modelling & forecasting
	FMS		Full-screen handler for APL*PLUS/PC. (AP124-based)
	CMS		Communications package.
	SOS	poa	Scheduled ordering and stock control.
FASTCODE	FASTCODE, MONITOR	\$239	Assembler written monitor for APL applications in APL*PLUS/PC
Gen. Software	PROPS	500+	Spreadsheet system for Product and/or Project Planning.
H.M.W.	INPUT	poa	Matrix manipulation package for data entry & report generation
	POWERTOOLS	poa	For faster primitive screen handling, forms validation (SAA / CUA conforming). Includes support for mice, touch-pads etc.
	PRINTPAK	poa	Block printing for VM/CMS
	VIEWPAK	poa	AP124 Protocol emulator for IBM/PC
	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
Inner Product	Viewcom	150	Control Viewdata from APL
	APL/dBASE II	150	Interface APL with dBase II
	APL/dBASE III	150	Interface APL with dBase III
	APL/LOTUS	150	Interface APL with Lotus
	APL/WORDSTAR	150	Interface APL with Wordstar
	APL/MULTIPLAN	150	Interface APL with spreadsheet
	CEMAS	3,500	EEC monetary and agrimonetary analysis.
M.B.T.	RHOMBUS	poa	Integrated Ofsu3suce System
	HASLEMERE	poa	Hotel Accounting System
Mercia	STATGRAPHICS 3.0	685	Integrated stat. graphic system for PCs.
	Upgrade 2.1 to 3.0	395	
	Upgrade 2.6 to 3.0	175	
	MICROSPAN	250	Comprehensive APL tutor
	LOGOL	poa	Logistics management system for PC, Forecasting, Inventory Control, Scheduling, Distribution, etc
MetaTechnics	MetaScreen	99	Full-screen handler for APL*PLUS/PC, based on VSAPL AP124
	MetaPack	495	Comprehensive utilities package for APL*PLUS/PC. Includes: MetaScreen, MetaWS, 3rowse, Toolbox, Numeric Editor.
	APL-IEEE488	99	Controls IEEE488/GPIB Bus from APL*PLUS/PC.
	PLOT/PC	99	2D & 3D Graphics package. Includes interactive diagram Editors.
	Browse	99	Scrolling of DOS files, large APL variables.
	ADAPTA DLS	poa	Production & purchasing scheduling for process manufacturing.
	ADAPTA MSP	poa	Job-shop loading & scheduling for multi-stage production.
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	For APL*PLUS/PC product

	PC TOOLS Vol 1	295	
	STATGRAPHICS Rel 1	495	
	STATGRAPHICS Rel 2	535	
Parallax	ExecuCalc	\$5,000	Mainframe-based electronic spreadsheet for VM/CMS & MVS/TSO with links to micro products.
	ExecuPlot	\$5,000	Mainframe-based colour graphics with micro links.
I.P. Sharp	GLOBAL LIMITS	poa	Exposure management for banks
	IPSA/CONNECT	poa	Mainframe to micro link
	MAILBOX	poa	Electronic Mail
	MAILBOX/PC V.2	65	Full screen front end to IPSA mailbox
	upgrade to V.2	15	
	NEWSFLASH	poa	Real time message exchange
	VIEWPOINT	poa	4GL - Info centre product
Sugar Mill	Stat 1	\$129.95	Statistical toolbox, menu driven
Uniware (mainframe)	STSC's ENHANCEMENTS	10,715	Quad-functions & nested arrays for IBM VSAPL under VM/CMS and MVS/TSO
	STSC's SHAREFILE	10,715	Component files for IBM VSAPL under VM/CMS and MVS/TSO and for IBM APL2
	PROGRAMMER TOOLS & UTILITIES	5,715	
	FILEPRINT	5,715	
	FILESORT	5,715	
	FILECONVERT	5,715	
	FILEMANAGER (EMMA)	5,715	STSC's database package.
	APL*PLUS COMPILER	21,430	First APL compiler. Complements APL*PLUS enhancements and Sharefile under VM/CMS and MVS/TSO.
	EXECUCALC	3,995	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL (VM or TSO).
(microcomputer)	STATGRAPHICS	call	Statistics & Graphics for PCs.
	STATGRAPHICS FCA	call	An add-on module to STATGRAPHICS: Factorial Correspondence Analysis.
	APL*PLUS/PC TOOLS		
	- VOL1	call	Incl. 327 x IRMA support, RAM disk, full screen data entry, menu input, report generation, games.
	- VOL2	call	Incl. File documentor, screen editor, exception handling.
	SPREADSHEET MNGR	call	APL spreadsheet with built-in ASCII, LOTUS and SYMPHONY interfaces.
	APL*PLUS/PC FIN. & STAT LIBRARY	call	Collection of financial and statistical utilities.
	POCKET APL	call	Smaller version of APL*PLUS/PC.
	UNIASM	call	Collection of assembler routines for APL*PLUS/PC users.
	UNITAB	call	APL*PLUS/PC spreadsheet-like data entry and validation system.
	The APL DEBUGGER	call	First released APL*PLUS/PC debugger.
	OVERLAYS	call	Fast assembler routines to handle overlays in APL*PLUS/PC.
	R:BRIDGE	call	Interface between APL*PLUS/PC & R:BASE 5000.
	DMA	call	A version of EMMA (APL database manager) for APL*PLUS/PC users.
	APL2C	call	Interface between APL*PLUS/PC and DATALIGHT C language.
	ADAPTA/DLS	call	Production & purchasing scheduling for process manufacturing.
	PLOT*PLUS	call	Graphics package for APL*PLUS/PC

Warwick University	BATS	500	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to 35 for academic institutions.
	FAB	free	Training program for the above.

APL CONSULTANCY

(prices quoted are per day unless otherwise marked)

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL Consulting	Consultancy	180-550	Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation.
APL People	Consultancy	from 120	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL68000, C/Unix, TSO/MVS, VM/CMS, graphics. Additional skills available: OR, statistics, financial, commerce, retail, engineering.
APL Software Technology	Consultancy	poa	Technical & business systems, micros, networking & communications a speciality.
Buckland Management Systems	Consultancy	poa	Business and Technical systems in commerce and industry. Designing, programming and implementing applications of O.R and statistics.
Camacho	Consultancy	poa	Specialising in programming & manual writing
Chapman	Consultancy	150-300	24-hour programmer; APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	175-250 250-350 300-450 350-600 400-750	Junior consultant Consultant Senior consultant Principal Consultant Managing consultant
Peter Cyriax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
Delphi	Consultancy	poa	Specialising in management reporting systems and APL on microcomputers.
Dyadic	Consultancy	poa	APL system design, consultancy, programming & training for Dyalog APL, VSAPL, APL*PLUS, IPSA APL etc.
E & S	Consultancy	150-250	System prototyping: all types of information system.
FASTCODE	Consultancy	poa	Specialise in improving performance of APL applications on micros & mainframes.
Gen. Software	Consultancy	100+	
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Ian A. Clark	Consultancy	poa	Computer-based Information Systems implementation where acceptance is critical. APL on PC and Macintosh. Human Factors of HCI; novice ease of use; online assistance, training courses; distance-learning materials.
Inner Product	Consultancy	200	On-site micro-mainframe APL, PC/DOS & Assembler
Lloyd Savage	Consultancy	poa	Decision support, particularly specialising in Sales & Marketing systems.
M.B.T.	Consultancy	poa	
Mercia	Consultancy	poa	APL*PLUS & VSAPL consultancy.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
M.T.I.	Consultancy	poa	Specialise in Maintenance and development of existing APL systems
Parallax	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	250	Specialising in Banking, Financial & Planning Systems.

Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
I.P. Sharp	Consultancy	poa	Consultancy & support service world-wide.
Uniware	Consultancy	call	Junior to managing consultancy in APL.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
I.P. Sharp	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APL Consulting	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
	Tim Perry	Jubilee House, Chapel Rd, HOUNSLOW TW3 1XT. Tel: 01-577 3541
APL Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
APL People	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
APL Software	David Alis	Jubilee House, Chapel Rd, HOUNSLOW TW3 1XT Tel: 01-577 3541
APL Software Technology	John Hagger	14 Rosewood Avenue, Alveston, Bristol BS12 2PP Tel: 0454415737
	Per Hultin	46 Vicarage Road, South Benfleet, Essex SS7 1PB Tel: 0374550501
Beta-plan APS	Kim Andreassen	Stengrade 75, DK-3000 Helsingør, Denmark. Tel: 45 2 21 48 48
Buckland Management Systems	John Buckland	Westwood, 9 Grove Road, Camberley, Surrey, GU15 2DN Tel: 0276 684327
Butel Technology Ltd.	Mike Munro	Butel House, 3 Great West Rd., London W4 5QJ Tel: 01-995-1433
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. Tel: St. Albans (0727) 60130
Paul Chapman		18, Trevelyan Road, London, SW17 9LN Tel: 01-767 4254
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE16 Tel: 01436 9481
Datatrade Ltd.	Tony Checkseid	38 Billing Road, Northampton, NN1 5DQ. Tel: 0604-22289
Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 03677-384
Dyadic Systems Ltd.	Peter Donnelly	Park House, The High Street, Alton, Hampshire. Tel: 0420-87024
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-24741
Farnell International Instruments Ltd.	R. Fairbairn	Jubilee House, Sandbeck Way, Wetherby, W. Yorks. Tel: 0937-61961
	Roger Attard	Davenport House, Bowers Way, Harpenden, Herts. Tel: 05827-69071
FASTCODE	Andrew Dickey	P.O. Box 281, Croton-on-Hudson, New York 10520, U.S.A. Tel: (914) 271-3200
General Software Ltd.	M.E. Martin	22 Russell Road, Northolt, Middx. UB5 4QS. Tel: 01-864 9537
H.M.W. Computing Ltd.	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, LONDON EC4Y 0HA Tel: 01-353 4212 Telex: 926604 HAMHSEG Fax: 01-353 3325
Ian A. Clark		9 Hill End, Frosterley, Bp. Auckland, Co. Durham DL13 2SX Tel: 038852-7190

IBM UK Ltd	Nat.Enq. Centre	414 Chiswick High Rd, London W4 5TF Tel: 01-747 0747
Inner Product Ltd.	Dominic Murphy	Eagle House, 73 Clapham Common Southside, London SW4 9DG Tel: 01-673 3354
Lloyd Savage Ltd	Philip Johnson	Cambridge House, Oxford Road, Uxbridge, Middx, UB8 2UD Tel: 0895-59826
Mercia Software Ltd.	Gareth Brentnall	Aston Science Park, Love Lane, Birmingham B7 4BJ. Tel: 021-359 5096
MetaTechnics Systems	John Stenbridge	Unit 216, 62 Tritton Road, London, SE21 8DE Tel: 01-670 7959
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 01-922 8866
M.T.I.	Ray Cannon	7 Pine Wood, Sunbury-on-Thames, Middx. TW16 6SH Tel: 09327 80848
Parallax Systems Inc.	Kevin Weaver	60 West 9th Street, New York, New York 10011, U.S.A. Tel: 212-475-4001
Peter Cyriax Systems	Peter Cyriax	213 Goldhurst Terrace, London NW6 3ER Tel: 01-624 7013 (Answerphone) 0860-377963 (Mobile)
Portable Software	Richard Smith	60 Aberdeen Ave, Cambridge, Mass. U.S.A. 02138. Tel: 617-547-2918
QB On-Line Systems	Phillip Bulmer	5 Surrey House, Portsmouth Rd Camberley, Surrey, GU15 1LB. Tel: 0275-20789
The Rochester Group	Robert Pullman	164 Pinnacle Rd., Rochester NY 14620 Tel: 716-454-4641
Shandell Systems Ltd.	Maurice Shanahan	12 High Street, Chalfont St. Giles, Bucks HP8 4QA. Tel: 02407-2027
I.P.S.A. Ltd.	Mike Butler	7th Floor B Block, Coventry Point, Market Way, Coventry CV1 1EA Tel: 0203 256562
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kika Place, Kailua, Hawaii 96734 Tel: (808) 261-7536
Tektronix UK Ltd.	Paul Morgan	Fourth Avenue, Globe Park, Marlow, Bucks SL7 1YD. Tel: 06284-6000
Uniware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France Tel: (1) 45-27-20-61 Fax: (1) 45-27-20-71 Telex: 648348F UNIWARE
Warwick Univ.	Dr Andy Pole	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel: 0203-523066 x2158

Reviews Section

introduction by Adrian Smith

Thanks to Dick Bowman and Nick Small for two very well matched reviews of IBM's APL2 for the PC and PS/2. Oddly enough, both reviewers seem to have arrived at very similar conclusions, although I can guarantee that neither has yet read the other's review!

In the next VECTOR we will have side-by-side reviews of the AFM File System for APL2/PC and the Myriade file manager for APL*PLUS/PC. These products are briefly described on page 44.

Software Received for Review

by Adrian Smith

AFM for APL2/PC

AFM/PC is a single-user version of the AFM file management system which Interprocess Systems Inc. has provided for IBM mainframe environments since 1978. It supports nearly all of the AFM file commands available on the mainframe, and emulates the mainframe library structure on the PC. Both keyed and component files are supported.

AFM/PC works with DOS 3.1 or later, and with APL2/PC 1.0 (or later) in either 16-bit or 32-bit modes. It is priced at \$175 with volume discounts available. For more details, contact Stella Chamberlain on (404) 992-8400, or write to Interprocess Systems, 9040 Roswell Rd, Suite 690, Atlanta, Georgia 30350-1131



The AFM Stand at APL89

Myriade File System for APL*PLUS/PC

The Myriade File Manager improves upon the functionality of APL*PLUS/PC component files by providing numeric or character keyed access, multi-variable components and efficient space reclamation. Components can be created, read and updated in any order.

Myriade can handle very large files (up to 192Mbyte under DOS) with over 2 million components: there is provision for effective management of the file space, and for crash recovery on startup. Full and incremental backups can be taken.

The latest release of Myriade allows 64K of the system to be loaded into EEMS memory, which practically eliminates the overhead on workspace.

MFM Version 1.0 for APL*PLUS/PC sells for 375 CAN\$ from:

Myriade
3652 Ch St Louis
Ste-Foy, Quebec
Canada G1W 1S9
Tel: (418) 658-1275

Product Review - APL2/PC

reviewed by Dick Bowman

IBM Product 38F1753 (APL2 for the IBM PC Version 1.0).
Price £348, available now from authorised IBM dealers.

Background

Having used APL2 in a mainframe environment for several years I've tended to feel a little reluctant to do PC-based development in the recent past; I value the expressive power of APL2's language enhancements and am frustrated when these things become unavailable to me. I think that one of the essences of APL is fast problem-solving, and the restrictions imposed by the implementations hitherto available in the PC environment can be very real ones. Depending on your particular application they can outweigh the benefits of improved user interface and so forth which are achievable on the smaller machines.

The product at hand is therefore exceptionally welcome; it offers the opportunity to use APL2 on your PC or PS/2 at a reasonable price (and I firmly believe that we have to see APL offered at prices comparable to those of other languages).

Hardware Requirements

In brief, any sensibly-configured PC or PS/2 should be satisfactory; this review was performed using a 640k AT equipped with EGA adapter, monitor and 80287 coprocessor. The product is supplied on both 5.25 and 3.5 inch disks. In fact you also receive two versions of the interpreter, one for 16-bit hardware, the other for 32-bit; regrettably I was unable to investigate the latter.

Does It Feel Comfortable?

Subjective impressions are hard to quantify, but important; in many ways this is the main question I was looking to answer. What I wanted was a sense that unless I deliberately intended to use specific machine features I should be unconcerned about whether I was using APL2 on a PC or a mainframe system (my reference frame is APL2/370 Release 1.3, running under TSO on Amdahl hardware).

Installation was straightforward, to start APL2/PC there are two steps (easily automated)...

- a) Load the APL2 character font; an option provides a 43-line display if you use the EGA adapter - I found it acceptable.
- b) Decide which Auxiliary Processors are needed for your session and start APL2 with this set. In fact APs may be added and discarded at will - and there are workspace size benefits from using only the necessary subset. As you correctly surmise, APL2/PC follows the IBM party line about using APs to provide facilities outside the core language; which is alright by me, but it is definitely different from `EVERYTHINGUNDERTHESUN`.

Once started there's a very strong impression of responsiveness which contrasts most favourably with TSOlag. Keyboard layout is 'IBM normal' and how you take to this depends on how conditioned you are to expect specific layouts on specific machines - if it's a problem you can reconfigure the keyboard as necessary; with all of the 'new' APL2 symbols present the default layout is reasonable although I'd very much like to see an agreed keyboard once more (both the IPSA 'union' and STSC 'unified' have their merits, even though they differ from one another). Expected, present, and welcome is a type-ahead facility - something which is annoying by its absence on APL2/370.

You have a choice of interface style; either 'PC-normal' with a continuous scroll or a session manager format which is a close approximation to that of APL2/370. Given that the latter is preferable in my accustomed mainframe context I used it most - although given a fully free choice my ultimate wish-fulfillment would be continuous scroll in both environments. The main irritation I get from the discrete-page style of the session manager being that the most interesting bit is always at the bottom of the page; on the mainframe PF8 is an escape valve which allows you to force a page throw. This doesn't work properly with APL2/PC.

Another (minor?) annoyance stems from my preference to use the non-default DISPLAY HOLD OFF option so that long output sequences come to their natural conclusion without my personal intervention. Although the session manager has a compatible look-and-feel to the mainframe version implementation is different and it's not possible to save a personal profile; what you have to do is to set the options up as you begin your session (again, you can automate the process).

Having made contact with an unfamiliar APL system my first instinct is to type `)LIB` and see if there are any interesting toys to play with; APL2/PC provides a novel response in that the display resembles 'DOS DIR' format rather than the conventional APL style - there's more information presented, but it does tend to overflow the page. A full set of workspaces come with the product in the accustomed IBM manner - for the sake of familiarisation I looked at two of them...

AP124An emulation of the obsolete mainframe Auxiliary Processor 124 for alphanumeric panel use is accessible via this workspace. All the facilities you were accustomed to in the days of VSAPL are present, augmented by functions FSSCAN and F\$INKEY providing the immediate response so necessary for PC applications (indeed, all applications) and analogous to STSC's INKEY function. I didn't have enough time with the product to explore the discrepancy between this workspace offering only a 25-line display and the clear evidence that 43-line support is achievable - possibly a documentation problem.

GRAPHPAK ...Another old faithful rears its head; the demo worked fine except that the documented instructions were missing. A major disappointment was that graphics were restricted to CGA compatibility.

At this point I felt that I had established that an APL system existed, it had a fairly familiar feel and seemed subjectively quite fast; the logical next step was to quantify the performance.

Duz APL 2PC Run Fst?

I have two reference points for performance; STSC's APL*PLUS/PC (Version 6.0) and mainframe APL2/370; using the customary benchmark set (see VECTOR Vol. 4 No. 3, p. 40) the following results are achieved...

	STSC	APL2/PC	RATIO	APL2/370
Z←+/VI	17	27	1.588	.1
Z←v/VL	1	0	.000	.1
Z←[/[1]MI	11	16	1.455	.1
Z←VI*.1	197	258	1.310	2.6
Z← VR	34	27	.794	.1
Z←VR[VI[120]]	5	11	2.200	.1
Z←VI[▽VI]	43	61	1.419	1.2
Z←-2 1↑MR	12	5	.417	.1
Z←VIεVI	55	6	.109	.8
Z←2 1QMC	22	132	6.000	.2
Z←VC○. =VC	38	34	.895	.8
Z←(150)○. +150	154	121	.786	.8
Z←VR . +VR	98	99	1.010	.2
Z←MR⊞10↑VR	82	94	1.146	.7
Z←FIB	1170	1395	1.192	14.5
Z←VR×3.14	49	38	.776	.1
Z←VR÷3.14	61	50	.820	.3
Z←⊙VR	89	82	.921	.7
Z←10VR×.1	224	231	1.031	1.8
AVERAGE			1.256	

All times are in milliseconds; with a clear workspace for both systems $\square$ WA is 357152 for APL*PLUS/PC and 335792 for APL2/PC. I think this is quite a reasonable performance; I also include the mainframe figures for their academic value - they show quite clearly that big boxes really are fast once they get going (it's just a pity there has to be so much hardware between you and the CPU to make the response time bad).

So much for 'standard' APL; to date I haven't seen many benchmarks for the newer features and I offer the following as a starter set...

```
NEST←2 2ρMC VI VR
```

```
(2 2⊖NEST)←cNEST
```

	APL2/PC	APL2/370
Z←VR=ϕVR	83	.1
Z←>c[⊖IO]MC	241	1.5
Z←εNEST	1917	1.6
Z←5+/VR	620	.4
Z←ε\''VI	2197	13.1

A set of results which I find somewhat less gratifying.

Much time and effort can be spent on benchmarks - I don't think any of the above are without flaw; I know of other people who've looked at the IBM product and produced better figures for it, no doubt STSC can come up with their figures as well. Add weighting factors to taste. For my purposes I'm happy that APL2/PC is at the very worst comparable to the well-established APL*PLUS/PC product.

A Small Application

Undaunted, the next task was to look at a small application and assess any problems in its implementation; just to make a change from the routine this was an application which takes data from a digitising tablet and converts to Ordnance Survey grid references. An incarnation using APL*PLUS/PC has been in use for several months, so the requirement is well understood - the questions being how closely would the two versions resemble one another and how 'usable' is APL2/PC.

Function editing is the first thing to sort out - APL2/PC has an inbuilt editor which is fairly rudimentary, but a pair of alternative fullscreen editors are provided within one of the supplied workspaces. Most of my time was spent using one of these (EDIT2) - it provides an excellent set of edit capabilities. Nevertheless, it has an inherent problem deriving from its own nature as an APL function; if you edit a suspended function you resume execution with the function as it was before you edited. Which is not particularly

good news if you're trying to do some serious debugging; APL2/PC sorely needs a fullscreen editor like the mainframe)EDITOR 2 (which for all of its defects at least lets you change the suspended function and resume with the new version). Even though EDIT2 is reasonably fast time-lags are noticeable and one might expect an intrinsic editor to be faster.

The next 'problem' is language incompatibility with APL2/370; there are a number of problems here which I regret, in particular

- a) Restrictions on the use of axis specification
- b) Selective specification isn't implemented at all

I find the latter most intrusive, because it tends to run naturally with the index function (which is implemented) and stylistically I would prefer to live without bracket indexing.

The application in question accessed a digitising tablet attached to the AT's COM1 port, which led to quite different code detail. The APL*PLUS version takes its input using `⎕ARB IN` statements; APL2/PC uses AP232 (in my case via the supplied cover functions). Once acclimatised to the different style everything's fine and works well - but I can't underemphasise that this is not just a funny version of the STSC product and that this has to have some influence on your detailed coding. I don't think either product has a monopoly on 'good' - they're just different and if you can't personally live with that then too bad. There was originally a problem in getting started, documentation and workspace differing from one another about setting parity options; the resulting error messages are none too helpful. I ran into this generic problem rather more frequently than I would have liked to, I feel that someone ought to smooth out these differences.

Another minor problem encountered with the AP120 session manager was that sometimes overtyped entry wasn't fully re-executed; most of the time it was alright, but occasionally it wasn't.

Graphics

Let's not beat about the bush; unless you have the readies for extra hardware like the 8514 monitor, or extra software like the Graphics Development Toolkit, APL2/PC graphics capabilities are pretty grim. Grim as spelt C-G-A.

I have a personal bias toward graphic applications, so I felt that after several years of at the very minimum EGA systems, this was more than a little miserable.

What there was worked fine, and if you don't need graphics it won't be very upsetting to not pay for what you don't want. But, if you do, then be ready to spend some more money. As the Enid Lumley of APL I was unable to explore the more exotic alternatives.

What Can/Should APL2/PC Be Used For?

So far, this might all seem grim reading; but I feel positive about the product - I'm sure it fills at least a niche, and probably a quite large one.

So far I've not commented on what I think is the most important innovation APL2/PC brings us - grandly called the Application Package Maker and with a description furtively hidden on pages 347-348 of the manual. What it does is very simple; it takes your APL workspace and converts it into an executable DOS file. Which means that whenever anyone calls you and wants a one-off solution to a computing problem, if you can come up with an APL2 solution then you can package it and ship it out with no fuss or ado about them having to have an interpreter of their own, or a run-time version, or indeed anything that costs money. Economies of scale were always good for multi-copy run-time APL applications, what IBM have done (and, to be fair, MicroAPL before them) is to make APL realistic for the cottage industry.

Another area I see particularly attractive is multi-environment systems; now you can use essentially the same APL on both the mainframe and micro components of the system. I can live with the niggles, if it means I can use the same utility code everywhere.

An option I find surprisingly less attractive is use of the PC as a local responsive development environment; I think the differences from the mainframe product might be more of an obstacle here. I suspect that a great deal of mainframe APL also has to concern itself with interfaces to other systems and languages which by their nature are not present on the PC; also the relatively poor editing options of the present APL2/PC product for me outweigh any gains in responsiveness. Also take a look at those APL2/370 timing figures if you're embarking on heavy computation.

Other aspects of APL2/PC which could be factors are AP488, which provides an interface to GPIB/IEEE488 hardware, and what looks to be a quite detailed explanation of how to write your own auxiliary processors.

What Improvements Are Desirable?

The most irritating aspect of APL2/PC ought maybe to be the easiest to achieve improvement with; there are too many places where the manual and the reality are inconsistent.

It seems to me that one of the most positive aspects of STSC's APL*PLUS/PC product is that it has a support structure; this would appear to be a major factor for success and I'd like to feel that IBM could set up a scenario by which APL2/PC queries could be handled promptly and easily (to some extent this also has to come from the users themselves). In all fairness my own problems were always dealt with in an exemplary manner, but I'm a little worried on behalf of Tom and Harry.

Graphics and editing are glaring weaknesses; the former may not concern everybody - but I definitely feel that both need attention.

And for a final shot, I'd like the gap between APL2/370 and APL2/PC to become smaller.

Conclusions

This review may sound negative - perhaps that's the nature of such things. But I wouldn't like to leave you with the impression that I condemn the product out of hand. Far from it; I enjoyed using APL2/PC, I intend to switch the PC end of my applications onto it, I look forward to sending out run-only APL applications without an interpreter order form. And above all, I look forward to having some of those end-users decide to try APL themselves because they liked the flavour of what they got.

APL2/PC (32-bit) - a Brief Review

by Nicholas Small, National Westminster Bank

Introduction

This offering from IBM, based on its mainframe APL2 interpreter (APL2/370), comes with 18 device drivers, 29 workspaces, two interpreters (16 bit and 32 bit) and a packager - hence the word "brief" in the title of this report.

Documentation consists of the APL2 language Reference manual (as distributed for APL2/370) and a 300 page manual specific to the PC. The former (the release 3 version) is very clear and easy to use, as reference manuals go, and its presence indicates the extent to which APL2/PC follows APL2 on the mainframe.

Indeed, when running with the full-screen APL2 session manager, which is almost identical to that on the mainframe, APL2/PC has a very familiar appearance, but leaves the habitual PC user longing for enhancements, in particular in respect of cursor movement and scrolling through the session log.

The testing for this review was carried out on an IBM PS/2 Model 80-071 with 2 Mbytes of memory and a VGA monitor, running IBM DOS 3.3, under which there was 563 kbytes of memory free of the 640 kbytes accessible. Most of the testing was performed using the 32 bit version of the interpreter.

Installation

A batch file is supplied to perform the installation; the documentation does not mention it, but installation is onto drive C:, so a quick bit of editing is required if the desired destination is some other drive (though a well-written batch file would have allowed parameter substitution).

Specimen batch files APLON and APLOFF are supplied to assist in the provision of an APL character set, for EGA, VGA, or PC Convertible monitors, and a number of printers. These use code page switching, so DOS 3.3 or later is required; the documentation describes how to modify AUTOEXEC.BAT and CONFIG.SYS for those not already using this facility. There is also a program APL2FONT for those unable or unwilling to use code page switching.

Clearly these bits and pieces need to be put together in a batch file that will be run to invoke APL2/PC. Example files are given, though they fail to include the command

CALL on the line ca'lling APLON (and ditto APLOFF), without which the batch file will terminate prematurely.

All that remains is to select a few likely auxiliary processor names to put on the line calling the interpreter, and away we go!...

... but to where? Once the initial excitement has worn off, one wonders how one might put values on function keys, to replace the defaults, and how to set up a few libraries. Here the documentation lets one down a little; a couple of references to the supplied workspaces PROFILE and EDIT, with a pointer to the use of)IN would save a lot of searching.

It was at this point that I met my first problem: when using one of the supplied functions for full screen editing, there was a one character delay between keyboard action and the updating of the screen. Having managed to extricate myself from the grasp of the editor, I set about investigating the matter. After rebooting from a DOS disk there was no problem; a few more rebootings revealed that the cause of the difficulty was the IRMA2 keyboard driver and keyboard macro driver. The same problem arose if in my AUTOEXEC file I used the DOS command KEYB to select the UK keyboard.

Differences between APL2/PC and APL2/370

Some ten pages of the PC documentation describe differences between APL2 on the PC and what is described in the APL2 Language Reference manual. Most of these differences fall into one of the following categories: deviations from APL2/370; implementation restrictions; system dependency.

The deviations relating to fill items and formatting may be irritating to anyone wishing to develop applications for a mainframe on the PC; other deviations are unlikely to cause any problem.

Of the restrictions, the following are the most significant:

- (i) No complex arithmetic (a workspace is supplied to emulate this).
- (ii) Axis specification on scalar functions and partition is not implemented.
- (iii) Selective specification is not implemented.
- (iv) Name association is not implemented, though a similar facility is available through AP2.
- (v))CONTINUE does not save the workspace, but acts like)OFF. (This seems dangerous - why not disable the command?)

- (vi) There are no facilities for setting the size of the workspace.
- (vii))EDITOR is not implemented. (It is a disgrace that there is no full-screen editor built in.)

The system dependent differences include a system function to perform peeks and pokes.

It should also be noted that storage for integer variables is only two bytes in the 16-bit version.

Auxiliary Processors

I had neither facilities nor time to investigate all the APs provided in the package, so the following sections give a brief idea of just some of them. For information, here is a complete list of the APs:

AP2	Interface to non-APL programs
AP80	IBM graphics printer control
AP100	PC/DOS host command interface
AP101	Stack and profile manager
AP103	BIOS/DOS interrupt handling and I/O port control
AP120	Full-screen APL2 session manager
AP124	Full-screen display management
AP172	IBM PC Network communications
AP190	IBM PC 3278/79 communications
AP190I	IRMA PC 3278/79 communications
AP190P	IBM 3270 PC, PC/G or PC/GX communications
AP206	Graphics processor
AP210	DOS file management
AP211	APL2 object library management
AP232	Asynchronous communications
AP440	Music generator
AP488IBM	IBM GPIB/IEEE488 communications
AP488NI	National Instruments GPIB/IEEE488 communications
AP998	Logic processor

There is at least one workspace to assist in the use of each of these APs. In my limited experience of these, I found them inadequately tested and generally short on comments and lacking in general description, but containing plenty of useful functions.

Should none of the above APs meet your requirements, create one yourself, following the instructions in the chapter entitled "How to Write an Auxiliary Processor".

A maximum of 15 APs may be loaded at any one time, though they may be dynamically loaded and unloaded via AP2.

AP2Interface to non-APL programs

The manual gives an example of how this may be used to invoke the DOS command FORMAT. I was unable to persuade this to work as described, invariably getting from DOS the message "Format failure" at the first attempt to run the program and "Invalid parameter" on subsequent attempts. There is a supplied workspace to assist in producing FORTRAN subprograms to use with APL2 functions.

AP101Stack and profile management

It is easy to edit the functions in the PROFILE workspace to set up function keys and define libraries, as required.

AP120Full-screen APL2 session manager.

This is a bug-ridden attempt to emulate the mainframe APL2 session manager, which will leave users who are accustomed to the speed and flexibility of the PC longing for facilities for key-controlled scrolling through the log, for rapid cursor movement, and for recalling previous commands.

Most of the bugs are mere irritations, but be warned not to try to alter the size of the log from the default 8k.

Another problem arises when one executes a DOS command using)HOST dos_ command or starts a DOS session using)HOST In the former case, output from DOS is splattered over the session manager screen (disappearing when control returns to APL2 and Enter, Page Up or Page Down are pressed), whilst for the latter the session manager screen is left as a background.

AP124Full-screen display management

This is IBM's FSM facility. An important deficiency from the point of view of a PC user is the absence of any means to constrain the movement of the cursor to specified regions of the screen.

Only one pair of shared variables is allowed, which means that one cannot readily perform full-screen editing for the dynamic debugging of functions using AP124.

AP210DOS file management

Facilities are provided for the handling of DOS files, whether for holding APL objects, bit strings, or ASCII with CR/LF delimiters; both direct and sequential access may be used. These appear to work well, and may also be used to send output to a printer, by treating the DOS device LPT1 as a file.

One omission is a facility to determine the size of a file. This means that native files cannot be read as a chunk with a single DOS call; instead, they must be read a line at a time, with repeated DOS calls, which is slow.

The only problem I noted was in the failure to deal adequately with the DOS end-of-file marker. This is exemplified by the function `READFILEV` in the workspace `FILE`: when reading an ASCII file with the usual CR/LF delimiters and terminated by an end-of-file marker, the lines of the file are returned as elements of a vector, but the last element is the last line of the file with its first character replaced by the end-of-file marker. The cause of the problem is the failure of the direct read facility of the AP to detect the true length of a record that is not terminated by a CR/LF pair.

Supplied Workspaces

A few comments follow on some of the supplied workspaces that I sampled in addition to those described in the previous section.

`DEMO206` this demonstrates what can be done with `AP206`, illustrating medium resolution graphics and including an animation effect.

`DOSFNS` this contains functions to carry out various DOS commands, such as changing drive or directory, via `AP103`. The function `DATE` illustrates how not to use backspace in conjunction with quote-quad input - the current date is displayed on the screen, but is not read as input if Enter is pressed.

`EDIT` this contains a number of functions for editing APL functions and variables. There is some lack of consistency between these functions, however: function `EDIT` uses F3 to save and end, whilst `EDIT2` uses F2 to save and F3 to quit (also, F3 always asks "Quit without saving?").

Keyboard Problems.

To switch between the APL keyboard and the National keyboard one uses Ctrl or Alt with Backspace. I experienced a number of problems with this facility, the chief being that, apparently unpredictably, I would get locked into switching between two graphics keyboards.

Another difficulty arises when invoking DOS or a DOS application via the `)HOST` command: one finds that the APL keyboard is still active, and that certain key combinations (e.g. those using Ctrl) are not recognized. Pressing Ctrl- or Alt- Backspace may solve or alleviate the problem. Sometimes pressing the Caps Lock key causes the

Num Lock light to come on; pressing Num Lock seems to stop this, though the Caps Lock light may end up at the wrong setting.

When one is in APL2/PC Caps Lock has no effect, and Num Lock does not activate the keyboard indicator.

When the APL2 session is terminated and one returns to DOS, the keyboard sometimes behaves as if Ctrl is held down; usually, pressing Ctrl-S twice restores normality.

Packaging Workspaces.

This facility is available only for the 16-bit version of the interpreter. It enables one to convert workspace applications into executable (.EXE) files, making it possible to run them on machines without an interpreter (or a run-time licence). It works well - need one say more?

Workspace Transfer.

The system commands)OUT and)IN, to write and read APL transfer format files, make it easy to transfer workspaces between the 16-bit and 32-bit interpreters.

Performance.

On my machine (with 2 Mbytes of memory), the 32-bit interpreter gives me a clear workspace of 1.4 Mbytes, nearly 0.5 Mbytes more than I get in APL*PLUS II. For basic arithmetic and function execution, however, APL2/PC is up to 50% slower.

The 16-bit version gave me a workspace of 378 kbytes, some 7 kbytes less than APL*PLUS/PC V7, which ran a little more slowly.

Conclusion.

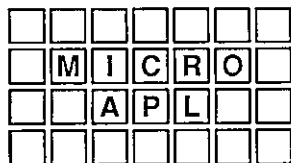
The overall impression of this system is that it is very powerful, but that a lot of elbow-work is required to make it into a really polished product.

As the documentation stands, this would be a difficult product for a first-time APL user, though a brief, well-written chapter directed at beginners could make all the difference (and be of use to the rest of us!).

MicroAPL for ...

APL and networking consultancy

MicroAPL has 10 year's experience with small-computer APL systems. We have pioneered the usage of networked microcomputers and can plan, specify, install and maintain PC networks of all sizes. We have experience with both Ethernet and Token-ring networks and their mainframe gateways. Our programming skills cover all areas of APL and systems programming. MicroAPL can also train and supervise your APL staff on your behalf. If you would like to review your systems requirements, contact David Eastwood at:



MicroAPL Ltd.
South Bank Technopark
LONDON, SE1 6LN
Telephone: 01 922 8866

MicroAPL for ...

APL on the widest range of micros

STSC's APL*PLUS Release 8
STSC's APL*PLUS II PC

£425
£1395

APL.68000

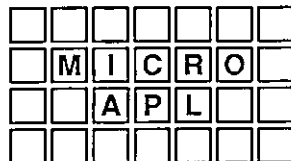
APL.68000 for the Mac, ST, Amiga
APL.68000 for the Mac II, SE30

£86.91
£260.82

APL.68000 runs on many multi-user computers, including:

Sun, NCR, Wicat, HP200/300, Spectrum, Aurora, Stride £1200 to £2000

All prices quoted exclude VAT



MicroAPL Ltd.
South Bank Technopark
LONDON, SE1 6LN
Telephone: 01 922 8866

Access and Barclaycard
Orders accepted

RECENT MEETINGS: APL 89

This section of VECTOR is intended to document the seminars given at recent meetings of the association; it is of particular value to members who live away from London. It also covers other selected events which may be of general interest to the APL community.

If you would like to speak at one of the regular British APL Association seminars, please ring the Activities Officer (address on inside back cover) who will respond enthusiastically to your offer.

Contents

Conference Roundup	Anthony Camacho	60
Statistics Tutorial Day	Chris Brady	65
Conference Summary	Peter Merritt	66
Detailed Notes on the Papers	Camacho, Searle et al	70
Tutorial Sessions	Jonathan Martin	101
The Exhibition	photos by David Ziemann	106
The Bonfire of the Variables	John Searle	108

An APL 89 Roundup

by Anthony Camacho

Thanks for a Great Job

The volunteers who ran APL 89 deserve our gratitude for the great job they did. Running a conference such as this is an enormous amount of work and every volunteer, when stretched to the limit, then discovers six extra things to do which would make all the difference. I don't think they could have worked much harder than they did. The conference ran smoothly and the hitches were few. Many things were really well organised. At the debriefing we learned of some crises that had to be handled at the last minute: it is amazing how little these showed.



Lynne Shaw (programme chair) at the Opening Session

The Invited Speakers

The invited speakers were marvellous. Tom Gilb, Stanley Jordan, Arthur Whitney, Stephen Wolfram and A. K. Dewdney each produced something special for us. Dewdney showed a video of part of the Mandelbrot set zooming into ever finer detail! These speakers were thought-provoking, entertaining and well-prepared with excellent material.

The Boat Trip Round Manhattan

The boat trip was an excellent idea well executed; it was on a large boat which could carry the complete conference and all hangers-on with ease, leaving plenty of space to move around, there was ample food and drink and nothing to do but talk to congenial people and watch the skyline and shore of New York as it gradually went dark.

The trip started at 42nd St wharf, went south round the statue of Liberty and up the east river but turned before the first bridge (Brooklyn) and then went north up the main river and turned just below George Washington bridge and returned to 42nd St. There was even one compartment with a Loud Noise in it (allegedly music) for people who preferred not to hear what their companions said! There was real music too - a barber-shop group who sounded great and who seemed to be having great fun. Getting such a large party fed

with food and drink without excessive waiting in line was a problem; it was not really solved but as there was two and a half hours to feed oneself in not even the most diehard line avoiders had to go hungry. The occasion was a great success and rounded off the first day of the conference on a high note.

More Printed Material Than Ever Before

The conference proceedings were supplemented by a tutorial volume. There was a separate book containing the Statistics day tutorial. The amount of written material a visitor to APL 89 could take away with him was more, I suspect, than was provided at any other APL conference. Lynne Shaw told me that the amount of work done to raise the standard of the typography and layout was greater than ever before and certainly the results are more uniform than some proceedings have been.



Per Gjerlov (chairman APL90) with some of the Danish Beer the Danes gave us for the boat trip

Panels and Tutorials

Perhaps I was unlucky with the tutorials I attended. Tutorials and panel sessions are very easy to do badly. I was disappointed by every panel in the conference, even the panel after Arthur Whitney's talk about 'A'. It takes a lot of effort to co-ordinate what various panellists are going to say, and if there is no such planning what too often results is words filling time. Some of the panellists were being recruited as the delegates arrived and the results were what you would expect - inadequately prepared contributions which failed to fit together well.

Rather a Sad End

The closing session was a case in point. The session was called 'Thought, Structure and APL' but the three contributions were independent and unlinked. Each of those speakers can hold an audience spellbound for an hour and a half. None of them did himself justice (and these were recruited months beforehand). No doubt each had material prepared for a longer talk. The time allowed was shortened at the last minute and the result was banality or incoherence. Rather a bathetic end, I felt.

The Self-service Banquet

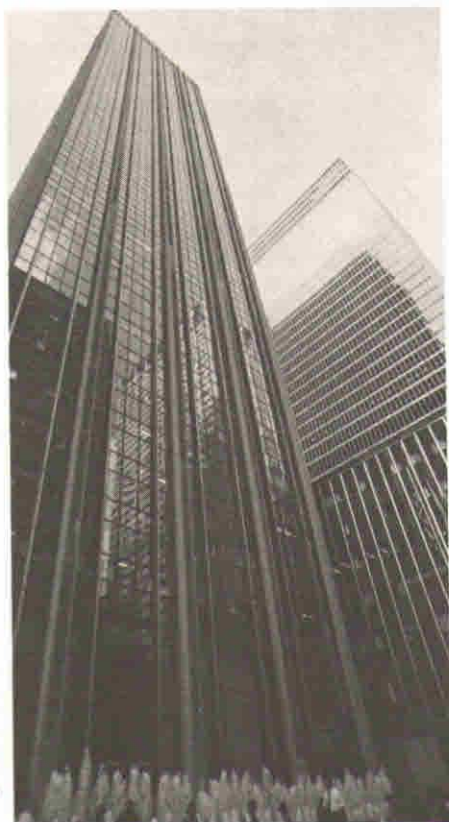
The banquet was self-service and there was not adequate control over when you joined the queues; the consequence was that the queues were too long. It had been intended that there should be a choice of flavours - there was Italian food, Chinese food etc. on offer. This was not announced nor clearly signposted so what you got depended on which queue you happened to join. It turned out that each queue led to a mixture, which in my case tasted good. There was no drink supplied as part of the banquet - we had to pay Hilton prices for a bottle. Many of us felt that the Hilton skimped rather on the banquet. It also skimped on other things. On the first day the coffee arrived with china cups and saucers. By the end of the conference we had grown accustomed to polystyrene beakers.

The Hilton

Hilton prices were too high: apart from a share in that bottle of wine and two beers, I bought no food or drink at the hotel. Instead we walked round the corner to a delicatessen for breakfast, lunch and dinner. Looking back on this I realise that eating together at other conferences brought the delegates together and gave more opportunities to meet and talk in a relaxed atmosphere - I missed this at the Hilton. We also bought wine; the Chandos Californian champagne from a Lexington Avenue shop at under \$10 a bottle was \$45 a bottle in the hotel. We carried it back and drank it out of tooth mugs after cooling it with free Hilton ice. I'm sure this way it tasted better.

The Conference Luncheon

There was a conference Lunch as well as the banquet. I applaud this innovation. The meal was served - no queuing. The food was good and the after-lunch speakers were first rate. APL is on the whole a very visual notation and it was a pleasant change to be asked to appreciate results through one's ears.



Fountains near the Hilton

Will APL 89 help to spread the use of APL?

What will APL 89 do for the growth of APL? Not many converts to APL will have been made. A few students attended and perhaps the Statistics day will have engaged the enthusiasm of some non-APLers. A conference at a big hotel which targets the company employee and tries to provide sufficient value to justify the cost of sending delegates at company expense, a conference that relies on income from a vendor exhibition and must try to deliver professional users to visit the stands and consider buying the products, such a conference can neither attract the uncommitted nor the poor (which in England includes most teachers) nor is spreading APL likely to be in the forefront of the minds of its delegates. Even the vendors' undoubted aim to sell more APL can only be pursued in such an audience to a limited extent; most delegates already have an APL and for most the cost of a change would be high.

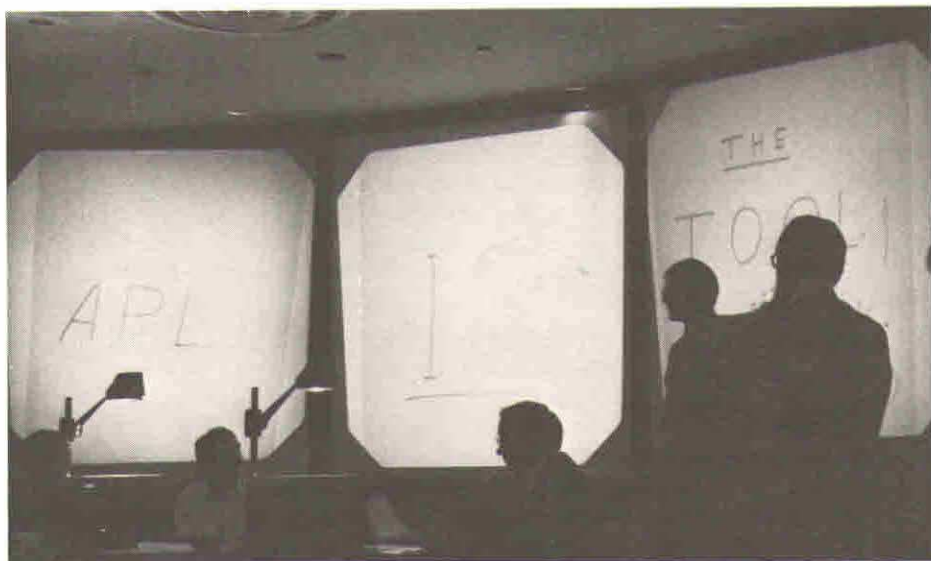
More Free Interpreters: now we must get them into use

The only initiative at APL 89 that might have a significant effect on the number of APL users was the launch of TryAPL2 - IBM APL2/PC as freeware (but without many auxiliary processors). To obtain such an effect the APL world will have to make a concerted effort to help and encourage teachers to try using APL with their pupils. This is a much harder task than the production of free APLs (as I have reason to know).

Pleasant Memories

What was memorable about APL 89? Apart from the highlights mentioned above I shall remember with pleasure:

- Phil Benkard's presentation on three overhead projectors simultaneously with David Landaeta, David Livingstone and Jim Brown as the foil changing machinery,
- Linda Alvord's Willie-the-snail who wanted to be an architect,
- Charles Waters' paper from the 'Institute of Geometric Entomology' at Scotch Plains-Fanwood school making observations on flight behaviour in the butterfly,
- The little torches with colour filters and light-guides which STSC gave away at their reception,



3xOHP Foil Changers! Silhouettes of David Landaeta; David Livingstone; Jim Brown; Phil Benkard (and someone standing in the photographer's way)

Last but not least Tangible Math and TryAPL2. Tangible Math is Sharp APL/PC with a workspace that sets error traps to provide help of many kinds and a booklet introducing APL by Ken Iverson. TryAPL2 is IBM's APL2/PC complete but without auxiliary processors. Each of these gives large workspaces and enhancements to APL. Both can be copied and given away. No PC-owning maths teacher should be without these products.

The Statistics Tutorial Day

Reported by Chris Brady

The Statistics Tutorial was of a very mixed standard in more ways than one.

The morning sessions were intended to be an elementary guide to statistical techniques but were marred by the hi-tech video projection equipment not locking on to the PC-generated graphics displays. These were prime examples of the need to have a low-tech back-up of slides or foils for when hi-tech things go wrong. Also the early sessions lost momentum (and many people's interest) with the use of too many complicated models (plastic fruit joined by dowel rods to illustrate the concept of stratification in populations - I think!!). Too much time was spent in constructing these during the presentations.

As is usual at conferences the published papers are more informative than were the actual presentations. The first was entitled "Statistics: An Inductive, Conceptual Approach", and the second was "An APL Sampler: A Tool for Illustrating the Central Limit Theorem". This latter paper could be of interest to many end-users of our APL systems.

The afternoon sessions were much better presented and aimed at a higher level of understanding. The papers presented were entitled "Graphical Techniques in Statistical Data Analysis and Inference", "Statistical Analysis of Two-way (and Higher Dimensional) Tables", "Statistical Linear Models Using APL", and "A Statistical Symmetry".

The first was not specific to APL but did illustrate a number of ways of graphing statistical data. The second could be of interest to end-users and developers of our APL systems, especially those in marketing and reservations. It is mainly about the analysis of multi-dimensional contingency tables of frequency data, for which APL is admirably suited. APL expressions for Chi-squared and Fisher's tests are given as well as for more sophisticated tests.

The third paper was about estimating the parameters of a linear model, and hence the 'signal', given a number of observations affected by 'noise' (or random errors). APL may be used very effectively for this since the least-squares regression operator (domino) is powerful and easy to use.

The "Statistical Symmetry" paper was about generating samples from various distributions using random number techniques, which can then be used for simulations and modelling.

Conference Summary

by Peter Merritt

The 89 conference in New York was to be my second exposure to large numbers of APLers in a confined space, the first being our own (that is, British) attempt at Manchester in 1986. I had really enjoyed myself there, with its fascinating series of talks and product displays, although equally enlightening was the opportunity to meet and talk with so many other like-minded, slightly crazy people. I'd had to miss Dallas in '87 and Sydney in '88, but a new job with more committed employers provided both the financial and professional impetus needed, so, together with an old colleague (Simon Barker, now of BUPA), I duly sent off my registration and hotel booking form (taking care not to think about the cost of staying at the Hilton...).

My conference began like the bulk of the other attendees with the Monday morning opening plenary session which was to be given by Tom Gilb (for reports on the Saturday talk/welcome and the Sunday statistics tutorial, see elsewhere in this issue). The talk centred on the development of a formal planning/specification language which seeks to express such vague but essential areas as user requirements, test criteria, cost/benefit tradeoffs etc., in a more quantifiable and structured format. Thus, the usual request for "a more efficient warehouse" would be broken down into 'statements' such as "OBJECTIVE: SHELF SPACE UTILISATION 91-95% [Objects less than 10Kg.]". The performance or characteristics of the existing problem (on which design decisions would be made) would again be broken down into smaller statements which would be easy to identify later, such as "PAST: USER LOAD ON STOCK SYSTEM 120 MAX[Average 85]". Or, later on, "TEST: USER LOAD ON STOCK SYSTEM[Development: 10 staff, 2 locations]". The three components of each statement represent the general statement type, the aspect of the problem to which it refers, and (in the square brackets) any refinement or qualifying statement. Comments may be appended to any of the statements at the end of the line. Another good feature was the introduction of a "SOURCE" statement type, in which specific book/chapter/paragraph references are given at the point at which such information may be used to justify a design decision, rather than in the form of footnotes or the almost totally useless list of titles in some forgotten appendix.

The development and use of this planning language has meant that, for Mr Gilb's systems at least, all requirements are stated and fully defined, both in terms of cost and specific benefits etc. Critical information on existing procedures/factors is stated and the source of that information listed with it, and so on. In this way, Mr Gilb has gone beyond normal systems analysis (which has always seemed obsessed with computer program design, at the expense of full problem definition) to such an extent that his specs have been used as the basis of legal contracts for the development of systems. Despite the fact

that Mr. Gilb, a Software Engineering specialist currently based in Norway, did not actually touch on APL, to me it was probably one of the most significant talks for business APL system developers that I've ever heard, especially given the increasing interest in quality assurance. Two short paragraphs do not do justice to a two and a half hour presentation, but hopefully readers will go away and read the proceedings or, better still, obtain a copy of his book, "Principles of Software Engineering Management" (Addison-Wesley, 1988).

The afternoon began with a visit to the Workshop on APL Problem Solving. I'd thought that this would be another opportunity for me to pick up some useful techniques etc., but in fact the two presenters were studying us, the audience, and the thought processes which we went through with a given problem. To provide the data for later study, the audience was paired-off, with one partner acting as the solver while the other recorded the thoughts and ideas as they occurred (the solver was supposed to vocalise ideas, the recorder being allowed to probe but not prompt in any way). After an excruciating 20 minutes (which seemed much longer) the roles were reversed. I'm not sure how valuable was the material gathered, but it was an interesting exercise to see the many differences, and similarities, of approach.

The last session before tea was by Diana Cook, High Productivity Systems, which looked at techniques for designing and/or maintaining large systems. By using a detailed analysis of more than 300 bugs encountered in running a large financial system, the various causes were discussed and the advantages of using structured programming methods illustrated. Of particular importance was the adoption of a set of project control guidelines or principles, many of which though 'obvious' are all too often ignored in a number of sites. The use of data dictionaries to control/translate program access was mentioned as this is a useful method of reducing many common maintenance problems, as was the more rigid control of modifications and enhancements (the cause of many of the worst bugs). Finally, the keeping and analysis of an error log was shown to provide useful data on underlying patterns or problem areas not necessarily discernible from installing individual "fixes".

The last two sessions on Monday were quite different, although equally interesting. Firstly Alan Graham, now ex-IBM, put forward his suggestions for a new, simplified version of APL which would also rationalise some of the variations between competing versions/implementations. Being 'simple', of course, means that the ideas in APL0 are too far-reaching to fit into a small article, but many of his suggestions seemed to produce quite a favourable reaction among the audience. Finally, Anthony Camacho gave an illustrated guide to the design of mazes (that's right, Hampton Court and all that). He'd recently been employed to help design and more importantly check-out some new mazes which had been commissioned around the UK. The most important test was to determine approximately how long it would take for someone to get out!

Tuesday is somewhat blurred for me, since I was suffering from extreme giddiness (delayed jet-lag), but I do remember a fascinating talk by young jazz star Stanley Jordan who used programs to help develop harmonics and musical relationships of which he would not otherwise have dreamed. The other session of note for me was given by Nancy Wheeler, covering an introduction to SQL/DS and later on how it can be used with/from APL2. Given APL's simple yet powerful analysis tools, the idea of using products such as SQL to access the vast amounts of existing data out there could give the language a whole new lease of life.

The Wednesday morning sessions **APL2 on the PS/2**, and **APL In The Workstation Environment** which followed it were more in the order of product demonstrations, although each was very well done and quite educational. The workstation demo in particular showed just how much power can now sit on a desk, and helped to emphasise the drift in some organisations toward downloading mainframe systems onto one or more such super-micros.

Just prior to lunch, Arthur Whitney (of Morgan-Stanley) told an increasingly interested audience about a streamlined version of the APL interpreter which he had helped to develop in-house. Among the important design principles adopted for this project were portability (I believe it was written in 'C') and simplicity. As an example of the latter, all "idiom recognition" sub-modules were excluded, attention being given to speeding up the basic procedures with assembler routines once the design was finalised. The significant time-savings achieved by this interpreter were illustrated by comparisons with benchmarks run elsewhere. Indeed, so interesting were the results that more than one person (myself included) sought more information afterwards, in particular as to whether it would be made more widely available (unfortunately not).

Immediately following lunch, a further plenary session was held at which Stephen Wolfram demonstrated a stunning package called **Mathematica**. This package utilises the high resolution Macintosh screen to allow "normal" mathematical symbols and constructs to be typed, except that now values can be assigned to abstract formulae. As in many instances values can also be formulae, the more complex examples began to resemble rule-based systems.

The highlight of Wednesday afternoon was a panel presentation by four experienced developers of **Practical Development Techniques for Large Systems**. What was fascinating to me as a business user/developer was the variety of approach, albeit in some cases dictated by external constraints on time or budget. Panel members who obviously were well in control of their fate extolled the virtues of comprehensive documentation, beautifully structured design, libraries of utilities with their own maintenance programs/procedures, and of course standards - all good stuff, but not always achievable, which is why the last speaker received so many sympathetic nods. This individual

explained about rushed projects with compressed deadlines, prototypes which were so successful that they were immediately installed as production systems, no time to develop standards other than the informal exchange of techniques, with the "strongest" becoming naturally the most common as it was generally accepted. What the last speaker did emphasise, however, were APL's speed of development and change, which helps them to keep their users more than satisfied.

Thursday came all too quickly and began with sessions on data typing and user-defined data types in APL2. After the break, Martin Gfeller and Manuel Alfonsaca continued with a look at **Object-Oriented Programming in APL**. I must confess to being an interested novice in this area, and was hoping to pick up some useful insights. In brief conversations with Mr Gfeller during some earlier coffee-breaks, his ideas for OOP and extensions or revisions of APL, were both stimulating and well structured, with potential advantages to those users not necessarily using OOP techniques. The session itself was rather marred, however, in a number of ways. Firstly, the audio-visual equipment was not functioning at all well, at times making the speaker almost inaudible. This, coupled with Mr Gfeller's lack of large-scale presentation skills, unfortunately blurred the very valuable message which he was trying to get across. What did come across was the suggestion (and I hope this is the right interpretation) that data and functions be grouped into general(-ised?) "classes", with rules governing their interaction just as there are in mathematics or interpreter construction. Unfortunately, neither of the two sessions seemed really accessible to the relative novice, with too much of the discussion descending into the realms of pure maths.

While the emphasis on the pure mathematical (not always related to APL) was rather more prevalent than I would have wished - being a simple "business solutions" APL-er - the conference was on the whole a great success for the ACM/SigAPL group which organised it. As with any event such as this, the organisers are somewhat hostage to those who submit material, and so not every session will suit all tastes - and rightly so. Strange indeed will be the taste of someone who finds nothing of interest, or who does not come away better informed than before. I encourage everyone with an interest in the language to attend at least one conference, therefore, so I will close by offering my thanks to SigAPL and wishing good-luck to the Danes for APL90 in Denmark. See you there.

Notes on the Papers

Tom Gilb: APL - A Planning Language

Reported by Maurice Jordan and Chris Brady

The first plenary session of the conference was a talk by Tom Gilb, an independent consultant who used to have a column in Computer Weekly.

His main theme was that every attribute of a project should be measured, e.g. throughput, response times, expected quality levels, test criteria, resources needed. Worst case figures (the project fails if there are not attained), planned figures (what is being aimed at), figures for existing systems (which might be manual), and record levels (best known performance) are all recorded. If you are aiming at beating the record, you have a research project on your hands, and are almost certainly going to overrun. You then attack the project in an evolutionary manner, doing as little as possible to achieve benefits at each stage. His thesis is that ALL projects can be done in this manner, and that this method consistently produces projects delivered on time, within budget, and highly satisfied users. The ideas are similar to those of Tom Peters ("In Search of Excellence", "Thriving on Chaos" etc.).



Tom Gilb

He has recently produced a book detailing his thoughts - "Principles of Software Engineering Management" published by Addison-Wesley (ISBN 0-201-19246-2). At 442 pages, it does ramble a bit, but is a thought-provoking book, and I would recommend it to everyone. It is available from Modern Books in Praed Street (near Paddington station) at \$18.95 (and probably from lots of other bookshops as well).

He started by briefly introducing the ideas in his book, by defining software engineering as a management and engineering problem, not a programming problem. Then he stated that the keys to successful software management were to focus on the following:

- people, not computer programming
- processes, not methods
- system attributes, not function
- eternal change, i.e. nothing is constant, we must always be responsive to end-users' needs, and
- regular communication, to meet users' objectives.

He stated that in today's rapidly changing world, an evolutionary delivery process to software management is the only really cost-effective one. He emphasised the need for close contact and frequent communication with the intended end-users, and that they and the developers should be on a learning curve whilst a project is progressing so that experience and feedback may be used to meet the requirements of the end-users as closely as possible. This is implicit in normal APL use.

He emphasised that the key to successful software management is to manage people not projects. In the future, software engineering methods will need to focus upon:

- multi-attribute capability;
- evolutionary change capability;
- adaptation to mid-project major change(s);
- rational engineering decision making (not dogma);
- designing to cost and value of results;
- process control (entry, exit, limits control), and
- evaluation, for what is best now, not what people think will be best for all forever.

The most powerful principles of control are:

- early delivery of at least part of a system, users will then be less concerned if there are problems later on;
- an evolutionary or iterative approach with frequent communication with users, (utilise the 'small wins' psychology);

- identify and define exactly the results expected by the users (customers), in writing and quantitatively, i.e. the positive results desired (the benefits) and for the constraining economic and practical limitations on solving the problem (the "costs" and the constraints), and
- design to cost.

For the above it is necessary to quantify:

- availability (reliability, maintainability, integrity);
- workability; adaptability; usability; and many other dimensions.

Tom has devised a complete software engineering management methodology to achieve the above and more.

Harry Bertucelli: What is an Argument?

Reported by Anthony Camacho

This paper began with a discussion of ways of representing numbers which would allow one to extend the number of digits in an integer without limit; then rational numbers represented by pairs of such integers would make unlimited precision arithmetic possible. Such rational numbers fill the number line sufficiently densely to represent all but the irrational part of real numbers. In extending such representations so that they can be held in arrays and in providing functions which can handle such arrays in the most general way the author was led by a sequence of trials and errors to propose a 'hide' and a 'reveal' primitive.

I was left with the impression that the trials were picaresque rather than empirical, and that there is no reason to suppose there are no errors in the latest version of the proposals. The presentation was didactic and enunciated clearly and slowly, with pauses for the audience to consider what had been said.

James A. Brown: Understanding Selective Assignment

Reported by Anthony Camacho

In essence the idea of selective assignment is that any expression which is used to select some items from an arbitrary array may be put in parentheses on the left of an assignment arrow. Then the expression on the right of the arrow, if its result is items

which match in shape, will assign that result to the positions of the selected items in the array. Such a mechanism is very powerful and such expressions seem natural in APL2. The paper discussed the problems that arise; for example as it is possible to write expressions which select an item more than once such expressions on the left of assignment arrows would assign multiple values in turn to a single position in the array. The advice we were given was 'don't do this - the results are not defined' from which I concluded that there is something seriously wrong somewhere. The presentation was a clear piece of exposition and did not much extend the ideas in the paper.

Mansour: Techniques for Avoiding Conditional Execute in APL2

Reported by John Searle

This paper was a good summary of methods for avoiding the use of conditional execute, which can be very expensive to process and which can create blind spots for workspace analyzers. Some of the content was specific to APL2, but the techniques were generally applicable to any interpreter. There is a place in all conferences for useful, nitty-gritty contributions such as this.

Cook: High Productivity for Large APL Systems

Reported by Chris Brady

This paper discussed some of the problems of large systems and how they can be resolved by using structured techniques from traditional project management. For over three years the author has maintained a large APL financial application for a major (magazine) publishing company. This was developed as a prototype designed to estimate complex manufacturing budgets, consisting of several hundred different cost-items, each with its own calculation rule. APL enabled the team to develop and modify programs very quickly. The users, delighted with the responsiveness of the initial model, soon began relying on it as a production system.

The major problem was updating and keeping track of the numerous and constantly changing system specifications. The system consisted of 10 APL workspaces, 20,000 lines of code, and several hundred magazine files each containing one Mbyte of data. The structured techniques used helped to reduce the number of bugs and decrease costs of maintenance to 1/3 of their previous levels. I recommend this paper to all managers, project leaders and developers of 'large' APL systems.

Charles Waters: Observations on Flight Behaviour in the Butterfly

Reported by Anthony Camacho

An exercise for a class was to devise an algorithm for the path (of a butterfly) which visited each vertex of a cube once and only once in a random sequence, and to calculate the length of the complete journey.

The presentation was enhanced by a demonstration, which ran throughout, of a cube drawn on a screen with a purple butterfly following such a random path, pausing and then restarting on a new cube.

The frequency of paths of different lengths (calculated by Eugene McDonnell) may surprise you; it should make you think whether you do the thinking before you see the table or afterwards.

As Charles Waters wrote:

"School activities too often emphasize remembering at the expense of more important mental functions. Composing APL expressions offers unique opportunities to extend the dimensions of student experience by furnishing exercises in thinking."

Alan Graham: APL0: A Simple Modern APL

Reported by Anthony Camacho & John Searle

Alan Graham has continued his exploratory voyage into the worlds of interpreter and language design, even after leaving IBM. APL0 is an experimental language that translates into APL2, but it is designed to be simpler and more consistent than existing APL's (hence the title).

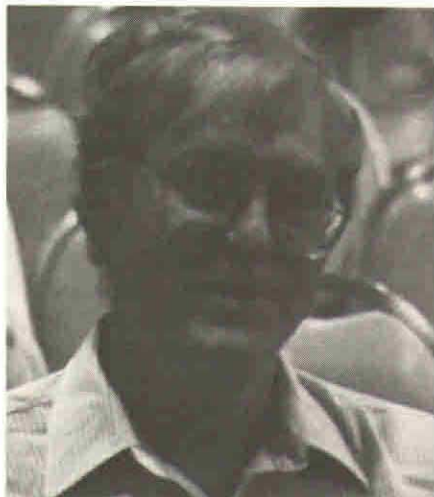
Alan's APL is written in APL2 (about 600 lines of code) and incorporates features from APL2, Dictionary APL, APL90, Scheme and Smalltalk. In his view APL has grown too big - starting simple was fine but the twenty years of extensions without any deletions make it unwieldy. There were flaws in the original design and in some extensions.

He rejects any concept of modes (□IO, □CT, edit-mode, etc.) bracket indexing and bracket axis. Instead, he uses the indexing function and the rank operator, and changes the behaviour of selected language features. The net result is a language which "performs reasonably well" and which implements the modern view that "Less is More". Alan also gave us his predictions for the subject matter of APL99.

Camacho: Workspace "Maze"

Reported by John Searle

Anthony entertained us with lots of nice slides of mazes, and left the technical detail to the reader of the proceedings. The concept of the coloured maze has revolutionised the market for mazes. We were challenged to solve a three-colour maze from a slide, but I must admit that I walked backwards into a black hole. Psychological poser: when you enter a maze, do you try to minimize or maximize the amount of time you spend in it?



Ed Shaw

Edward J. Shaw Jr: Making APL Error Messages Kinder and Gentler

Reported by Anthony Camacho

Ed is the man who promoted the idea of 'APL as a tool of thought' - the title of a series of professional development seminars held by NY SigAPL. He began his presentation with some slides of New York Harbor. The simple idea of his paper is that APL could give much more helpful error messages. Why DOMAIN ERROR and a caret under the division symbol when you divide by zero, when it would be possible to say 'Improper Value - denominator must not be zero' and put the caret under the zero? The paper had a dozen and a half detailed suggestions like this.

Ed had hurt his back and had to give his presentation from a stool; he missed some of the conference as a result. Get well soon.

Friis: A Fusion of Lotus 1-2-3 and APL2

Reported by John Searle

Eric Friis described the results of an IBM-internal project "FUSION", which models the contents of a Lotus 1-2-3 spreadsheet as an APL2 workspace. The advantage of this approach is that the full power of the APL2 mainframe product can then be applied to the

data, e.g. SQL/DS, DB2 and GDDM. Various techniques were described which overcome the 'WS FULL' and 'SYMBOL TABLE FULL' problems inherent in such a model. The presentation was a concise summary of the published paper. Unfortunately, neither the presentation nor the paper referred to any of the previous non-IBM work done in this field. This paper extends these previous ideas by using the mainframe environment, although the data upload and download problems were glossed over. Hopefully, in the future this work will be made available externally as an APL2/PC workspace, with either an interface to the PC Auxiliary Processors, or with an appropriate workspace transfer mechanism to or from APL2 on a mainframe.

Gary Bergquist: Vendor Forum on the Zark APL Tutor

Reported by Anthony Camacho

Vector hopes to review this product in due course. It is a tutor for APL*PLUS/PC in 26 'units' and on four floppies. It can load itself into your workspace at any time when you may need its help and will clear itself out when you have done with it. Gary explained that many tutorials are not much better than page turning machines, with the disadvantage that you can't easily use them on the beach or in the bath. His objective with this product was to do as much as possible to make it do as much as possible beyond turning pages. The user can set several bookmarks for quick access to particular parts of the course and there are four other methods of reference:

1. Resume where you left off
2. Use the table of contents
3. Move the bounce bar to one of 100 or so functions
4. Look in a 300-entry subject index



Gary Bergquist

Throughout the course the bottom third of the screen is available for immediate execution of trial expressions. The course will also offer you 'read' problems (what does this do?), 'write' problems (what expression will do this?) and drill on a range of functions. It is also intended to make you smile occasionally.

Dick Bowman: APL and GDDM - A High Performance Toolkit

Reported by Sylvia Camacho

The paper presented in this session is a description of some workmanlike cover functions for GDDM written in APL 2 and replacing the vanilla-flavour IBM offering. Dick made some characteristically robust comments along the way about the inadequacies of IBM software.

Dick made it clear that the prime object of the toolkit is to achieve maximum speed and that this is the justification for having global variables 'hanging around in the workspace'. Some interesting speed comparisons are given in the paper.

It should be held in mind that the applications for which the author uses the toolkit include both graphics and text and that this is the requirement for a truly general purpose package. AP126 is not very APL-like but it has full graphics capability (compare the Dyalog quad-SM paper where only text screens are described).

A.D. Curtin and J.M. Scholes: Quad-SM: A Full-Screen Manager for Dialog APL

Reported by Sylvia Camacho

Since full-screen interaction between user and machine became commonplace, the most memorable characteristic of packaged software is its public 'face'. A glance at a screen will tell you what your computer is doing unless, of course, it is using one of those older full-blooded languages such as APL, Fortran or Cobol. This lack of image is a disadvantage to those of us who try to promote APL in competition with mainframe and micro packages. I am sure I am not alone in welcoming a friendly and versatile full screen manager. In the short time available John Scholes did an excellent job of introducing the Dyalog offering.

The ability to localise quad-SM and the built in scrolling of nested data is a natural for APL. The combination of the facilities of quad-SM and quad-SR seem well matched. I am sure I am not alone in wanting to hold and present my data to the screen in a format

which is convenient for other purposes. Similarly I welcome a simple method to control the cursor - so many GDDM based cover functions lamely accept tabs always in row major sequence.

This was a good presentation of welcome software. Am I alone in wishing that there were a full-screen standard for APL which we could all come to find as familiar and comfortable and instantly recognizable as the proverbial 'old shoe'?

Learning and Teaching APL: Demonstration and Discussion

Reported by Anthony Camacho

This session began with a delightful children's story about Willie the snail, who wanted to be an architect. The story begins with Willie going to school and ends with a logarithmic spiral in a series of golden section rectangles made by dividing each into a square and a smaller rectangle. The logarithmic spiral is the elevation of the snail's 'house'. Linda Alvord did the drawings and told the story.

Since 1985, when Tama Traberman was invited to speak to APL 85 about APL in Education there have been no significant advances in the use of APL in schools. The New York Times recently said there has been no revolution in computer education; nowhere is anything being taught with computers which couldn't be learnt equally well without them - and there is still only about one computer between 20 students. It was suggested that we should get APL into use through the hierarchy of education - attack the power centres where the decisions are made.

Lee Dickey said that Computer Science leaves students short- changed. They think they are going to learn how to make computers do what they want and end up writing Pascal! At Waterloo the use of APL has reduced. The computer science department is prejudiced against it even though there are Sun workstations running SAX. He has recently announced two courses - Modern APL (for the faculty and graduates) and a student course for actuarial students.

Anthony Camacho was asked to report progress on I-APL; he reported:

1. Steady sales: over 1000 copies now sold
2. A new full screen workspace for the PC version
3. Version 1.1 with all known bugs removed
4. BBC Master and B version with colour graphics and sound

5. Archimedes version with colour graphics and sound
6. A new version of the manual printed
7. APL in Social Studies by Tama Traberman printed
8. An APL Tutorial reprinted with minor revision
9. An encyclopedia of APL second edition printed
10. Education Vector sent out to 1500 on mailing list
11. Porters working on Apple, Atari ST, Amstrad WP etc
12. APL for the Maths Classroom by Norman Thomson published
13. A 32-bit version of I-APL is now working on Archimedes
14. Several British schools are beginning to try APL

Robert Brown felt that the first job when teaching people APL was to engage their enthusiasm; show them things they are already familiar with and delight them with new insights from there. He uses a cardboard box as an analogy of a workspace.

Howard Smith told how he got into trouble by beginning a course with an explanation of scalar, vector and matrix. He asked the students to leave and come back in in five minutes so that he could begin again. His second try was to call to their attention the way information was arranged in the real world, and how people looked things up. This proceeded through share prices in the paper to telephone directories and dictionaries until he was able to put on the board a description of a general array. Then he introduced APL and it was accepted.

Bruce Prior suggested that access to computers ought to be a civil right.

Apologies to people who contributed and have not been reported.

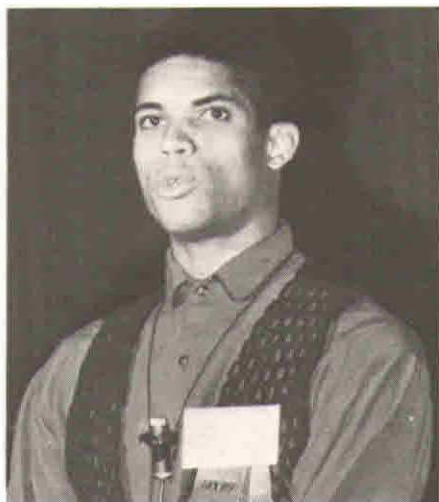
Stanley Jordan: APL for Music

Reported by Paul Chapman

At the APL89 conference lunch, we were treated to two talks on computer music. The first, by David Steinbrook, was a rather brief and simple introduction, while the second, by Stanley Jordan, bowled the audience over. It is this second talk which I will try to review, without the benefit of notes,

Stanley Jordan

Stanley Jordan was introduced to us as a professional musician who has had a number of albums published. Neither he nor his music was known to me, and I had not read his paper in the conference proceedings before hearing him talk. Perhaps I expected him to be a musician dabbling in computers, as many do now with the advent of MIDI (the Musical Instrument Digital Interface) and the use of micros as music sequencers. Nothing in the first fifteen minutes or so of his presentation led me to believe anything else, but then my ears pricked up, as he began to talk in more detail about his use of computers, and of APL in particular, when he suddenly mentioned Markov Chains.



Stanley Jordan: APL for Music

Markov Chains

My own memory of such things would be very dim now, were it not for regular visits to the country house of an old friend, Saul Jacka, full-time research statistician at the University of Warwick, and part-time sheep farmer. He presents me from time to time with puzzles in the field of stochastic processes, the latest of which involved random walks across a Sierpinski carpet. Whilst I do not know of any direct application of this to music, it served as a reminder of the mechanics of Markov Chains, which turn out to be sequences of numbers, where each successive number is derived from those most recently written down by some function whose result is partly determined randomly. Jordan's application was brilliant, and solved a problem I have given thought to for some years - how to produce random music which sounds tuneful.

Random and Tuneful

He explained, or I understood, that certain musical intervals (pairs of notes) are more tuneful than others, and that music produced randomly can sound more rational, at least to Western ears, if note follows note with a high probability that the interval will be consonant (ie a perfect fourth, fifth or octave, or a third or sixth). He went further, and combined groups of four notes in a sequence into a single chord (something akin to partitioned reduction, for the APL theorists among you), explaining that the resulting chord sequence would make good sense. To make his point, he then asked for a tape recording to be played, and the result was marvellous! We sat entranced by this simple device.

The Music Played

Recording followed recording, with explanations for the algorithms for each piece. I found the results very beautiful, and only became less interested when Jordan began introducing the Human element into the pieces. Not, you understand, because his compositions were bad or even not to my taste, nor because of any lack in the ability of the session musicians employed to enhance the creations of the computer, but because I had heard for the first time what a clever algorithm was capable of producing in the way of Baroque beauty.

Computer Music vs Programmer Music

In his own talk at the conference banquet, A. K. Dewdney made the distinction between Personal Computing and Personal Programming. The first is what millions of middle managers do with Lotus 1-2-3 every day, whereas the latter is what I do sitting at home with my Archimedes. Now, thanks to Dewdney, I can describe my job (and hobby) to lay friends without lumping myself with the acres of blessed users. But Dewdney's language prompts a plea that a similar distinction be made in the application of computers to art. Computer Art is what is done by Ad-men and Paintbox users, and Computer Music is produced by sound sampling and bit-wiggling. But graphics from those old Moire demos through Conway's Life to the Mandelbrot Set are Programmer Art, and Jordan showed us the first steps in Programmer Music. The Musician's Union may not like it, but Bach would be in there hacking.

Questions and Applause

During the questions after Jordan's talk, a request that he acknowledge his use of APL on the cover of his next album was greeted with some acclamation. But the applause at the end went on longer than for any session that I attended, and afterwards, between autographs, he revealed hints of deep mathematical and musical knowledge. For a while

I have expressed my belief that in not too many years' time we will be able to buy black boxes to attach to our stereos to compose and play trios in the style of Bach in real time. Jordan's work has brought that a step closer.

Astrophysical APL - Diamonds In The Sky

Reviewed by Dave Ziemann

The paper was presented by Glenn Schneider, a young astronomer from the Computer Sciences Corporation, Space Telescope Science Institute, in Maryland USA. Paul Paluzzi and James Webb were named as co-authors.

The presentation was not about statement delimiters in APL, but was rather more concerned with using APL to model the structure of white dwarf stars. What exactly is a white dwarf star? Glenn explained that a star spends most of its life creating energy by nuclear fusion, and that this energy flow supports the weight of the outer layers in hydrostatic equilibrium. But when the nuclear fuel is expended this equilibrium is violated, and the star collapses under its own gravitational force. The final configuration depends on the mass of the star; if the mass exceeds the famous Chandrasekhar limit then neutron star formation or even supernova collapse are possible, otherwise a small dense star called a white dwarf is formed.

Glenn and his colleagues chose to model the structure of these stars as a way of putting APL to the test. They looked for an astronomical application that was fundamentally iterative and would at worst involve small amounts of data in vectors and matrices rather than higher rank arrays. White dwarf stars fit the bill perfectly; the model performs simultaneous numerical integration of the structure equations using a Runge-Kutta procedure to "step out" from the centre of the star to its surface in appropriately small increments.

The presentation involved many excellent foils which summarised the general concepts of stellar structure; the hydrostatic equilibrium, the mass distribution with stellar radius, energy loss versus energy generation, and the temperature gradient across a spherical shell of star. These equations were presented in standard mathematical form along with their APL representation.

By its nature a white dwarf has no energy generation mechanism and little or no temperature gradient, leading to a great simplification of the equations. On the other hand, general relativistic effects must be taken into account because the high internal gas pressure leads to partial degeneracy, the gas no longer obeying the "perfect gas laws".

The model was restricted to white dwarfs made of carbon, although it could be extended to deal with other compositions. The compressed core of the star, which is often nearly pure carbon for stars of the appropriate initial mass, is crystallised during the collapse, a fact which at last fully explains the title of the paper.

The authors were refreshingly concerned with the readability of the code, which appeared to be modular and well-commented. They are also very aware of how well APL lets them get to the heart of the problem, avoiding the excess baggage of many other languages. (In the paper Glenn reveals that a colleague of his once had a sign above his door which read "Today I am" followed by a tiltable arrow which pointed to "an astronomer", or "a computer programmer"). The APL model was originally programmed in Waterloo MicroAPL on a Commodore SP9000 (SuperPet), in only 28K of workspace. The version presented was coded in APL 68000 on a Macintosh.

Unfortunately, their enthusiasm for APL is not universally shared by the traditional astronomical community; many of the authors' colleagues would apparently be unhappy with the speed of such systems in APL - they feel that using APL to prototype the system and then re-coding in C would be a more effective way of selling the virtues of APL, until at least they can get APL on the Cray...

Disturbingly, when they attempted to publish their paper in a well-known astronomical journal, it was rejected, with a "unanimously concurrent" request that the "algorithmic representation should be removed" and replaced with something that is "machine readable". Needless to say the authors plan to re-submit their paper with some suitable explanations; Glenn, we await the outcome with great interest - please let us know what happens...

Norman Thomson: Generic Binary Trees in APL2

Reported by Anthony Camacho

Norman likes the elegance of direct definition, and in this paper he combines it with the power of APL2 to create functions to perform the basic operations (and a few others) on binary trees. The functions are recursive and the amount of code required is astonishingly small. Norman's thesis is that trees should be thought of as a data type with valuable special properties; APL2 enclosure makes it possible to store anything whatever in a node. Of course to execute the examples you must write a direct definition processor in APL2 or translate them into del form.

Kristen Sternberg: Reflections on Teaching APL to Eighth Graders

Reported by Sylvia Camacho

It was very useful to have an account of teaching experience over eight years - there cannot be many Schools having so long a continuous exposure to APL. The Dalton School course nevertheless is like all other schools experimenting with APL in that the courses are all elective and tend to attract only the pupils of higher academic ability. As only volunteer pupils are exposed to APL we have not yet any evidence to decide whether it is truly more difficult for average pupils or whether it attracts the brighter ones because they are more confident and more curious.

The questions following the paper were dominated by a discussion about the proportion of girls electing to take the APL course. The evidence at Dalton seemed to be that there is much variation from year to year although probably there is the usual bias in favour of boys.

There were very few references to APL texts. Perhaps if there were more teaching material which uses APL more teachers could be persuaded to try teaching with it. Let us hope the APL Community will make an effort and get some produced.

ACM/SigAPL Open Meeting

Reported by Anthony Camacho

Marilyn Pritchard opened the meeting, suggesting an agenda as follows:

1. Introduction
2. Finance
3. Membership
4. New Projects
5. Quote-quad
6. Future conferences
7. Questions and answers

In her introduction she reported the resignation of Paul Berry and thanked him for his work. Linda Alvord has agreed to take on the role of SigAPL historian. The Bay Area User Group has become a Sig in its own right.

On finance SigAPL is operating near the break-even level on an annual turnover of approximately \$35,000. Dallas made a profit of about \$45,000 and the reserves now stand at \$65,000. The major income is from membership dues and the major expenditure is on Quote-quad.

The membership is falling steadily. Marilyn felt we must give better value for the money and hoped for 50 or 100 new recruits from this conference. She showed a slide of the membership since 1983.

1983	2100
1984	1960
1985	1850
1986	1750
1987	1570
1988	1460
1989	1350

There is a time lag between starting a new project and getting any results. There are three projects to mention now:

1. An index of APL articles (interim version in the delegate pack) which was supervised by Dick Bowman
2. A phrasebook of extended APL which was being worked on by Paul Berry
3. A new SigAPL award for extending awareness of APL - please send in your nominations now.

Proposals for further projects which will make APL or SigAPL stronger would be welcome.

Lee Dickey reported on Quote-quad. He hopes to solve some of the problems of setting mixed APL and text. He wants to be able to do APL characters in various sizes. Joey Tuttle has produced an APL font. Lee is looking at Knuth's TeX because it allows hardware independent font designs. It would be nice if both author and editor could see on the screen what will eventually be printed on the page.

Lee thanked his sub-editors:

Norman Thomson:		Algorithms
Howard Peelle:		Education
Roger Hui:		Recreation
Jim Brown:		Language
Vacancy:		New Products

He would welcome a volunteer to take over the vacant post.

An algorithm project has been running for two years and many of the algorithms are in the software exchange.

The recent delays in publication are the editor's fault: the problems have now been overcome.

A roster of members of SigAPL is being created, and an index to Quote-quad which it is intended to publish in a special promotional issue in about a year's time.

Lynne Shaw reported on future conferences. APL 90 will be at Elsinore, near Copenhagen. Per Gjerlov, the conference chairman, spoke promoting it and asked everyone to give urgent attention to getting their papers ready. There are no arrangements for APL 91. Madrid has been mentioned for APL 92, but will need a significant partner as the Spaniards do not feel able to shoulder the whole burden on their own. It was pointed out that APL 90 and SHARE were planned for the same week next year.

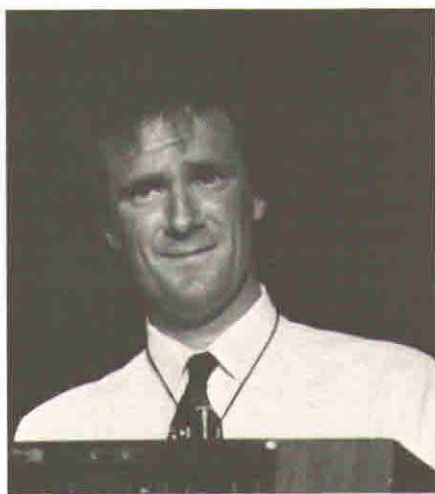
Marilyn said the ACM will allow members of the committee who are willing to continue in office after their two year term for another two years. There are now two vacancies on the committee, a membership co-ordinator and a member at large. She would like to see this occupied by someone resident outside the USA.

Linda Alvord listed the initiatives on membership now ready for the new co-ordinator to take over. There is a new brochure. A list of professional materials is in preparation. There is a disk of statistical material from the Statistics day. Version 1.1 of I-APL is now available for circulation.

Plenary Session Arthur Whitney: Streamlining APL

Report by Anthony Camacho

This was the highlight session of the conference. I had not heard of the Morgan Stanley APL interpreter before and was delighted to see the limitations of every APL I use so dramatically overcome. Workspace and object size and execution speed far exceeding anything on the commercial market made it the keynote of the conference for me.



Arthur Whitney

Arthur Whitney introduced himself as working in Analytical Proprietary Trading for Morgan Stanley, whose transactions exceed \$100,000,000 a day and who have around six gigabytes of databases used for real-time trading. They do considerable mathematical and statistical analysis in the process.

The needs are to provide hardware, software and data platforms, to create models and optimal trading strategies and to write and manage real-time trading systems. The hardware is IBM 3090/600E with Vector Facility and NAS XL80, Sun 3, Sun 4 and Solborne workstations; the software is APL2 and SharpAPL on mainframes and APL*PLUS and SAX on workstations.

The nature of the work is well suited to APL but Morgan Stanley dislike paying for this in reduced speed or increased processor usage so decided to produce an APL interpreter which more nearly meets their needs. The goals were:

- Implement a low-overhead streamlined interpreter
- Allow large workspaces and objects (use disk to give virtual memory)
- Integrate with Unix and other languages
- Write in C and provide an ASCII version

The result is called A. It outperforms the mainframes when running on workstations, gives 500 megabyte workspaces, provides access to C, Fortran and Unix utilities and runs on several machines. It is about ten times the speed of any other APL.

The features of A:

- Enclosed arrays as well as 'each'.
- Generalised rank operator, which
 - applies functions to subarrays
 - generalises the notion of scalar functions
 - rationalises the bracket-axis operator
 - encourages and simplifies modular programming
 - can run with no overhead in time or space
- An indexing function
- User-defined operators
- All operators take any functions as arguments
- Direct definition
- Any number of arguments per function
- Control structures: IF, ELSE, WHILE
- A 'symbolic' data type
- Scalar encoded data, functions and operators

The main design principle was to keep it simple. Overheads were kept low and primitives were made as fast as possible so that no special casing was required. The rank operator is built-in. There is no garbage collection - freed space is made available immediately. Movement of objects is minimised by intelligent use of pointers. A makes use of memory mapped files and has no practical limit on object or workspace size (at present the limit is 2 gigabytes, but this is temporary and will be increased). Library routines (Unix, C or Fortran) are loaded dynamically when APL calls them and APL expressions can be evaluated from C with no overhead. A can read from and write to Unix std I/O and import and export objects through 'sockets'.

A thus becomes the lingua franca or the glue which links programs written in any of these languages. The applications of A so far are in numerical analysis, simulation, plotting, 3D graphics, continuous rule-based market monitoring, real time trading and in various optimisations involving C and Fortran.

Here is a routine in C:

```
bin(v,n)A v;{
  int l=0,h=v->n,m;
  while(l<h)
    if(n>v->p[m=(l+h)/2])l=m+1; else h=m
  return l;}
```

and in A:

```
"bin.o" dyld "IT bin(IA,IT)"
```

Here is a direct definition of nub.

$$\text{unique } \omega: ((\omega \upharpoonright \omega) = 1 \rho \omega) / \omega$$

Here is the SAPL of binary search beside the A version, to show how with $\square 10$ permanently set to zero [this got a round of applause] and WHILE, IF and ELSE, a short and elegant function results.

```

    ▽ l←v bin n;h;m
[1]  l←□lo
[2]  h←□lo+pv
[3]  L0:→(l<h)+0
[4]  →(n>v[m←l.5×l+h])+L1
[5]  l←m+1
[6]  →L0
[7]  L1:h←m
[8]  →L0
    ▽

```

Direct Definition

... binary search

```

v bin n:{
  l←0; h←pv;
  while(l<h)
    if(n>v[m←l.5×l+h])l←m+1
    else h←m;
  l }

```

Arthur then explained some timings using variables i, f, m, v, and e.

Timings

i←?10000P10000

f←i÷10000

m←1000 10P i

v←i[Δi]

e←i'' ?10000P5

Dynamic Programming

... sparse matrix product

m - market transition probabilities
s - state objective values

m +.×#2 s

→(<[1 2] m) +.×'' <[1 2] s

Optimizer Formulation

... sort the rows of a matrix

m - matrix of breakpoints (by stock)

(Δ#1 m)[]#1 m

→(c''Δ''m)[]''m←<[1]m

→(Δ''m)→''''c''m←<[1]m

Database Query

... find opening price

(valid 1#1 (1))[]#2 price

>((=[2]valid)1) > "[2] price

Boxing

recursion, each and #

```
i bx v:{
  r←ρu+Δd+(i>m){v};
  b←d[u[1+(j+1r+j)×1j+n[i]]];
  if((ρn)=i+i+1)b
  else b,i bx j←u[v]
```

... workarounds

m +.×#2 s

$$r \leftarrow (\rho m), k \leftarrow 1 + \rho m$$

$$+ / (r \rho k / [0] m) \times r \rho k / [1] s$$

(Δ#1 m) []#1 m

$$v \leftarrow m$$

$$d \leftarrow (r/v) - l/v$$

$$(\rho m) \rho v [\Delta v + (-1 + \rho m) / d \times 1 + \rho m]$$

	A	APLPLUS	SAX	APL2	SHARPAPL
v bin n	1	3	6	3	4
+/' e	1	5	17	5	38
unique i	1	3	4	*2	5
unique f	1	7	13	100	3
unique m	1	1500	1000	800	600

Normalized

%

	A	APLPLUS	SAX	APL2	SHARPAPL
boxing	1	4	4	6	3
dynamic programming	1	18	14	4	*
optimizer formulation	1	17	23	16	*
database query	1	13	35	18	*

Normalized

Arthur concluded that APL is well positioned for the future, because:

- APL primitives, especially with rank and each, can naturally take advantage of vector and parallel processors.
- APL workspaces and objects can take effective advantage of large physical and virtual memory.
- APL can be open to the outside world and take advantage of improving operating systems and libraries of code.
- APL can be a tremendously productive prototyping and programming environment

Questions

The talk was followed by questions; some of the points raised are noted below.

Making A into a marketable product is 157th priority - that is to say it comes nowhere; Morgan isn't in that business and would need persuading that there was enough benefit in it to be worth giving up a competitive advantage and diverting people from highly profitable work.

Alan Graham pointed out that 'each' and 'rank' apply to functions; wouldn't it be nice to allow them to apply to a phrase as APL0's lambda proposed. Arthur confirmed that lambda was a nice feature and has been included in A in earlier versions and may be put in again in later ones. It is like having direct definition for operators.

Adin Falkoff asked (a) whether part of the speed was obtained by making A single-user and (b) how arguments after the second were passed to a function. Arthur explained that workspaces were just files and many users could access a file at a time; A turns a workstation into a nice multiprocessing workspace environment. There is nothing 'single-user' about it. Multiple arguments are enclosed in braces. Adin suggested that brackets and semicolons would be a neat way to punctuate multiple arguments.

Eugene McDonnell asked whether a full description of A would be published; Arthur said that he would like to do that but gave neither undertaking nor date.

Panel Session

The questions were followed by a panel session which the chairman introduced by suggesting that each panel member should say what improvements would be likeliest to help APL make ground.

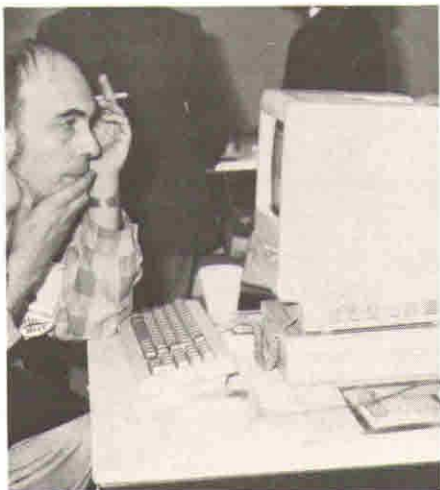
Roy Sykes thought A sounded marvellous, but that not being able to set the index origin was not so marvellous. Was the applause from the origin-zero camp. He asked the audience to raise their hands to vote for zero-origin fixed, one-origin fixed or variable origin as now. More people would fix the origin at zero than at one, but at least as many would like to keep the option. Roy felt that parallelism would be important and gave it as his opinion that the syntax analyser was not an appropriate place to introduce it.

Maurice Jordan (of British Airways) wanted to stick to measurements rather than religion and the best measure of work he had yet found was the function point. BA was now achieving 300 function points a man-month and with facilities like $\square$ SM he felt the target should be 100 function points a man-day. The two leading APLs offer differing notations and ways of thinking, but the difference is academic; Maurice's estimate of the cost to BA of conversion to an APL where the enclose of a scalar did not equal the scalar would be zero.

Robert Brown (of Shearson, Lehman, Hutton) wanted to broaden the use of APL and felt that the three most important things to go for were 1: speed, 2: speed and 3: speed.

Gerard Langlet (of CEA dept of Physical Chemistry and the French ISO group [AFNOR]) said that the main problem was portability. To obtain maximum portability he was now using (after 17 years!) a subset of the ISO standard. He too foresaw great importance in parallel processing and felt that it would be a worthwhile simplification to program only in vectors and to eschew branches and labels. Ideally he would like a notation for student use which could be converted to APL or to other languages.

David Ziemann (of Cocking and Drury Ltd and the British ISO group [BSI]) felt that the APL vendors had hijacked the domain of APL extensions and that he would like to see the APL community regain control. He wondered why 'each' had been included in A. Was it to simplify porting from APL2? How did Arthur rate the priority of 'each' and 'rank'? Arthur Whitney confessed that he felt ambivalent about 'each'. By allowing people to avoid multidimensional arrays it delayed their progress to becoming really good APLers. On the other hand this was the first time that 'each' and 'rank' had been implemented in



Gerard Langlet at the MicroAPL stand

the same system and when the data was ragged he used 'each' and when it was rectangular he used 'rank'.

Adin Falkoff joined in from the floor and complained of three tendencies that reduce readability of code; the use of lower case, strand notation and failing to use brackets and semi-colons. There are times when one feels Adin would like to make it a rule that every APL function must have at least one use of brackets and semicolons.



The Panel: Ziemann, Langlet, Brown, Jordan and Sykes

Phil Benkard (of IBM) was astonished at Maurice's low estimate of the cost of conversion from floating to grounded enhanced arrays. The thing to bear in mind is whether the objects you create are within the domain of the functions you wish to process them with.

Jim Lucas' idea of APL pie in the sky was that APL should 'believe in' screens and shapes and colour. On APL and parallel processing he would like to see parsing and interpreting done in parallel, not just the processing of arrays.

Robert Brown felt that faster addition and subtraction would bring more benefit than any new feature, and that the quality of the system depended most upon the coding and commenting style.

Somebody asked how far interpretation preceded execution, but his question was ignored.

Dave Weintraub also addressed the question of portability, asking to what extent it could be automated, but his main thrust was to complain about the standard of the code issued by the major vendors. Too much of it sets a really poor example and unfortunately is copied. The APL professionals have a lot of power to influence newcomers to use good practices and too often give a bad example.

Linda Alvord did not believe the audience understood the problem of $\square 10$ when Roy Sykes took his vote earlier. She suggested that to understand it you needed to look at the effect of having both iota (for index origin one) and gamma (for index origin zero). She would also like to see the graphics origin in the centre of the screen.



The Audience!

Rosemary Snow asked why we bothered optimising general purpose computers when surely a computer designed from the ground up to process APL would surpass any general purpose machine.

Carol Stein felt that there was room for two standards of interpreter; one would be for thinking about the problem and experimenting and the other for when finished code had to be executed really fast. Her idea of an improvement would be to display variable names in a way which indicated e.g. their rank by the colour and their depth by the typeface.

Gerard Langlet would like to be able to speed up interpretation by asking for a reduction in arithmetic precision.

Neidinger: APL Approach to Differential Calculus

Reported by John Searle

Richard Neidinger described a simple APL2 approach to calculus. His workspace of 14 one-line functions permits the automatic differentiation of "practically any function", by using a representation of a function $f(x)$ as a vector $\{ f(a), f'(a) \}$ at point a . The rules for differentiation (e.g. product and chain rules) follow from this notation. He has achieved the exact calculation of the tenth derivative of $f(x) = (x.\text{csc}(x))/\ln(\arctan(\exp(x)))$ in less than four seconds using APL2/PC. Try it using Mathematica (but don't hold your breath)!

Banquet Speaker A.K. Dewdney: A Loose Cannon Takes Aim at APL

Reported by Paul Chapman

With alcohol available only at exclusive New York Hilton prices (prices, in other words, designed to exclude even the moderate drinker), coffee was the one free drug on offer at the APL89 banquet. With its arrival, A. K. Dewdney, the *Scientific American* computer recreations columnist, took the stage for his presentation, entitled "A Loose Cannon Takes Aim at APL". This slightly balding, initially rather shy character turned out to have a dry sense of humour. His delivery was a little reminiscent of Bob Newhart's.

His talk was divided roughly into three parts: Wallpaper for the Mind, Cellular Automata, and the Mandelbrot Set. It was really a reprise of some recent topics from his column, but the pictures he showed, together with detailed algorithms, were interesting and enlightening. I do not have the benefit of notes, but I remember that he showed an IPSA one-liner, worked out earlier that day with the help of Roger Hui, for Conway's Life.

To many of us who are, in Dewdney's words, Personal Programmers, the topics were familiar, but it was nice to see the algorithm for generating graphics from the Mandelbrot Set fully explained for once, rather than just touched upon. I myself took notes on a Cellular Automaton which he described, but I have not yet had time to implement it.

The highlight was definitely a video film entitled "Nothing But Zooms," a series of slow zooms into various parts of the Mandelbrot Set each lasting a few minutes. Even on a large mainframe, detailed pictures of large magnification take seconds to calculate, so the tape as a whole represented the expenditure of a great deal of CPU. Dewdney said the tape was available from Art Matrix, PO Box 880, Ithaca, NY 14851, USA. He did not know the price, nor whether it is available in PAL, the British video format.



Lee Dickey with A.K Dewdney

After the banquet, I retired like many others to the conference hospitality suite in room 4141 of the Hilton, where the fridge was stocked with large amounts of beer which had fallen off the back of a boat earlier in the week. Dewdney followed in time, and I discovered that Anthony Camacho had lapelled him (a rather more insistent version of buttonholing) and persuaded him to talk to me about the Life algorithm I have developed, since I wished to find out if he knew of any other similar implementation.

After finding the New York Cafe in the Hilton's basement closed, we settled in a bar in the lobby of the hotel which made Gatwick Village look stylish. I discovered that Dewdney is rather shy about treading in the footsteps of Martin Gardner and Douglas R. Hofstadter. He is a full professor, earning a $3/4$ salary for a $3/4$ timetable, and also runs a small software company to supply the readers of his column with copies of various program fragments, and is often in a panic about what to write about next. I heard a little about Hofstadter's departure from the magazine, and I wonder if Dewdney fears to follow too closely.



The Band at the Pre-banquet Concert

The main part of the all too brief discussion was made up of the discussion about Life. I described my algorithm, and he told me that he had not come across anything similar. We then went on to explore another potential improvement - that of temporarily eliminating currently 'stable' areas of the pattern, far enough removed from areas of

instability to be unaffected by them for some fixed number of generations, and continuing the iteration until this many generations had passed. For small patterns (up to 1000 cells on a side), the overhead in examining the plane to see what could be left out would probably slow the program down, but for large universes (as would be necessary to implement a Life Computer, for example) the improvements would be dramatic.

Over drinks, we explored again Conway's original intention - that Life could be a simple model for a real universe - and came to a startling conclusion when examining the new algorithm in this light. I shall assume a passing familiarity (which is all I have) with Quantum Mechanics in the following. Our own universe is believed to be non-deterministic in its evolution. That is to say, certain small scale quantum events occur randomly, though in statistically measurable ways. The evolution of a quantum state, however, is completely deterministic, and it is only when the wave function collapses to give 'reality' (some believe this happens when an observation is made) that the randomness encroaches. Now, we asked, why has God designed such a complex universe? Surely a simple mechanistic universe would be easier to design and implement on his cloudtop?

Let us suppose that he started with the simpler design, but it failed to give the organisational richness which our universe undoubtedly possesses. He tried more and more complex ideas, and eventually had to go non-deterministic to hope to get any worshippers out of the program. But his cloudtop is not the latest generation model, and all those pseudo-random numbers were taking an age to calculate. After all, at the rate at which random events occur in the nanoworld, a short cycle would result in clear determinism at the macro-level, and thus fail to achieve the desired complexity.

Ah, but wait! why not have most of the universe evolve deterministically most of the time for the sake of efficiency, and only introduce randomness where observers are present and observing. Why bother to actually simulate all those photons from that distant quasar taking every possible Quantum Electrodynamical route from source to target? Why not just have them appear, with statistical instantaneity, at the eyes of the sentient products of evolution? CPU cut by a fraction less than 100%, and no shortage of souls to save. WE were pleased with this discovery. I, for one, have been looking for a reason for all those integrals for some time.

But before the more pious readers of this piece cry "blasphemy", let me assure them that I have already been chastised. In the middle of this frenzied discussion we became aware, through the haze of smoke from St Moritz Extra Lights, of the strange figure of a man hovering near our table. We pretended to ignore him and he went away. Two minutes later he was back, and resolved this time. He stared at us like a Rangers fan finding a group of Celtic supporters in the corner of a Glasgow pub. Someone had to say something, and Dewdney came out with the beautiful reverse, "Was I looking at you?"

I remember only one sentence from our visitor in the short and uncomfortable exchange that followed. He was perhaps not an unlikely character given that we were in New York. He looked the worse for a drug stronger than coffee. Most of what he said was incoherent. But his admonishment, delivered with a zealot's passion, still rings clearly in my mind: "Remember: mathematicians are not philosophers."

Claire Sykes: A Paediatric Management Information System in APL

Reported by Anthony Camacho

The presentation was a demonstration with a large screen display of the system described in the paper. What came over was the amount of consultation with the nurses and doctors who use the system and the frequent adjustment of the system necessary to make the users feel completely at home with it. It is a well canvassed idea in the APL community that a system that meets 100% of the users' needs (and is kept up to date so that it continues near that level) is twice as valuable in the long run as one that only meets 90%. Of course APL can do this more easily than other languages but it is easy to underestimate the time and cost of the continual review and consultation with the users that is needed. It is rare and welcome to hear a fairly detailed account of how this is done: plainly Claire and Alan have made allies of their users, have not tried to impose their own views on how the work should be done and seem to have a very successful system as a result.

The prognosis is good too; they see no end to the adjustment and enhancement process and it seems quite likely that the information collected will have great value in improving the way such paediatric units care for their patients.

Eugene McDonnell: Phrasal Forms

Reported by Anthony Camacho

The suggestion is that a meaning is given to syntactic forms:

	function function omega	and
alpha	function function omega	and
	function function function omega	and
alpha	function function function omega	

or (as Iverson and McDonnell would say)

	verb	verb	pronoun		and
pronoun	verb	verb	pronoun		and
	verb	verb	verb	pronoun	and
pronoun	verb	verb	verb	pronoun	

Of course there is already an interpretation for some cases; + adds each item in alpha to the reciprocal of the corresponding item in omega. The extension is more obvious when plus-divide in parentheses is the left argument to scan, thus $(+ \div) \backslash$.

He displayed a table of logic functions with SAPL equivalents:

This was one of the cases where the standard half hour allowed for presentation was not adequate. Eugene McDonnell could only touch on the criteria which had led to the choice of the proposed interpretation. Your reporter is not familiar enough with the background and could not follow everything. A fuller explanation would have been welcome to at least this member of the audience. Eugene allowed for at least some of the people in this category by quoting many references for the interested to follow up:

- Combinatory Logic, Curry, Feys & Craig, North Holland
- Can programming be liberated from the Von Neumann style? A functional style and its algebra of programs. Backus CACM 21,8 (August 1978)



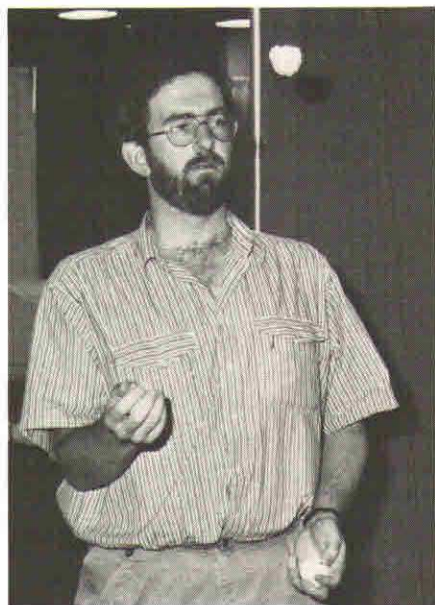
Eugene McDonnell

- To kill a mocking bird, Smullyan, Knopf
- The Mathematical Intelligencer, Springer-Verlag

After a short detour into programming without (explicit) variables (one of the criteria was minimising the number of assignments) Eugene ended with what he said is a solution to the coupon collectors problem. This reporter doesn't know what the problem is and doesn't understand the answer, but he wrote it down twice so this reproduction of it is quite possibly correct for anyone who does understand the problem:

$(x + / \text{ } \text{ } \div \text{ } \text{ } \text{ }) \omega$

[times plus over paws divide paws iota]



Jonathan Martin Juggling!

APL89 - Tutorial Sessions

Reported by Jonathan Martin

At APL89, I seemed to spend a great deal of my time in the tutorial sessions. This also gave me some ammunition for one of my great hobby horses - that is that APL is very good at doing some things, but there are commercially available packages that do the other things a hundred times better than APL can ever hope to. The only way forward is to enable APL to be the binding, to take up when these packages cannot cope. Here's what I thought of them.

Nancy Wheeler: APL2 & SQL

This was of great interest to me as the Dyalog APL-Oracle link that I use has its origins in this. I also feel that one of the major hindrances that I have come across has been

the inability of the rest of the world to access the data that we have had available to our APL systems. At last, here is a standard product that enables everybody to access some common data.

The talk was split up into three sections: SQL, APL2 and SQL & APL2.

The first section on SQL was a basic introduction to Relational Databases, the simple ideas behind them, the simple data constructions and a fairly brief introduction to the SQL language. This was great - about the limit of my knowledge!

Then Nancy talked about the APL2 Auxiliary Processor that they had written at the IBM lab in Santa Teresa. To use an APL-SQL link there is one major prerequisite - you must have a nested array APL. Relational data fits so well into nested arrays that one would appear to have been made for the other. The AP comes with a supplied workspace of about 20 functions which emulate the corresponding SQL features. Fortunately, there were lots of examples of the use of it e.g.:

```
PREP 'NULL' 'select * from ABC where NAME='WHEELER''  
OPEN 'NULL'
```

FETCH 'NULL' which gets all the fields from database ABC into the workspace for which NAME equals Wheeler.

The most important message of the talk, though, was left to the end. "To write a successful application one needs to consider such issues as database design, performance, storage usage, locking, and authority schemes as well as just 'getting the right answers'". How true - a tool as powerful as a relational database mis-used can cause far more damage than, say, a run-of-the-mill third generation language. A relational database used with APL could be the answer to all your problems - or just the start of them.

Kevin Weaver, Eric Baelen, Rob Lauer, Roy Sykes:

Software Tools for Enhancing Performance

Reported by John Searle

This session was a "must" for any APL*PLUS/PC application developers. The presenters have combined their resources and considerable experience to give us a free workspace of techniques for speeding up PC applications, achieved mostly by providing cover functions for Assembler or compiled C code. Eric Baelen presented a function for speeding up the wrapping of words in a text vector into a neatly formatted matrix, and functions for improving the robustness of joining text matrices together, and determining whether a sufficient number of files can be accessed for an application. Roy Sykes presented functions for rapidly searching the symbol tables in a library of workspaces for a wanted name, and for speeding up the operation of `⌵WGET` and `⌵WPUT`. Rob Lauer presented software which implements a complete Windows-like pulldown menu system, with mouse support. All the software was made available on diskette as part of the registration package, and is in the public domain. The benchmarks and documentation were printed in the "Session Tutorials" booklet.

Harry Bertuccelli: Exchanging Workspaces

My feeling on entering this tutorial was that it was going to be one of the more important of the bunch. Porting workspaces from one machine to another is something that all of us will probably get involved in someday with the advent of more and more powerful DOS and Unix machines. Fast going are the days when APL was solely a mainframe language.

But... it disappoints me to have to admit that this was one of the worst presentations I have ever attended at a conference. I lasted getting on for 30 minutes (it was scheduled for 90) before I decided that enough was enough. Most of what I say here is cobbled from the paper in the proceedings - little of note was said during the time I was in the room.

The method itself is fairly simple and standard.

1. Convert workspace into some text representation
2. Transfer text file to target machine
3. Convert text file back into workspace

However, Mr Bertuccelli tries to cloud the issue somewhat by writing out his text file with the last eight characters of his "80-byte 'card image'" with a sequence number. I may be short in the tooth but I can just about remember decks of cards - and yes, sequence numbers were fairly useful for the times you decided to juggle with them. But how often do IBM mainframes randomise the order of text files? Maybe I'm being unfair - the method is based on compatibility with the format of APL2's)OUT &)IN (which is a major plus point as this appears to be becoming the standard form) and maybe this is the reason.

The paper itself is somewhat disappointing also. Reading it, I found, was very hard work, I think because it is pitched at too low a level, preferring to explain the intricacies of "the loop ... beginning at line 5" rather than at the overview level.

Anyway, try reading the paper yourself if you can get hold of it - I'd be interested to hear what other people think about it and to know whether my view is clouded by the fact that I thought so little of the presentation itself.

Pamela Baron & James Kraemer: Desktop Publishing

This was billed as an introductory tutorial on Desktop Publishing (DTP) and was both professionally presented and pitched at the right level. Four major areas were covered: Hardware considerations, Software considerations, DTP Style and a small amount of DTP and APL.

The Hardware and Software considerations were based around the two scenarios:

"I have a computer. What software will give me the best publishing abilities?" (probably the most common problem) and "I want to publish long documents using Ventura Publisher. What hardware do I need?"

The basics for a decent DTP system are adequate memory (2Meg is a good base to start from), a monitor and a mouse. As far as extra memory and faster machines go, the need is regulated by the expectations of the user. I get frustrated using Pagemaker on a 286 machine with 640K memory though find it more than palatable on a 386 with 2Meg.

As to which package to go for, the list is endless with Pagemaker and Ventura being the front-runners in the DOS world. Although I have only ever used Pagemaker, the feeling seems to be that Pagemaker is best suited for small documents (say up to a dozen pages) while Ventura scores on much larger tomes.

Style, and this was rightly stressed in the presentation, is the most important factor in producing any document with a DTP package. Suddenly, going from a standard WP package to a fully blown DTP system, you are faced with what seems like a hundred and one different ways of doing the same thing. Should you use Helvetica for the text and NewCenturySchoolBook for the headings? What about some italicisation for section headings? How about some stresses here and there and should I use bold or underline? Some nice boxes around everything would look nice. Shall I use thin or thick lines?

As ever, simple is best. Use no more than two different fonts, one for headings (sans-serif) and one for the body of the text (serif) but try to break up pages and pages of text with the addition of graphics, pictures or something else appropriate. Standardize on the format throughout the document (fonts, pitch, headers & footers, margin widths etc.) and make sure you stick to it.

The paper provides a wonderful example of what Pamela Baron referred to as the "Ransom Note style". Needless to say, in eight lines of story (written by her young son James) four or five different fonts are employed along with random use of bold, capitals, italics and even underlined italics.

The majority of the talk managed to avoid mentioning APL, but I guess it had to rear its head sometime. Desktop Publishing and APL have, up to now, been very much like oil and water. Despite the numerous fonts available, the chances of getting an APL font were zero. Up to now that is, because the tail of the talk was to have been a demonstration by Ed Cherlin showing Pagemaker, running on a Mac, with a real, live PostScript APL font. Could this be the answer to the APL community's publishing problems. Very much yes and no! After showing off the APL fonts listed amongst all the other available fonts, as soon as Ed selected one of them and tried to use it, the package went haywire, spreading what he had typed in in a rather random fashion across the screen. End of demonstration!

One day (and I don't think it will be too far away now) we will have a true, fully scalable PostScript APL font. I just wish one of the big companies would fund it and have it done properly rather than relying on individuals having to spend their own time cobbling it together themselves. Then we could take another step towards acceptability in the outside world.

Judson Rosebush: Using APL for Computer Graphics Notation

The presentation and the paper in the proceedings were very different for this talk. The presentation mainly consisted of a number of slides and videos of some stunning computer-generated graphics, some that Judson Rosebush had been involved in, some that were just so good he wanted to show us anyway.

However, it is the paper that I will discuss, as there are some very nice algorithms for playing around with graphic shapes. Given an image, there are three main transformations you can perform on it: **translation** moves it left, right, up or down; **scaling** makes it bigger or smaller; **rotation** gives a rotary motion about the origin.

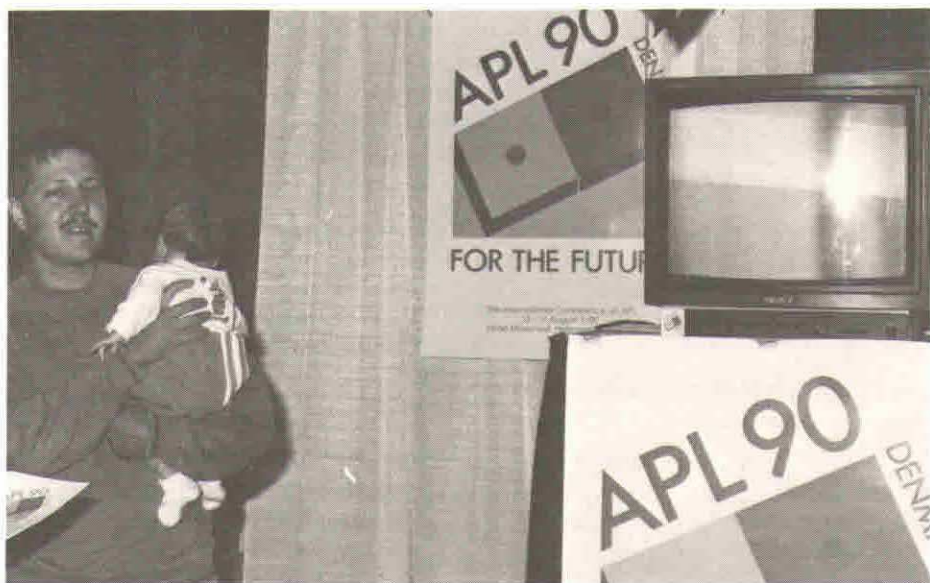
- Translation is the simplest - if your object is in coordinate form, simply add or subtract constant values to it to move it around.
- Scaling is almost as simple. Rather than add or subtract, multiply your coordinate representation by the proportions you wish to scale the object along each axis.
- Rotation, which is probably the most interesting, is, of course, also the most difficult. Simply, it involves a couple of sines and cosines and both a minus and a plus reduction.

A much neater way of doing these translations is through the use of homogeneous coordinates and transformation matrices. A homogeneous coordinate transformation matrix consists of as many rows as there are vertices in the object and columns one greater than the number of axes. The extra column is referred to as the homogeneous coordinate and has the value 1. The transformation is performed using $+ . X$ with the data being the left argument and a Transformation Matrix the right. This transformation matrix, in its various forms, copes with translation, scaling and rotation, simply by using different versions of itself.

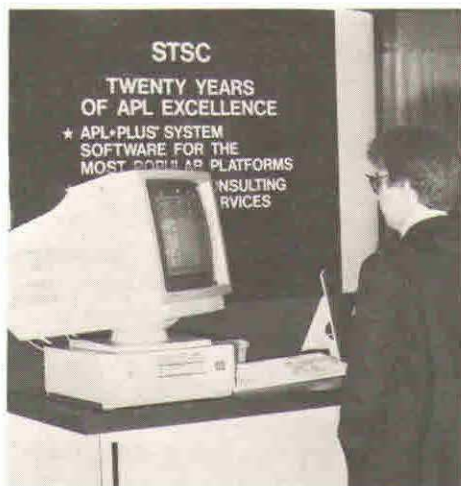
This method of transformation matrices also enables us to combine transformations, without transforming our data until the very end. For example if you wish to scale and translate your object, it is possible to combine the two TM's into a third and operate that on your data, rather than having an intermediate form. Again $+ . X$ provides the link, converting our two initial translation matrices into the new third matrix

TM3 gets $TM1 + . X TM2$ then DATA gets $DATA + . X TM3$ If you are at all interested in graphics and some simple animation techniques, then this is a good, fairly readable paper with quite a few examples that work. My only trouble was trying to cope with plotting in 3-D.

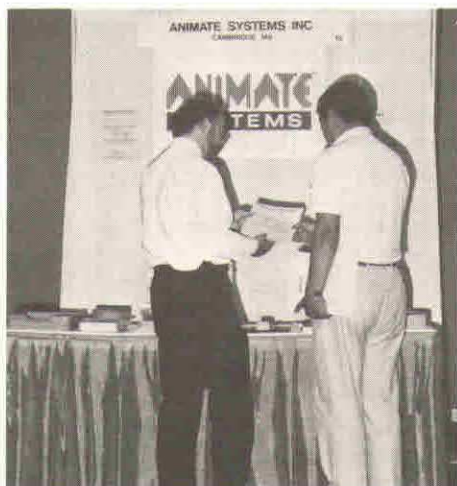
The Exhibition



The Youngest Delegate at the APL90 Stand



The STSC Stand



Animate Systems

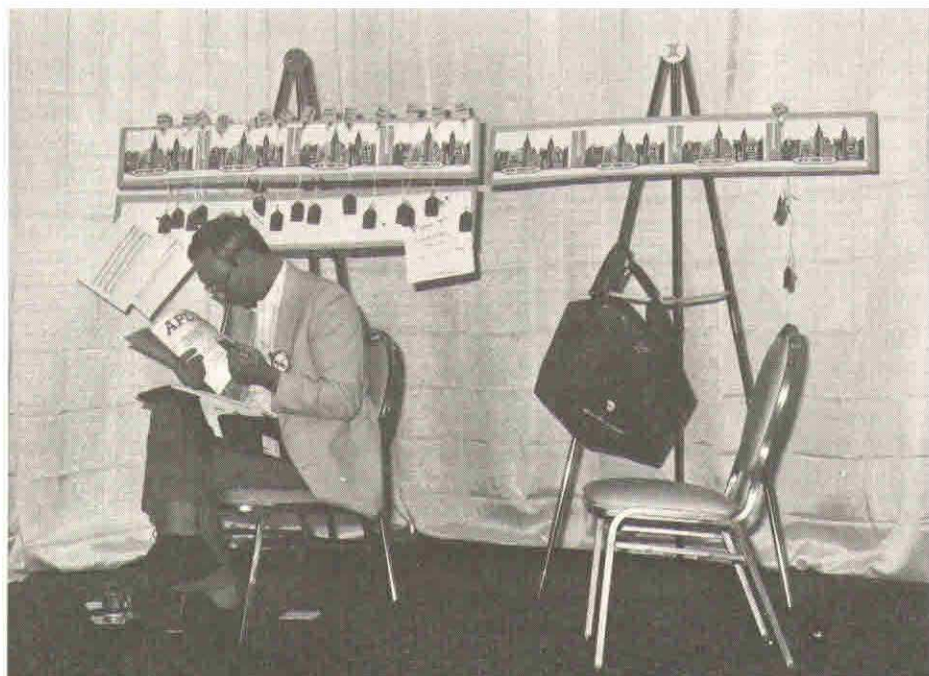
photographed by David Ziemann



Ron Wilkes with APL2/PC



Nancy Blachmann demonstrating Mathematica



APL as a Tool of Thought

The Bonfire of the Variables

by John Searle

APL 89 was a "steady as she goes" conference. No new or revolutionary products were announced, but it was heartening to see the major vendors pushing ahead on several fronts. Several new releases of existing interpreters are pending.

The conference theme was "APL as a Tool of Thought", borrowed from the title of a series of past conferences run by New York SIGAPL. Many papers illustrated the usage of APL as an effective medium for describing and expanding upon an idea or concept. The practical nature of APL also makes it "a tool for Measuring the Effectiveness of Thought".

The plenary sessions of the conference brought together speakers from various disciplines who have all pushed against and extended the known boundaries of their fields. They were not all APLers, but they have all used the essential ideas of APL in their search for simple yet powerful methods of specification and analysis.

The impact of Mathematica on the computing community will be interesting to monitor. Its author, Stephen Wolfram, acknowledges the parentage of APL in devising its functionality. However, its notation and "flavour" is more closely related to Lisp and Prolog. It uses only the standard symbols found on a keyboard. It is available for 386/DOS machines and the Apple Macintosh, and comes bundled free with the purchase of a NeXT computer. It will probably sell more copies in its first year than APL has ever sold.

The conference program was a good blend of the practical and the theoretical, with something for everyone. Presentations were generally more ambitious than in the past, and the necessary high-tech equipment usually worked well.

The refereeing of papers was questionable. Did the referees know in advance that their names wouldn't appear in the proceedings? Several papers were thinly disguised commercials for vendors, notably "APL in the Workstation Environment" (STSC). A workstation was defined as anything from a PC upwards, and the advertisement went on from there. Other papers did not acknowledge highly relevant work from recent conferences. This is not the author's fault, but that of the referees. SIGAPL is trying to rectify this in part through its Index of APL Articles.

The social side of the conference was, like the weather, hot and cold. The boat ride around New York Harbor was excellent. It was amazing how pretty Manhattan became after it got dark and you could only see the lights, not the architecture. The queue for dinner was a bit long, but the price was right. Thanks to the Danes for providing enough Carlsberg

to lubricate the evening (and the next two evenings of private suite hospitality as well). The conference lunch, with musical entertainment by Stanley Jordan, was a real highlight. He uses APL to transform his musical score, using a simple notation. He played a tape of some of his pieces, and they were surprisingly tonal. His next jazz album will apparently acknowledge APL amongst the synthesizer credits.

The conference banquet was a mixture. The food was superb but its distribution was disorganized. I realise that the conference finances did not allow for the provision of any free drinks, but an announcement to that effect would have saved confusion and hoarseness. A.K.Dewdney's talk was entertaining and his Mandelbrot video was riveting. Afterwards, he was bombarded by people hoping to prove that anything Scientific American recreational writers can do, APLers can do better. We already know this, but does the world want to?

Conference handouts and freebies tend to reflect the character of the host city. Previous U.S. conferences have featured coffee mugs, Manchester gave us an umbrella and Sydney a T-shirt. Thus it was entirely appropriate that APL 89 delegates should receive a pocket knife, saw and screwdriver set - a personal defence mechanism suitable for late night subway rides. Readers of Tom Wolfe's famous novel of New York low-life certainly appreciated this data.

Overall, the conference was good value but the Hilton was expensive. It was ironic that the Hilton was put up for sale on the day after the conference. Are APLers improving at running conferences? On this evidence, the answer has to be "no". The American style of big inner-city hotel conventions just seems to be too inflexible, too expensive and too impersonal. The greatest minds in the APL community are not improving at getting their message across to a live audience. The lessons of past conferences are not being learned. Over to you, SIGAPL.

See you in Copenhagen!

British APL Association Prize Draw

Win A Free Trip to APL90 In Denmark!

As part of this year's campaign to recruit new members, the British APL Association is running a **PRIZE DRAW** open to all existing UK members who manage to convince another APLer of the advantages of joining!

All you have to do is to introduce a new member before 1st February 1990 and your name will be entered in the Draw to win a fabulous free trip to next year's International APL Conference!

The Conference will be held from 13th to 17th August 1990 at the Marienlyst Hotel, Helsingor, near Copenhagen, and the lucky winner will travel out at the British APL Association's expense from anywhere in the UK. As well as having the delegate fee paid for by the British APL Association, accommodation and meals at the Marienlyst for the duration of the Conference will also be included.

Helsingor is a picturesque but thriving seaside town on the North coast of Denmark. It has plenty to offer the visitor: a historic port, nearby beaches and scenic countryside, a mediaeval 'Old Town' with cobbled streets, numerous parks, museums and monuments, and a plethora of cafes, restaurants and shops.

The theme of the Conference at Helsingor is 'APL For The Future', and the emphasis will be on the importance of APL as a tool for Information Management in the next decade. As well as presentations designed to illustrate this, there will be plenty of opportunity to meet and exchange experiences and ideas with other APLers from all over the world!

You can enter your name in the Draw each time you persuade someone new to join up! The more new members you sign up, the more chances you have of winning! But they must be genuine NEW members and not lapsed members who are rejoining. Photocopy the form on the opposite page and complete a copy for each new member who joins through you.

Send the completed forms to Alison Chatterton, at the address opposite. Please mark your envelope 'APL90 Draw'.

APL90 Prize Draw: Entry Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April. This form may be photocopied: send one form for each new member you introduce.

Name of existing member

(to be entered in Draw):

Organisation: _____

Address Line 1: _____

Address Line 2: _____

Address Line 3: _____

Postcode: _____

Telephone Number: _____

I have introduced the following person to membership of the British APL Association:

Name of new member: _____

Organisation: _____

Address Line 1: _____

Address Line 2: _____

Address Line 3: _____

Postcode: _____

Telephone Number: _____

Date Joined: _____

Please send the completed form to:

BAA Administration,
Alison Chatterton, 9 Oak Grove,
HERTFORD SG13 8AT

Rules

This competition is not open to members of the BAA committee, the Vector Working Group, or their families. If, for some reason, the winner is unable to make use of the prize trip to APL90, then a cash sum of £500 will be offered instead. The closing date for receipt of entries is 1st February 1990.

The APL Connection

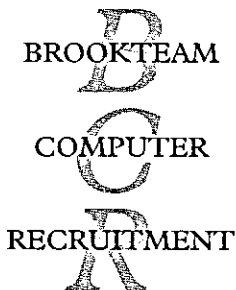
Brookteam Computer Recruitment has become the market leading recruitment consultant in the APL arena. Our professional approach and daily contact with major software houses and end users in the APL market ensures we are able to offer the best service possible, for both clients and applicants alike.

For Applicants:

We always have a good range of nationwide vacancies, from Graduate to Project Manager level and we are always happy to offer our expert advice on salary expectations and relocation packages, you can be assured of our absolute confidentiality at all times.

For Clients:

If you are not already utilising our recruitment services - we need to talk! Due to our high visibility within the weekly computer press, we have the most comprehensive database of applicants available, and we are always happy to advise you on salary levels and staff requirements.



5/6 SPRING BRIDGE MEWS, LONDON W5 2AB.
FAX: 01-579 5291. TEL: 01-840 5516

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner	Adrian Smith	114
APL Standards Update	David Ziemann	118
Punctuation in APL	Joseph De Kerf	123

Both I.P. Sharp and IBM have offered us Error Trapping tutorials along the same lines as those published in VECTOR 6.1. These will appear in the January VECTOR as this one is already over-full!

Hacker's Corner: Getting Another 128 Colours on your EGA

by Adrian Smith

Once again, Hackers' Corner takes a delve into the innermost secrets of your PC, but this time it is all decent and honest BIOS-compatible stuff! Sorry John, and all you hackers out there who found that getting at your PSP was less than easy under the PLUS II system: I hope this one will work for you!

Those of you with colour screens will have noticed that several packages allow you the full range of 16 background colours as well as the normal 16 in the foreground. However if you are tempted to try this with say `□WPUT 143`, all you will get is blinking white on black, rather than white on a nice dark grey. Now one of the packages which does give you the option of bright backgrounds is APL2/PC, so I rang Dave Selby and asked how he did it. In fact it is a standard BIOS call, but only applies to the EGA and VGA monitors, so none of the usual PC reference books mention it.

The little function below will do the trick, or you could just close your eyes and type `4099 □INT 16` if you prefer.

```

▽ BLINK SW;RG
[1]  A FLIP BACKGROUND FROM BLINKING TO INTENSIFIED
[2]  A AH = 10H      ... BIOS SERVICE 10
[3]  A AL = 3H      ... SUBSERVICE 3
[4]  A BL = 0/1     ... 0 = BRIGHT; 1 = BLINK
[5]  RG←(4099,SW>0)□INT 16
▽

```

A couple of small caveats: Ctrl-Esc resets the screen to the default (i.e. blink) state, as does Desqview if you run your APL session in a window. I have incorporated the above code in all my screen routines (specifically `WINΔINIT` and `FSMΔFMT`) so that I don't have any problems restarting after a break.

The improvement in appearance of standard data-entry screens is considerable (I recommend almost any colour on dark grey as a background, bright white on bright blue for fields, black on yellow for warnings, bright white on bright red for critical errors), and you can do a fair impression of OS/2 Presentation Manager with plenty of bright white as background!

Have fun!

Technical Correspondence

Hackers' Corner: DOS Environment Variables

From: John Sullivan

17th July 1989

What is all this hacking around with Program Segment Prefixes? Has anybody tried to do this in APL*PLUS II? No? - Good, because it doesn't work. However, try the following which does work in both APL*PLUS/PC and APL*PLUS II:

```
VZ←DOSΔENV
[1]  □CMD 'SET > SETOUT.TXT'
[2]  'SETOUT.TXT' □NTIE 158
[3]  Z←□NREAD 158 82, (¯1+□NSIZE 158), 0
[4]  □NUNTIE 158
[5]  □CMD 'DEL SETOUT.TXT'
[6]  Z←SSTOMAT □TCNL,Z~□TCLF
V
```

Where SStoMAT is supplied in STSC's ASMFNS and is the equivalent of your ΔTAB.

24 Mounthurst Rd
Hayes, Bromley
Kent BR2 7QBN

More on Finding the Location of that PSP ...

From: Dave Phillips

15th September 1989

I was delighted to read that most useful piece of insight regarding how to acquire DOS Environment Variables from an APL*PLUS/PC system. (VECTOR Vol.5 No.4 - Hacker's Corner). The ability, as you so rightly say, for one's APL system to get runtime information from the external environment (e.g. the actual name of a data-file to use - almost like tying a DDNAME to a DSNAME in MVS/JCL, only better!) is most appealing.

I would like to point out, however, that the piece of code you supplied will only work for the Standard APL*PLUS/PC System. This is because the location of the Program Segment Prefix address is no longer at the same offset to the function key table in the new Extended System (with Session Manager) that is available in APL*PLUS/PC r8. All is not lost; an alternative method of locating the PSP is provided by DOS itself and is applicable to all flavours/releases of APL*PLUS/PC.

I was recommended to closely examine the IBM DOS Technical Reference, (I have DOS 3.30), wherein is to be found a list of programmed calls that can be made via Int21. A call 62h to this interrupt yields, as a return value, the address of the PSP directly. At least it does for DOS 3.0 and above; for earlier DOS releases, use of the undocumented call 51h produces the desired effect. From there on, your code, or rather M. Philippe Denoyelle's code, works as is. (Monsieur Denoyelle: merci tres bien!)

I am including a copy of the reworked code below. I hope this may be of some additional benefit. One further point I should make is that this approach is not applicable to the APL*PLUS II System. However, I do have a solution for that environment, care of a private communication from STSC, should you or other VECTOR readers wish to have it.

Dave Phillips
Principal Support Consultant
Cocking & Drury Ltd.

```

▽ R+DOSΔENV SEL;AV;DOS;ENV;PSP;V;X;IO;SEG
[1]  AVDETERMINE DOS ENVT VARIABLES WITH OPTIONAL PREFIX <SEL>
[2]  AVUNDER APL*PLUS/PC, INCLUDING SESSION MNG IN /PCR8, ET SEQ.
[3]  AVUSAGE: R+DOSΔENV ' ' RETURNS ALL DOS VARIABLES
[4]  AV R+DOSΔENV 'C' RETURNS ALL DOS VARS. STARTING 'C'
[5]  IO←0  V←0
[6]  A DETERMINE DOS VERSION USING INT21H CALL 30H
[7]  A (N.B. CALCULATE 30H←1613 0 0 0 - I.E. 12288DEC)
[8]  DOS←1+0 256 256 T''012288 INT 33
[9]  A FIND PSP ADDR. USING INT21H CALL 62H (25088) FOR DOS 3.0+
[10] A OR CALL 51H (20736) FOR PRE-DOS 3
[11] A PSP SEGMENT ADDR. IS SECOND ELEMENT OF RETURN FROM CALL
[12] PSP←((25088 20736)[DOS<3]INT 33)[IO+1]
[13] A OFFSET 2CH (44) IN PSP GIVES SEGMENT ADDR. FOR ENVIRONMENT
[14] SEG←PSP 0 ENV+2561PEEK 45 44
[15] L1:SEG←ENV 0 V←V,PEEK1256 0 X← 0 0 0 V 0 →(V/X)/L2
[16] ENV←ENV+16 0 →L1 A LOOP UNTIL WHOLE TABLE IS READ
[17] A TRANSLATE RETRIEVED NUMERIC VALUES INTO TEXT
[18] L2:AV←AV
[19] A AV[64 36 61]←'ΔΔ' A OPTIONAL TRANSLATE TAILORING
[20] V←AV[(X11)1V] 0 R+SSTOMAT TCNUL,V
[21] SEL←,SEL 0 →(0SEL)0 A IF <SEL> IS REQUESTED...
[22] R+(R[10SEL]Δ.=SEL)R A ...TAKE SUBSET
▽

```

Ambivalent Operators ...

From: Phil Last

12th July 1989

With reference to David Piper's letter in VECTOR 5.4 ... The right operand to David's operator must be parenthesized to force the value rather than the `□NC` function itself to become the operand:

```
[2] →(↑(RUN AVE)+ (ρNUMS)(AVERAGE ΔMONADIC(□NC 'SIZE'))NUMS)/0
                                     ↑           ↑
```

Once this is done, it works! However both the operator itself and the convoluted expression required to call it are perfectly described by the editorial from the same edition! Nevertheless, if we're after a tricky solution, the following avoids all conditional code, be it branch or execute, it tells you what it's doing, and it works faster:

```
▽
[0] AVE←SIZE AVERAGE NUMS
[1] A MOVING OR NORMAL AVG DEPENDING ON EXISTENCE OF SIZE
[2] AMBIVALENT 'SIZE'
[3] AVE←(SIZE+ /NUMS)÷↑SIZE, ρNUMS
▽

▽
[0] AMBIVALENT AMBIVALENT
[1] A DCL CALLING FN AMBIVALENT WITH NAME OF ITS LEFT ARG.
[2] AMBIVALENT←□FX('R←', AMBIVALENT, ' B')('R←B')
▽
```

The function AMBIVALENT, given the name of the left argument to its calling function, attempts to fix a function with that name which returns its only argument. In the example, this would be:

```
[0] R←SIZE B
[1] R←B
```

This can only be fixed if SIZE does not already exist, in which case the call to plus reduction is genuinely monadic and the right argument to divide defaults to the shape of NUMS.

Phil Last
Wisteria Lodge
140 Crossbrook Street
CHESHUNT Herts

APL Standard Update

by Dave Ziemann (as convenor of BSI APL Standards Panel)

This report primarily covers events that occurred at the ISO meeting held in Princeton University, New Jersey, USA from July 31-August 3rd this year, just before the APL conference in New York city.

The four day meeting of Working Group 3 (WG3) attracted fourteen delegates from Belgium, Canada, France, UK and the USA. Two BSI (British Standards Institute) representatives attended the meeting, Paul Chapman and myself. The meeting was convened by Lee Dickey. Ken Iverson, now retired from I P Sharp Associates was a notable attendee.

The main agenda item was of course the Extended APL Standard, although the status of the original APL Standard, affectionately known as IS8485, was also on the agenda.

The First APL Standard

What news of IS8485 then? Lee Dickey had submitted the document to Sub-Committee 22 (Programming Languages) in Geneva for "International Standards processing" some time ago. Although this at first appears to be a technical term for a bureaucratic black hole, it now seems that we are in imminent danger of having an APL standard. If there are no hold-ups, we can expect publication before the end of this year, apparently.

If you want to get hold of this 275 page document, request document IS8485 from:

Secretariat Central ISO
Case Postale 56
CH1211 Geneve 20
Suisse

The standard will be published in English and French, so you should state which you want. A price is not currently available.

Once IS8485 is published as an ISO standard, it can be "endorsed" or "adopted" by the national standards bodies, such as the BSI. "Endorsing" effectively means that the national body establishes a pointer to the international standard, and you have to send to Geneva to get a copy, whereas "adopting" means that it is accepted as a national standard and printed locally.

The original text of IS8485 is currently being transferred from Script into Tex, a project which has been funded by SIGAPL and more recently by I P Sharp Associates. This will

bring the document under the control of WG3 once more, and should make work on a new document much easier.

The Extended APL Standard

For some time the ISO group has been struggling with the problem of how to resolve some fundamental differences in the way that certain enhanced facilities should be standardised. In particular, nested arrays (where an array can contain items other than simple scalars) and the reduction operator are subjects upon which effective agreement cannot be found.

At the ISO meeting the BSI submitted a paper which summarised and enumerated the logical possibilities open to WG3, and suggested that WG3 reconsider its direction. The choices open to us are:

1. Standardise the intersection of the extended facilities in the different dialects
2. Standardise the union of facilities by a very general approach
3. Standardise a new dialect of APL which combines desirable facilities from each dialect
4. Standardise multiple facilities, allowing implementations to select the preferred approach
5. Pick one approach to standardise (essentially APL2 or Sharp APL)
6. Develop two distinct extended standards
7. Postpone work on the Extended APL standard

None of these possibilities is particularly attractive, each resulting in severe technical, political or practical difficulties.

Although possible, compromise solutions involving the combination of facilities, such as in option 3, damage the consistency and synergy of the language, and would result in a standard language which does not actually exist. It has long been the policy of the working group to standardise on the basis of what exists rather than to invent, wherever this is possible. But standardising from what exists in two distinct self-consistent dialects can produce a monster without the consistency or usability of either one.

A standard which includes both APL2's nested and Sharp's boxed arrays, for example, would be a standard for a language which no-one has used, and perhaps one which no-one would want to use. The behaviour of all the associated primitives in one school would have to be carefully defined on arrays of the other school, and vice versa. Even if it were

possible, the vendors could not afford to change their systems to conform to such a standard, and the document would then serve little purpose.

A Way Forward

Work on the standard proceeds by discussing and voting on proposals received by WG3. We decided that proposals should be as near "standard-ready" as possible, ie they should be in the same format as pages in IS8485. Before the Princeton meeting many topics had already been rejected by agreement or vote, although several contentious proposals still remained.

After discussion WG3 decided to prepare two publications, a Supplement and a Technical Report. Our main effort, a Supplement to the existing standard, will contain only those proposals upon which agreement has been reached. The Technical Report will describe those topics which have been proposed, but on which consensus has not been reached.

During the course of the meeting twenty-six topics were identified for likely inclusion in the proposed Supplement, and nineteen topics were identified for likely inclusion in the proposed Technical Report. Two topics were dropped from further consideration because proposals in the appropriate format had not been received.

The topics currently marked for inclusion in the Supplement are: replicate, grade extensions, without, underbar/overbar in names, ambi-valent defined functions, ambi-valent primitive functions, transfer form ($\square TF$), defined operators, canonical representation of defined operators, locking of defined functions and operators, shared variable extensions to $\square SVR$ and $\square SVQ$, $\square NMS$ and $\square OPS$ system commands, N-wise reduction, comma-bar, nub, left and right, GCD and LCD, commute, complex numbers, formatting complex numbers, heterogenous arrays, fill-items for heterogenous arrays, function rank operator, alpha and omega as names, event handling ($\square EA$ $\square EM$ $\square ES$ $\square ET$) and component file cover functions.

The topics marked for inclusion in the Technical Report are: reduction, nested arrays and fills, enclose/disclose/box/open, formatting for nested arrays, pick, partition, depth, link, enlist, strand notation, match, primitive operators applying to non-scalar functions, compose, dual, first, each, national language characters, function rank in standard metalanguage, function assignment.

The two topics dropped from further discussion were $\square FMT$ and the "find" function for string searching.

Future Issues

It is unfortunate that the projected Supplement does not include any nested or boxed array extensions, although this is ameliorated to some extent by inclusion of the function rank operator. Many people feel that an extended standard should include nested arrays, whereas others think that the field is still undergoing evolution, and that it is too early to standardise. No doubt this subject will arise again.

Another interesting area is event-handling. Where the model in the standard signals an error, a conforming implementation may replace that error behaviour by some other behaviour - this is the "consistent extension" mechanism by which implementors may add facilities to their conforming systems. Therefore an APL program cannot strictly rely on this error being signalled, and so such a program is deemed non-conforming.

The whole purpose of including the (APL2-style) event handling system functions in the standard is to allow users to trap events and handle them, so currently we have the strange situation that a facility defined in the standard cannot be used by a conforming program! Clearly something will have to change in the next standard.

The meeting also agreed to liaise further with WG2, the ISO committee involved with the standardisation of character sets, and we will be sending them a list of APL characters for inclusion in the new multiple octet character set standard.

Another sad divergence between the APL2-school and the Sharp approach is the reduction operator, and therefore inner product. In APL2-type systems the primary definition of reduction is on vectors, the function being applied between the items of a sub-array of vectors. The Sharp philosophy is to apply the function between the items of a vector of sub-arrays.

In addition, the APL2 definition involves an implicit enclose operation, which means that the result is often nested, whereas the Sharp definition does not involve enclosure, the result being simple. The enclose allows the APL2 definition to preserve the familiar rank-reducing identity, but in the Sharp view that effect was merely due to the domain of the operator being restricted to scalar functions, and is not a general identity. Perhaps for that reason the true Sharpie prefers the name "over" to "reduction". I heard one Sharp advocate offer the opinion that Bob Smith, the inventor of STSC's Nested Arrays Research System, had been "imprisoned by the beauty of the identities he discovered".

Of course, none of these differences manifest themselves in old-fashioned APL, but only become apparent with the requirement to generalise operators to non-scalar functions.

Standards meetings evoke a mixture of feelings; they can be extremely fascinating, sometimes boring, and very often humorous. When faced with a 6-3-3 vote to include the function rank operator, the convenor was not sure that this indicated consensus. So a

metavote was agreed, and we each picked a number greater than 50 and not greater than 100, the percentage which we thought fairly represented consensus. We agreed to pick the median number, and the result was 70%. A revote was then called for the function rank operator, and this time it passed 8-3-1.

The Timeline

The next scheduled meeting will take place in August 1990, somewhere in Europe. If sufficient progress on the Supplement has been made by February 1 1990, then an additional meeting will be scheduled for April 1990 in Waterloo, Ontario.

It is hoped that the new document will reach the first Draft Proposal stage by March 1991.

UK Involvement at Risk

Travel to International Standards Organisation meetings for BSI representatives is funded in this country by the Department of Trade and Industry. One representative, nominated the "Principal Expert" is entitled to a fixed amount based on the location of the meeting. For example, for the Princeton meeting the figure was £500, enough to cover a cheap air fare to New York. Accommodation and living expenses are not paid; the expert must rely on a very generous employer or bear the cost directly.

All BSI Technical Committees have now been informed that future claims may not be met, and those that are may have to be at a reduced rate, possibly no more than 50%. Perhaps as many as nine out of ten claims will be rejected.

It therefore looks likely that future UK involvement in ISO APL standards meetings will not be possible. If anybody has any complaints about this they can write to:

with a copy to:

Mr Alec Berry
Dept. of Trade and Industry
Kingsgate House
68-74 Victoria Street
LONDON SW1 6SW

Mr Tony Valvona
Business Support Group
BSI Standards
2 Park Street
LONDON W1A 2BS

A Note on Separators in APL

by Joseph L.F. De Kerf

Abstract

Some details and commentaries are given about the use of the semi-colon character, mixed output, the diamond statement separator, and the functions lev and dex. It is claimed that Extended APL, the new ISO APL Standard under development, should include the diamond statement separator.

The Semicolon Character (;)

The APL character set may be subdivided into the following four sub-sets:

- the alphabetic characters;
- the numeric characters;
- the special characters;
- and the blank character.

Most of the special characters are used to represent the primitive functions and operators. Some of them however are used for other purposes such as punctuation and as delimiters/separators. One of them is the semicolon, which is used:

- as a separator for heterogeneous or mixed output. A sequence of expressions separated by semicolons is evaluated from right to left and the values of those expressions are displayed from left to right, e.g.:

```
X+2
Y+3
'THE SUM OF X AND Y IS ' ; X+Y
THE SUM OF X AND Y IS 5
```

- as a separator for bracket indexing of higher rank arrays (e.g. $A[I;J]$, $A[I;J;K]$, etc.) and as a separator for specifying subarrays when using the bracket axis operator (e.g. $Z+F[KZ;KB]B$ and $Z+AF[KZ;KA;KB]B$). The bracket axis operator is only supported in APL2 IUP (IBM). The facility is not supported, either in APL2 PP (IBM), or in any other commercial implementation of APL.

- as a separator in the list of local variables in the header of a defined function or operator (V-----;L1;L2;L3;...). In APL2 IUP & PP (IBM), separating 'local names by semicolons is optional on input, but on display the semicolons are always included in the operation header.
- with the dyadic format system function $\square$ FMT, as a separator in the right argument list of expressions to be formatted:

```
Z←A  $\square$ FMT B OR
Z←A  $\square$ FMT (B1;B2;B3;...)
```

- with the APL68000 (TCC - MicroAPL) string search and replacement system function $\square$ SS, as a separator in the right argument list specifying the target string, the substring(s) to be searched, and eventually the replacement substring(s):

```
Z← $\square$ SS (STR;SUB)
Z← $\square$ SS (STR;(SUB1;SUB2;...;SUBN))
Z← $\square$ SS (STR;SUB;REP)
Z← $\square$ SS (STR;(SUB1;SUB2;...;SUBN);REP)
Z← $\square$ SS (STR;(SUB1;SUB2;...;SUBN);(REP1;REP2;...;REPn))
```

a similar mechanism is used with the text editing system function $\square$ TE in Xerox APL (Atkins) and OS4000 APL (GEC).

- with the nested arrays implementation of AOS/VS APL (Data General), as a separator in list notation, a shorthand for catenation of two or more enclosed arrays:

$$(A;B;C;...;Z) \leftrightarrow (<A),(<B),(<C),...,(<Z)$$

Note: In APL90 (SYNC), the underscored semicolon (;) may be used with mixed output, suppressing the display of the result of the right hand adjacent expression.

Mixed Output

Heterogeneous or mixed output was introduced in APL360 [1] to allow the mixture of character and numeric output on a single line. Expressions are separated by semicolons. As stated above, the sequence of expressions is evaluated from right to left and the values of these expressions are displayed from left to right. Each value separated by semicolons is displayed - even if the expression is an assignment. Arrays of rank two and greater however are displayed on separate lines.

```

A←20
'THE VALUE OF A IS';A
THE VALUE OF A IS 20

```

```

A←2 3ρ16
'THE VALUE OF A IS';A
THE VALUE OF A IS
1 2 3
4 5 6

```

Mixed output of course does not produce a single array as a result: the output cannot be assigned to a variable and cannot be used as the argument to a function or an operator.

In 1970, A.L. Anger [2] recommended adopting a character-conversion function - possibly monadic $\overline{\epsilon}$ - which would produce a character array identical in displayed appearance to the numerical argument. Such a primitive function was implemented, originally as the monadic enquote $\overline{\epsilon}$, later on as the now well-known monadic format $\overline{\epsilon}$.

Combination of catenation and this monadic format, is much more powerful than mixed output [3]. Results to be displayed are combined into a single character array. In addition, character arrays and formatted numeric arrays of higher rank may be displayed side by side.

```

A←20
'THE VALUE OF A IS ',⍕A
THE VALUE OF A IS 20

A←2 3ρ16
(2 18ρ36⍕'THE VALUE OF A IS'),⍕A
THE VALUE OF A IS 1 2 3
                  4 5 6

```

The use of catenation, combined with monadic format, may be more desirable and more powerful, but it is less handsome. In addition, the type of the numeric data is lost. In APL2-like implementations however - those which permit heterogeneous arrays - the results to be displayed, whatever their type or rank, may be juxtaposed as items of a heterogeneous nested vector. Vector notation may be used, and as such displaying even the most complicated configuration of results becomes very simple, preserving the type of the numeric results.

```

A←20
THE VALUE OF A IS' A
THE VALUE OF A IS 20

```

```

A←2 3⍲16
THE VALUE OF A IS' A
THE VALUE OF A IS 1 2 3
                  4 5 6

```

Taking it all in all, mixed output is considered obsolete [4]. It was a convenience whose necessity has been outlived [5]. On the other hand, several rather recent implementations of APL still support mixed output. A list of more than fifty past and current implementations of APL, indicating whether mixed output is supported or not, is given in Appendix 1.

The Diamond Separator (◇)

Heterogeneous or mixed output is not a so-called "proper APL construction" [5]. In any case, it is not a statement separator:

- The sequence of expressions are evaluated from right to left;
- Results of rank zero or one are displayed side by side;
- All expressions listed must return a value;
- Some constructs like branching are not allowed.

An authentic statement separator, the diamond was introduced by the Scientific Time Sharing Corporation - now STSC - in its APL*PLUS System in 1974 [6]. In execution mode as well as in a defined function or operator, statements normally listed from top to bottom may be written from left to right:

```

-----
STATEMENT1
STATEMENT2
STATEMENT3
-----

BECOMES FOR INSTANCE

-----
STATEMENT1◇STATEMENT2◇STATEMENT3
-----

```

The sequence of statements is executed from the leftmost statement to the rightmost. A branch within the line transfers control immediately. A conditional branch in a line to an empty vector however executes the next statement to the right in the same line, if there is one. If an error is encountered, the error report is displayed and execution is suspended in the usual way. Statements to the right of the statement in error are not executed. Execution can resume only at the leftmost statement in the line. A weak interrupt has an effect only between successive lines, not between statements in the same line.

In addition, within a defined function/operator, the following interpretations also apply [6]: the entire line has only a single line number; a label can only appear at the left end of the line; a branch to the line transfers control to the leftmost statement in the line. Tracing a compound statement displays the function/operator name, the line number, the value for the leftmost statement, and on separate lines, a diamond, some blanks, and the value for each subsequent statement on the same line. Finally, the comment feature is independent of the presence of diamond separators.

The diamond separator feature makes it easier to write more concise programs. Whether replacing a list of statements on successive lines by a one-line list of statements separated by the diamond improves readability, is open to discussion. In any case, "it reduces the temptation to write obscure statements just to reduce the number of function/operator lines".

```
I←0 ♦ V←1 ♦ R←3 ♦ J←1
```

is clearer than

```
J←x-R+3, 0pV←1~I←0
```

In addition, as a side effect, replacing a list of statements on successive lines by a one-line list of statements separated by the diamond may save memory space and CPU time [7].

IBM implementations of APL do not support the diamond separator. On the other hand, most other current implementations do support this feature. A list of more than twenty of them is given in Appendix 2. Results of a benchmark, showing that for the examples, memory space and CPU time is saved, are reported in Appendix 3.

The Lev (-) and Dex (←) Functions

In a note published in 1981 in the Technical Supplement of the I.P.Sharp Newsletter [8], R. Bernecky and K.E.Iverson maintain that "a statement separator introduces a new syntactic element into APL" and that "the use of statement separators forces an order of

reading and writing which is contrary to that used in APL expressions". They conclude: "It is the opinion of the authors that the benefits of statement separators might be better achieved in other ways, such as introduction of a LEV function which returns its left argument. LEV, being a normal APL function, would lead to no new rules."

But let's go back in history. In a paper presented at a Working Conference on Mechanical Language Structure, Princeton, New Jersey, August 1963, K.E. Iverson [9] discussed the four boolean "unary" functions: $X \alpha Y \leftrightarrow Y$, $X \omega Y \leftrightarrow Y$, $X \alpha Y \leftrightarrow X$, and $X \omega Y \leftrightarrow X$ (x and y being the logical negation of x and y respectively). These functions however, were not implemented, either in APL360, or in any of its successors.

Extending in 1972 P.S.Abrams' notion of a selection operation [10], A.Hassitt and L.E.Lyon used the defined function AFTER, giving as explicit result the left argument [11]:

```

      ▽ Z←A AFTER B
[1] Z←A
      ▽

```

Possibly inspired by the application of this defined function, MCM Computers Limited implemented in 1974 in its MCM/70 APL [12] the dyadic primitive function $A \circ B$, evaluating the right argument B and deferring the left argument A. The function serves to evaluate A, then ignore that result, and proceed to evaluate B [7].

As a side effect, the function may be used as a substitute for the diamond separator. The advantage is that - if this is an advantage - no new syntactic construct is needed. In contrast with the diamond separator however, the sequence of statements is executed from the rightmost statement to the leftmost, such that:

```

-----
STATEMENT1
STATEMENT2
STATEMENT3
-----

```

should be written

```

-----
STATEMENT3∘STATEMENT2∘STATEMENT1
-----

```

The one-line list of statements must be written and read from the right to the left, which is said to be the normal APL syntax, but if this is desirable with statement separation, is open to discussion.

In fact this dyadic primitive function $A \circ B$ is a forerunner of the dyadic lev and dex functions, the names being derived from the latin words "laevus" and "dexter". The dyadic function lev $A \circ B$ returns the left argument A and the dyadic function dex $A \circ B$ returns the right argument B (lev and dex "commute"). Both functions were made available for SHARP APL (IPSA) in 1983 [13] and both functions are included in K.E. Iverson's "APL Dictionary" - being called "left" and "right" respectively [14].

Motivations for including these functions are given in the dictionary and at a talk, presented by K.E. Iverson at APL87 [15]. As an example of the use of dyadic lev as a substitute for the diamond separator, either of the following expressions:

$$N \leftarrow \rho R \quad \diamond \quad B \leftarrow N \rho 2 \quad \diamond \quad T \leftarrow B T 1 \times / B \quad \diamond \quad S \leftarrow (11 + N) \circ . = + / T \quad \diamond \quad P \leftarrow R \times . * T \quad \diamond \quad C \leftarrow S + . \times P \\ C \leftarrow S + . \times P \leftarrow P \leftarrow R \times . * T \leftarrow S \leftarrow (11 + N) \circ . = + / T \leftarrow T \leftarrow B T 1 \times / B \leftarrow N \rho 2 \leftarrow N \leftarrow \rho R$$

will determine the coefficients of a polynomial equivalent to a polynomial with roots r, but where the second expression may be simplified (? - note of the author of this paper) to:

$$C \leftarrow S + . \times P \leftarrow P \leftarrow R \times . * T \leftarrow S \leftarrow (11 + N) \circ . = + / T \leftarrow B T 1 \times / B \leftarrow N \rho 2 \leftarrow N \leftarrow \rho R$$

It is also shown how the lev function may be used to avoid parentheses, as for example in the expression:

$$S \star .5 \leftarrow S \leftarrow + / A \star 2 \quad (\text{or } S \leftarrow + / A \star 2 \quad \diamond \quad S \star .5 \text{ vs } (+ / A \star 2) \star .5 - \text{the author})$$

... which may be read "the square root of s, where s is the sum of the squares of a" (the symbol $\leftarrow$ being read as "where"). Whether such clever constructs avoiding the use of parentheses are really simplifications is doubtful. The author of this note for instance, is still asking himself "what's wrong with parentheses?" As a matter of fact, in a lot of cases, the use of parentheses - even when not needed - is shorter and/or improves readability!

In the meantime, IBM implementations of APL do not support the lev and dex functions. Only SHARP APL (IPSA) and APL90 (SYNC) support them. Surprisingly, both implementations support the diamond separator as well.

ISO APL and ISO Extended APL

ISO APL (DIS 8485) [16] does not include mixed output. In fact, this means that mixed output is a consistent extension. Indeed, a consistent extension is a facility not specified in the standard that, for a construct the standard specifies as producing an error, gives some effect other than signalling the specified error. This means too, that a conforming implementation may support mixed output. A conforming program however, shall not use consistent extensions, and as such may not use mixed output.

ISO APL (DIS 8485) [16] includes three optional facilities: the shared variable protocol, the statement separator facility, and trace and stop control. A conforming implementation may provide an optional facility, but if it does, the optional facility shall behave as specified by the standard. A conforming program may use an optional facility. The diamond separator, as described above, behaves as specified by the standard. Consequently, a conforming implementation may support and a conforming program may use this diamond separator.

In the meantime, the former ISO/TC97/SC22/WG3-APL, now ISO-IEC/JTC1/SC22/WG3-APL, is preparing the new standard Extended APL. At the La Hulpe Meeting (25-29 April 1988) [17], it was decided to make the diamond separator "a deprecated feature, with the intention of removing this facility from the standard in some future revision". On the other hand, at the London Meeting (1-4 October 1988) [18], a proposal of E.E. McDonnell [19] for including the dyadic functions lev and dex (and the function identity - the monadic form of dex) has been endorsed.

We don't discuss here the desirability of including the functions lev and dex in the new standard Extended APL. We only establish that more than twenty current implementations of APL support the diamond separator, while only two of them support the functions lev and dex. In addition, a lot of publications and software made available over the past ten to fifteen years make use of the diamond separator.

In a paper published in the SIGPLAN Notices, N.W.Rickert [20] proposes a very commendable set of guidelines for language standards committees:

1. Changes proposed by a standards committee should be evolutionary, not revolutionary.
2. The new standard must respect the philosophy and tradition of the language.
3. Changes in the language should reflect the needs of the users, rather than be changes imposed by the standards committee.
4. Standards must attempt to preserve the investment in existing software written in the language.

5. New standards should maintain as high a degree of compatibility as possible with previous standards.

Taking it all into account, we claim that ISO Extended APL should include the diamond statement separator.

References

- [1] A.D.Falkoff and K.E.Iverson;APL360 User's Manual; Form GH20-0683-0;IBM Corporation,Technical Publications Department,White Plains,New York; September 1969.
- [2] Arthur L.Anger;On Mixed Output; APL Quote Quad,Vol.2,No.3,23 September 1970,pp.18-19. See also APL Quote Quad: The Early Years; APL Press,Palo Alto,California,1982;pp.127-128.
- [3] Raymond P. Polivka and Sandra Pakin;APL:The Language and Its Usage; Prentice-Hall Series in Automatic Computation; Prentice-Hall,Eaglewood Cliffs,New Jersey, 1975;pp.248-250.
- [4] Paul Berry;SHARP APL Reference Manual; I.P.Sharp Associates Limited,Toronto,Ontario,Canada,March 1979;pp.48-49.
- [5] Jon McGrew;An Introduction to APL2; Form SH20-9229-0;IBM Corporation,Programming Publishing,San Jose,California,August 1984;pp.254-255.
- [6] H.A.Driscoll;A User's Guide to Enhancements in the APL*PLUS System; First Edition(+ Supplements);Scientific Time Sharing Corporation,Bethesda,Maryland,December 1974.
- [7] Garth H.Foster;What Lies Beyond the Branch Arrow?; APL75 Congress Proceedings,University of Pisa,Italy,1975;Edited by P.S.Abrams; Association for Computing Machinery,New York,New York,1975;pp.115-122.
- [8] Robert Bernecky and Kenneth E.Iverson;Statement Separators; I.P.Sharp Newsletter,Vol.9,No.5,September/October 1981 - Technical Supplement 34,pp.T4-T5.
- [9] Kenneth E Iverson;Formalism in Programming Languages; Communications of the ACM,Vol.7,No.2,February 1964,pp.80-88 See also:A Source Book in APL; APL Press,Palo Alto,California,1981;pp.17-25.

- [10] Philip S Abrams; An APL Machine;
SU-SEL-70-017; Stanford University; Stanford Electronics Laboratories, Digital
Systems Laboratory, Stanford, California, February 1970. Also available from
the US Department of Commerce as Document AD-706-741.
- [11] A. Hassitt and L.E. Lyon; Efficient Evaluation of Array Subscripts of Arrays;
IBM Journal of Research and Development, Vol.16, No.1, Jan 1973, pp.45-57.
- [12] MCM/70 User's Guide;
Micro Computer Machines, Willowdale, Ontario, Canada, 1974.
- [13] Language Extensions of May 1983; SHARP APL Technical Note - SATN 45;
I.P. Sharp Associates Limited, Toronto, Ontario, Canada, 2 May 1983.
- [14] Kenneth E. Iverson; A Dictionary of APL; Publ Code - 0402 8703 E 3; I.P. Sharp
Associates Limited, Toronto, Ontario, Canada, March 1987 (45 pages).
See also: APL Quote Quad, Vol.18, No.1, September 1987, pp.5-40.
- [15] Kenneth E. Iverson; APL87; APL87 Conference Proceedings,
Dallas, Texas, 10-14 May 1987; Edited by John Aliburton; APL Quote
Quad, Vol.17, No.4, June 1987, pp.64-68
- [16] Second Draft Proposal ISO/DP8485-Programming Language APL;
Document ISO/TC97/SC22/NO61 - May 1985 (Prepared by ISO/TC97/SC22/WG3
- Edited by L.A. Morrow); ISO/TC97/SC22 Secretariat - Canada SCC; Circulated
for voting by 15 July 1985. Reprint: BACUS Monograph 85001; Belgian APL-
CAM Users Society BACUS, Duffel, Belgium, 1 November 1985.
- [17] Draft Minutes of ISO-APL Working Group Meeting held at IBM IEC, La Hulpe,
Belgium, Monday-Friday, April 25-29, 1988;
Document ISO-IEC/JTC1/SC22/WG3/N172; E.E. McDonnell; 13 May 1988.
See also: APL-CAM Journal, Vol.11, No.1, 16 January 1989, pp.4-12.
- [18] Draft Minutes of ISO-APL Working Group Meeting held at I.P. Sharp
Associates Ltd, 10 Dean Farrar Street, London, SW1H 0DX, England,
Saturday-Tuesday, October 1-4, 1988; E.E. McDonnell; 26 October 1988.
See also: APL-CAM Journal, Vol.11, No.1, 16 January 1989, pp.30-39.
- [19] Left, Right and Identity; Document ISO-IEC/JTC1/SC22/WG3/N202;
Prepared by E.E. McDonnell; 17 August 1988; Presented at the London ISO-APL
Working Group Meeting, October 1988.
- [20] Neil W. Rickert; The role of the language standards committee;
SIGPLAN Notices, Vol.23, No.4, April 1988, pp.51-55
See also: Document ISO-IEC/JTC1/SC22/WG3/N174.

Appendix 1

List of APL Implementations versus Mixed Output

APL\360	+ International Business Machines IBM	68
APLSV	+ International Business Machines IBM	73
VS APL	+ International Business Machines IBM	76
5100 APL	+ International Business Machines IBM	75
5100/5120 APL	+ International Business Machines IBM	77
APL2 IUP	- International Business Machines IBM	82
APL2 PP	- International Business Machines IBM	84
APL PC/PS	- International Business Machines IBM	83
APL2 PC/PS	- International Business Machines IBM	88
Xerox APL	+ Atkins Computing Services Ltd	76
MAINSTREAM APL	+ Boeing Computer Services BCS	77
APL/700	+ Burroughs - Unisys	73
APLB	- Burroughs - Unisys	85
APL/66	+ Cii Honeywell Bull	75
APL-64	- Cii Honeywell Bull	80
APL/CSTS	+ Computer Sciences Corporation	80
APL470	+ Computer Usage Company	81
APL*CYBER	+ Control Data Corporation CDC	73
APL 2.0	+ Control Data Corporation CDC	76
NOS/VE APL	+ Control Data Corporation CDC	84
AOS/VS APL	- Data General Corporation DG	81
APL-11	+ Digital Equipment Corporation DEC	77
APL-Basic	+ Digital Equipment Corporation DEC	77
APLSF	+ Digital Equipment Corporation DEC	77
VAX APL	+ Digital Equipment Corporation DEC	82
Dyalog APL	- Dyadic Systems Limited	82
APL90	+ Ecole des Mines de Saint-Etienne-SYNC	84
APL90 Macintosh	+ Ecole des Mines de Saint-Etienne-SYNC	87
OS4000 APL	+ GEC Computers Limited	79
Harris APL	+ Harris Computers Systems	78
APL\3000	- Hewlett Packard HP	76
I-APL	- I-APL Limited	87
2900 APL	- International Computers Limited ICL	79
SHARP APL	+ I.P.Sharp Associates Limited IPSA	69
SHARP APL +	+ I.P.Sharp Associates Limited IPSA	81
APL/LW	+ MIPS Software Development	80
APL/LWSF	+ MIPS Software Development	80

APL*MYRIADE	- Olivier-Ouellet-Morin Incorporated	81
PortaAPL	+ Portable Software	83
PortaAPL/PC	+ Portable Software	83
APL16	+ SEMS	79
APL1	+ Siemens AG	77
VIZ::APL	- Sigma Accounting & Management Sciences Ltd	82
APL-NET	+ SLIGOS-BARTS	78
APL 1100	+ Sperry Univac - Unisys	77
APL*PLUS	+ Scientific Time Sharing Corporation -STSC	69
APL*PLUS/PC et al	- Scientific Time Sharing Corporation -STSC	69
APL.68000	+ The Computer Company - MicroAPL Ltd	81
TIS-APL	+ Telecompute Integrated Systems TIS	80
Tiivis APL	+ TMT-Team Oy	81
Tymshare APL	+ Tymcom Program Products	79
APL/Z80	+ Vanguard Systems Corporation	79
APL/V80	+ Vanguard Systems Corporation	79
Waterloo microAPL	- Waterloo Computing Systems Ltd	81
WATCOM APL	- WATCOM Products Incorporated	83

Listed are:

1. The name of the APL implementation
2. Whether Mixed output is implemented (+) or not (-)
3. The name of the vendor
4. The year of the 1st release of the implementation.

Appendix 2

List of Implementations of the Diamond Separator

APL/66	Cii Honeywell Bull	77
APL-64	Cii Honeywell Bull	80
APL 2.0	Control Data Corporation CDC	76
AOS/VS APL	Data General Corporation DG	81
VAX APL	Digital Equipment Corporation DEC	82
Dyalog APL	Dyadic Systems Limited	82
APL90	Ecole des Mines de Saint-Etienne-SYNC	84
APL90 Macintosh	Ecole des Mines de Saint-Etienne-SYNC	87
OS4000 APL	GEC Computers Limited	79
Harris APL	Harris Computers Systems	78
APL3000	Hewlett Packard HP	76
2900 APL	International Computers Limited ICL	79
SHARP APL	I.P.Sharp Associates Limited IPSA	78
SHARP APL +	I.P.Sharp Associates Limited IPSA	81
PortaAPL	Portable Software	83
PortaAPL/PC	Portable Software	83
APL1	Siemens AG	79
APL-NET	SLIGOS-BARTS	85
APL*PLUS	Scientific Time Sharing Corporation -STSC	74
APL*PLUS/PC et al	Scientific Time Sharing Corporation -STSC	82
APL.68000	The Computer Company - MicroAPL Ltd	81
TIS-APL	Telecompute Integrated Systems TIS	80
Tymshare APL	Tymcom Program Products	79
WATCOM APL	WATCOM Products Incorporated	83

Listed are:

1. The name of the APL implementation
2. The name of the vendor
3. The year of implementation of the diamond separator (as far as known to the author).

Appendix 3

Benchmark - Diamond Separator

In order to be able to compare memory allocation and CPU times when using the diamond statement separator, a benchmark has been carried out with six such defined functions. Each function performs the same twenty assignments, but dispersed over twenty, ten, five, four, two and one line respectively (cfr. display below). The benchmark was carried out with VAX APL Version 3.0, under VMS, on a VAX 8700 (DEC).

Memory allocation was measured by individually copying the defined functions from a library, and making each time the difference between workspace availability before and after the copy ($\square$ WA). CPU times were measured in 3 series of 1000 executions for each function and averaged. CPU times were gathered by using the VAX APL monitoring facility ($\square$ MONITOR). Results of the benchmark are given below.

Figures 1 and 2 illustrate that, for the defined functions explored, memory allocation and CPU times are linear functions of the number of lines. The slope of the linear functions being positive, it is shown - as stated in the text - that using the diamond statement separator may save memory space and time.

DF	N	20N	Byte	CPU Times (ms)		Rel T
SEP1	1	20	3072	9.042	(9.046)	1.000
SEP2	2	10	2792	8.681	(8.662)	0.957
SEP4	4	5	2652	8.458	(8.470)	0.936
SEP5	5	4	2624	8.418	(8.431)	0.932
SEP10	10	2	2568	8.360	(8.354)	0.923
SEP20	20	1	2540	8.321	(8.316)	0.919

Results: Memory Allocation (bytes) and CPU Times (ms)

▽ SEP1

- [1] A1+1
- [2] A2+2
- [3] A3+3
- [4] A4+4
- [5] A5+5
- [6] A6+6
- [7] A7+7
- [8] A8+8
- [9] A9+9
- [10] A10+10
- [11] A11+11
- [12] A12+12
- [13] A13+13
- [14] A14+14
- [15] A15+15
- [16] A16+16
- [17] A17+17
- [18] A18+18
- [19] A19+19
- [20] A20+20

▽

▽ SEP2

- [1] B1+1 ◊ B2+2
- [2] B3+3 ◊ B4+4
- [3] B5+5 ◊ B6+6
- [4] B7+7 ◊ B8+8
- [5] B9+9 ◊ B10+10
- [6] B11+11 ◊ B12+12
- [7] B13+13 ◊ B14+14
- [8] B15+15 ◊ B16+16
- [9] B17+17 ◊ B18+18
- [10] B19+19 ◊ B20+20

▽

▽ SEP4

- [1] C1+1 ◊ C2+2 ◊ C3+3 ◊ C4+4
- [2] C5+5 ◊ C6+6 ◊ C7+7 ◊ C8+8
- [3] C9+9 ◊ C10+10 ◊ C11+11 ◊ C12+12
- [4] C13+13 ◊ C14+14 ◊ C15+15 ◊ C16+16
- [5] C17+17 ◊ C18+18 ◊ C19+19 ◊ C20+20

▽

▽ SEP5

- [1] D1+1 ◊ D2+2 ◊ D3+3 ◊ D4+4 ◊ D5+5
- [2] D6+6 ◊ D7+7 ◊ D8+8 ◊ D9+9 ◊ D10+10
- [3] D11+11 ◊ D12+12 ◊ D13+13 ◊ D14+14 ◊ D15+15
- [4] D16+16 ◊ D17+17 ◊ D18+18 ◊ D19+19 ◊ D20+20

▽

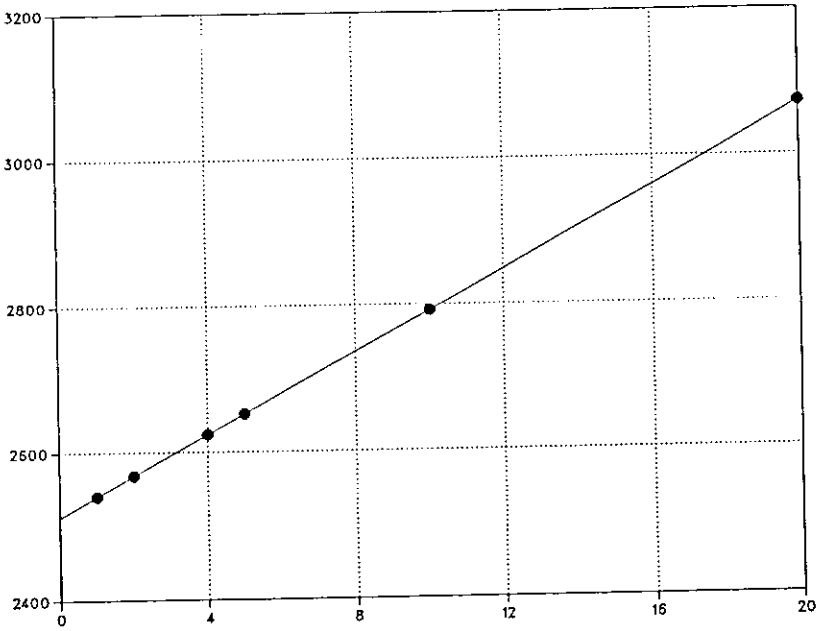


Figure 1: Memory Allocation in Bytes versus Number of Lines

```

▽ SEP10
[1] E1←1 ◇ E2←2 ◇ E3←3 ◇ E4←4 ◇ E5←5 ◇ E6←6 ◇ E7←7 ◇
    E8←8 ◇ E9←9 ◇ E10←10
[2] E11←11 ◇ E12←12 ◇ E13←13 ◇ E14←14 ◇ E15←15 ◇ E16←16 ◇
    E17←17 ◇ E18←18 ◇ E19←19 ◇ E20←20
▽
    
```

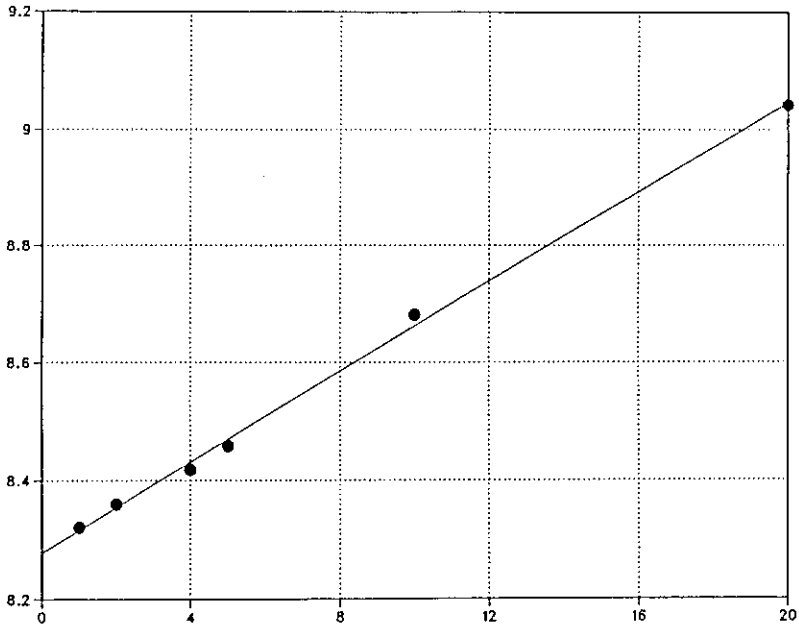
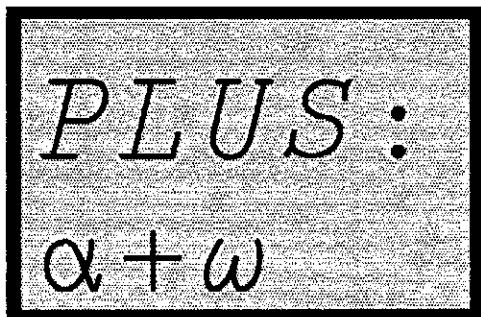


Figure 2: CPU time in milliseconds versus Number of Lines

```

V SEP20
[1] F1+1 ◊ F2+2 ◊ F3+3 ◊ F4+4 ◊ F5+5 ◊ F6+6 ◊ F7+7 ◊
    F8+8 ◊ F9+9 ◊ F10+10 ◊ F11+11 ◊ F12+12 ◊ F13+13 ◊
    F14+14 ◊ F15+15 ◊ F16+16 ◊ F17+17 ◊ F18+18 ◊ F19+19 ◊
    F20+20
V
```


... but the font isn't really designed to be blown up quite to this size! To be technical about it, it uses *stroked* rather than *filled* characters. These are much easier to handle (for the PostScript novice) and give quite a fair impression of the 2741-golfball original as long as you don't look too closely! Just for interest, here are the definitions of α and ω as used in the above font:



/ALPHA

```
{ newpath 1 setlinecap 1 setlinejoin
  40 setlinewidth
  504 448 moveto
  508 280 420 20 280 20 curveto
  56 20 56 448 280 448 curveto
  450 450 490 -240 574 130 curveto
  stroke
}
```

/OMEGA

```
{ newpath 1 setlinecap 1 setlinejoin
  40 setlinewidth
  200 450 moveto
  -100 0 220 -170 340 300 curveto
  220 -170 640 0 530 450 curveto
  stroke
}
```

Once you get the hang of Bezier Curves, you need astonishingly little code to define even quite complex shapes. Of course I prototyped my PostScript in APL, and quite a lot of the 'boilerplate' in the above definition was generated automatically.

The things I would most appreciate comment on:

- how closely should I stick to the 2741 original? I have tried to ensure that overstrikes really are true compound characters, but you will notice that this leads to a slightly asymmetric Φ , and that the stile only just penetrates the top of the delta in Φ . I have made τ a little more curly than standard, and have reshaped the leg of the R . Also I cheated on π which is not quite a true overstrike.
- the lower-case letters. The IPSA font (see page 89) seems to me to be totally inappropriate; I have tried for something much more like IBM's font as used in the APL2 Reference Manual ... basically Courier-Oblique sized to match the APL caps. Comments please!

My target is to run the January 1990 issue on an HP Deskjet+ using GoScript. Incidentally, this text is set in *Palatino*, rather than the standard Times-Roman. Did you notice the difference?!

British APL Association Software Library: Order Form

To: Alison Chatterton
9 Oak Grove,
Hertford,
Herts SG13 8AT, ENGLAND

Please supply the disks circled below:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53
54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71
72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89
90	91	92	93	94	95	96	97	98	99	100							

Total number of disks (please check): _____

at 2 Pounds per disk: BAA Member's rate _____
OR at 3 Pounds per disk: Non-member's rate _____

I want to join the BAA, and enclose
the 10 Pounds annual membership fee: _____

Postage and handling: 1.00

Add 2 Pounds for orders outside the UK: _____

Total order - remittance included: _____

Your signature: _____

Your name: _____

Full address: _____

Please make cheques payable to the British APL Association. Payment can be made in \$ - pay 1.5\$ (US) for each £. All orders must include payment - allow 30 days for delivery.

Software is accepted by the British APL Association on good faith and we do not vouch for or make any claim regarding donated software. The BAA shall not be held responsible for damage caused by the use of library software, and shall not be liable in any way in the event that submitted material proves to be the subject of third party copyright. Your signature above accordingly indemnifies the BAA from all liability.

Index to Advertisers

APL People	16
British APL Association	110
Brookteam Recruitment	112
Cocking and Drury	10
Dyadic Systems Ltd	4
HMW Computing	6
MicroAPL	58
Readers Digest	2

All queries regarding advertising in VECTOR should be made to Alison Chatterton, at the address on the inside back cover.

BRITISH APL ASSOCIATION Membership Application Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____
Department: _____
Organisation: _____
Address Line 1: _____
Address Line 2: _____
Address Line 3: _____
Post or zip code: _____
Country: _____
Telephone Number: _____

Membership category (please circle):89/90

Non-voting student membership (UK only) £5
UK private membership £10
Overseas private membership £18
Airmail supplement (not needed for Europe) £8
Corporate membership £85
Corporate membership overseas £140
Sustaining membership £360

For student applicants:

Name of course: _____
Name and title of supervisor: _____
Signature of supervisor: _____

If you would like to pay by credit card ...

Card no (Access/Visa): _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque "The British APL Association", or you may quote your Access or Visa number. Please send the completed form to:

BAA Administration,
Alison Chatterton, 9 Oak Grove,
HERTFORD SG13 8AT
England

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1989/90 Committee

Chairman:	Peter Donnelly 0420-87024	Dyadic Systems Ltd., Park House, The High Street, ALTON, Hants GU34 1EN
Secretary:	Graham Parkhouse 0483-571281 (x2379)	Dept. of Mech. Eng., University of Surrey, GUILDFORD GU2 5XH.
Treasurer:	John Sullivan 01 462-3628	24 Mounthurst Road, Hayes, Kent BR2 7QN
Journal Editor:	Adrian Smith 04393-385	Brook House, Gilling East, YORK YO6 4JJ
Activities:	Dr Peter Branson 0737-242950	Oaklands Cottage, Wray Common, Reigate, Surrey RH2 0LE
Education:	Dr Alan Sykes 0792-295296	Dept of Management Science University College of Swansea, Singleton Park, SWANSEA SA2 8PP
Publicity:	Jonathan Martin 01 562-5697	(S499) British Airways, PO Box 10, Heathrow Airport, HOUNSLOW, Middlesex TW6 2JA
Technical:	David Eastwood 01-922 8866	MicroAPL Ltd South Bank Technopark, 90 London Road, LONDON SE1 6LN
Recruitment:	Jill Moss 0225-462602	APL People Ltd, The Old Malthouse Clarence St., BATH, Avon BA1 5NS
Projects:	John Searle 01 948-6737	13A Mount Ararat Road, Richmond, Surrey TW10 6PQ
Administration:	Alison Chatterton 0992-552489	9, Oak Grove, HERTFORD, SG13 8AT

Journal Working Group

Jonathan Barman	048839-575
Anthony Camacho	0727-60130
Cathy Dargue	0895-31313
Adrian Smith	04393-385
David Ziemann	01-436 9481

Typeset using MS Word 4.0 on Xerox 3700 (Covers on Apple Laserwriter)

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaux and on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People
The Old Malthouse
Clarence St, BATH, BA1 5NS
Tel:0225-462602

Cocking & Drury Ltd
180 Tottenham Court Rd
LONDON, W1P 9LE
Tel:01-436 9481

HMW Computing Ltd
Hamilton House,
1 Temple Avenue,
LONDON, EC4Y 0HA
Tel:01-353 4212
Fax:01-353 3325

Inner Product Ltd
Eagle House
73 Clapham Common Southside
LONDON SW4 9DG
Tel:01-673-3354

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel:01-922 8866

Peter Cynax Systems
213 Goldhurst Terrace
LONDON NW6 3ER
Tel:01-624-7013
Mobile:0860-377963

Intelligent Programs Ltd
9 Gun Wharf, 130 Wapping High St,
LONDON, E1 9NH
Tel:01-265 1120

Dyadic Systems Ltd
Park House, The High Street
ALTON, Hants
Tel:0420-87024

APL Impetus Ltd
Rusper, Sandy Lane
Ivy Hatch, SEVENOAKS
Kent, TN15 0PD
Tel:0732-885126

MetaTechnics Systems Ltd
356 West Barnes Lane
Molsour Park, Surrey
Tel:01-949-6238
Fax:01-336-1062

I.P. Sharp Associates
1-4 Singer St.
LONDON EC2A 4BQ
Tel:01-867 1166
Fax:01-867 9792



The British Computer Society, 13 Mansfield Street, London W1M 0BD.